---

id: pqart-source-docs-secondary-development-pq-api说明文档 title: PQ API说明文档 status: active tags: [PQArt, 二次开发, API] source: pq-skill/references/source-docs/secondary-development/PQ API说明文档.md updatedAt: 2026-05-09T10:21:40.777Z

PQ API说明文档

文档版本：V3.13

修订时间：2025.9.15

产权说明

注意：此处包含的所有信息均为华航唯实机器人科技股份有限公司的财产。未经华航唯实机器人科技股份有限公司的事先书面许可，本出版物的任何部分（无论是硬拷贝还是电子形式）均不得复制、存储在检索系统中，或以任何形式或通过任何方式如电子、机械、影印、录音、或以其他方式传输。请注意，即使未与包含最终用户许可协议的软件一起分发，本指南中的内容仍受版权法保护。PQ和PQArt的标志是华航唯实机器人科技股份有限公司的注册商标。所有其他商标均为其各自所有者的财产。

本出版物和此处的信息按原样提供，仅供参考，如有更改，恕不另行通知，且不应被视为华航唯实公司的承诺。华航唯实公司明确声明对可能出现在本指南所载信息中的任何错误或不准确之处不承担任何责任或义务，对本文件不作任何（明示、暗示或法定）保证，并明确表示不承担任何关于适销性、适用于特定目的和不侵犯第三方权利的保证。

更新记录

版本编号	版本日期	PQ Kit支持版本	更新说明
V3.3	2023-07-25	9.1.0.6105及以上	1.7-1.13、6.2接口变更，新增6.8，7.1（新增导入场景）
V3.4	2023-08-01	9.1.0.6118及以上	5.2功能异常修正

版本编号	版本日期	PQ Kit支持版本	更新说明
V3.5	2023-08-20	9.1.0.6150及以上	新增1.21-1.23、2.6-2.9、3.5-3.8、4.2-4.4、5.25
V3.6	2023-08-31	9.1.0.6159及以上	新增5.26、5.27、7.3.1
V3.7	2023-09-07	9.2.0.6177及以上	新增1.24、2.10、4.4、4.5、5.17、5.28
V3.8	2023-11-22	9.2.0.6267及以上	新增1.26、1.27、1.28、1.29、6.12
V3.9	2023-12-14	9.2.0.6309及以上	新增7.6、第8章
V3.10	2024-8-22	10.0.0.6698及以上	新增1.36、1.37、7.5.2、7.5.3
V3.11	2024-12-22	10.0.0.6890及以上	新增1.44，1.45，1.46，6.10
V3.12	2025-4-30	11.0.0.7168及以上	新增3.20，3.23
V3.13	2025-9-15	11.1.0.7440及以上	新增1.56，2.16，2.17，修改5.35，5.42，6.4，9.5，9.6，9.7
V3.14	2025-12-12	11.2.0.7685及以上	新增1.59，2.19，6.30
V3.15	2026-1-1	11.2.0.7721及以上	新增5.1.2
V3.16	2026-3-20	11.2.0.7731及以上	新增2.20，2.21，3.32
V3.17	2026-4-9	11.2.0.7785及以上	新增5.47

文档

PQ执行模块

PQKit提供了一系列封装好的模块供用户调用,如生成轨迹、启动编译、启动仿真等等。通过调用Doc_start_module接口传入模块ID即可调用该模块。常用模块ID可参考7.7章节。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
SDK:10.0.0.7068	
Parameters：	
i_whModuleID	模块ID，如进入仿真 传"RO_CMD_SIMULATE"

C++参考代码段

```
wchar_t whMoudle[] = L"RO_CMD_SIMULATE";
m_ptrKit->Doc_start_module(whMoudle);
```

结束模块

通过调用Doc_end_module接口传入模块ID即可结束该模块。

HRESULT Doc_end_module (const WCHAR* i_whModuleID)	
SDK:10.0.0.7068	
Parameters：	
i_whModuleID	模块ID，如进入仿真后要退出仿真 传"RO_CMD_SIMULATE"

C++参考代码段

```
wchar_t whMoudle[] = L"RO_CMD_SIMULATE";
m_ptrKit->Doc_end_module(whMoudle);
```

打开robx工程

调用接口Doc_open_robx传入欲打开的robx工程文件绝对路径。

HRESULT Doc_open_robx(const WCHAR* i_whRobxPath)	
SDK:9.2.0.6472	
Parameters：	
i_whRobxPath	robx文件绝对路径

C++参考代码段

```
QString fileName = QFileDialog::getOpenFileName(this, tr("open robx file"), "", tr("Robx files(*.ro
if (fileName.isEmpty()) {
return;
}
// std::wstring wsFilePath = fileName.toStdWString();
m_ptrKit->Doc_open_robx((LPWSTR)(wsFilePath.c_str()));
```

保存robx工程

调用Doc_save_robx保存当前打开的robx工程文件。

HRESULT Doc_save_robx()

SDK:9.2.0.6472

另存为文件

调用接口Doc_saveas_robx传入指定另存为的绝对路径。

HRESULT Doc_saveas_robx(const WCHAR* i_whRobxPath)	
SDK:9.2.0.6472	

HRESULT Doc_saveas_robx(const WCHAR* i_whRobxPath)	
Parameters：	
i_whRobxPath	文件另存为的绝对路径

C++参考代码段

```
QString fileName = QFileDialog::getSaveFileName(this, tr("open robx file"), "", tr("Robx files(*.rc
if (fileName.isEmpty()) {
return;
}
// std::wstring wsFilePath = fileName.toStdWString();
m_ptrKit->Doc_saveas_robx((LPWSTR)(wsFilePath.c_str()));
```

关闭robx工程

调用pq_CloseDocument接口传入欲关闭的robx工程文件名称，即可关闭该文档。调用Doc_get_name可以获取当前打开的robx工程名称。

HRESULT pq_CloseDocument(BSTR i_bsDocName)	
SDK:9.2.0.6472	
Parameters：	
i_bsDocName	已打开的robx工程文件名，不包含.robx后缀

C++参考代码段

```
CComBSTR strFilePath = "";
m_ptrKit->Doc_get_name(&strFilePath);
m_ptrKit->pq_CloseDocument(strFilePath);
```

设置文档背景色

可以自上而下即从From到To指定绘图区为渐变背景色，也可指定绘图区单一背景色即From和To传一样值即可。

HRESULT PQAPIPutViewBGC(DOUBLE i_dFromR, DOUBLE i_dFromG, DOUBLE i_dFromB, DOUBLE i_dToR, DOUBLE i_dToG, DOUBLE i_dToB)	
SDK:9.2.0.6420	
Parameters：	
i_dFromR	From color R value
i_dFromG	From color G value
i_dFromB	From color B value
i_dToR	To color R value
i_dToG	To color G value
i_dToB	To color B value

C++参考代码段

```
DOUBLE i_dFromR = 139.0;
DOUBLE i_dFromG = 58.0;
DOUBLE i_dFromB = 58.0;
DOUBLE i_dToR = 205.0;
DOUBLE i_dToG = 92.0;
DOUBLE i_dToB = 92.0;
m_ptrKit->PQAPIPutViewBGC(i_dFromR, i_dFromG, i_dFromB, i_dToR, i_dToG, i_dToB);
```

获取对象ID

PQKit中每一个对象都有一个唯一的ID标识与其对应，PQKit与调用者之间的数据交互都需要通过ID来维系。

PQKit中有多种方式可以获取到对象的ID。静态获取有两个api来获取对象ID，它们都返回对象名称和ID的匹配对，需要调用者根据对象名称来匹配正确ID。动态获取参考章节1.29。

pq_GetAllDataObjectsByType返回的o_sNames是指定类型的多个对象名称中间添加“#”拼接的字符串，如“C型架地轨#变位机#ER8_1500#”，o_sIDs是指定类型的多个对象ID中间添加“#”拼接的字符串，如“536870913#536870914#536870915#”。相应的C型架地轨的ID为536870913，变位机的ID为536870914，以此类推ER8_1500的ID为536870915。

HRESULT pq_GetAllDataObjectsByType (PQDataType i_eObjType,BSTR* o_sNames,BSTR* o_sIDs)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQDataType最低支持版本)	
Parameters：	
i_eObjType	指定对象类型，枚举类型值详见9.1数据类型定义
o_sNames	所有指定类型对象的名字，以“#”分割
o_sIDs	所有指定类型对象的ID，以“#”分割

C++参考代码段

```
CComBSTR bsName = _T("");
CComBSTR bsID = _T("");
m_ptrKit->pq_GetAllDataObjectsByType(PQ_ROBOT, &bsName, &bsID);
```

Doc_get_obj_bytype是以数组形式分别返回指定类型的对象的名称和ID，名称和ID按序匹配，即Names[0]对应的ID是IDs[0]。

HRESULT Doc_get_obj_bytype (PQDataType i_eObjType, VARIANT* o_sNames, VARIANT* o_sIDs)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQDataType最低支持版本)	
Parameters：	
i_eObjType	指定对象类型，枚举类型值详见9.1 数据类型定义
o_sNames	所有指定类型对象的名称集合
o_sIDs	所有指定类型对象的ID集合

C++参考代码段，获取一个指定类型和名称的对象ID

```
void GetObjIDByName(PQDataType i_eType, std::wstring i_wsName,
ULONG &o_uID) {
CComVariant vNamePara;
CComVariant vIDPara;
m_ptrKit->Doc_get_obj_bytype(i_eType, &vNamePara, &vIDPara);
if (NULL == vNamePara.parray || NULL == vIDPara.parray) {
return;
}
// BSTR* bufName = nullptr;
long lenName = vNamePara.parray->rgsabound[0].cElements;
SafeArrayAccessData(vNamePara.parray, (void**)&bufName);
int nTarIndex = 0;
if (!i_wsName.empty()) {
for (int i = 0; i < lenName; i++) {
if (0 == i_wsName.compare(bufName[i])) { nTarIndex = i;
break; } } }
SafeArrayUnaccessData(vNamePara.parray); // ULONG* bufID = nullptr; long lenID = vIDPara.parray->rg
SafeArrayAccessData(vIDPara.parray, (void**)&bufID);
o_uID = bufID[nTarIndex];
SafeArrayUnaccessData(vIDPara.parray);
}
```

获取/设置对象名称

<1>根据对象ID获取名称

HRESULT Doc_get_obj_name(ULONG i_ulID, BSTR *o_bsName)	
SDK:10.0.0.7068	
Parameters：	
i_ulID	欲获取名称的对象ID
o_bsName	接收名称的字符串

C++参考代码段

```
ULONG ulRobotID = /**填入具体数值**/;
CComBSTR bsName;
m_ptrKit->Doc_get_obj_name(ulRobotID, &bsName);
```

<2>设置指定对象名称

HRESULT Doc_set_obj_name(ULONG i_ulID, BSTR i_bsName)	
SDK:10.0.0.7068	
Parameters：	
i_ulID	欲设置名称的对象ID
i_bsName	设定的新名称

C++参考代码段

```
ULONG ulRobotID = /**填入具体数值**/;
CComBSTR bsName = _T("ER8_1500_NewName");
m_ptrKit->Doc_set_obj_name(ulRobotID, bsName);
m_ptrKit->Doc_update_view();
```

获取/设置对象关节角

<1>获取指定对象关节角

HRESULT Doc_get_obj_joints(ULONG i_ulID, INT *o_nJointCount, DOUBLE **o_dJoints)	
SDK:10.0.0.7068	
Parameters：	
i_ulID	欲获取关节角的对象ID
o_nJointCount	传出关节角数据个数
o_dJoints	传出关节角数据(弧度单位),需要调用PQAPIFreeArray释放内存

C++参考代码段

```
ULONG uID = /**填入具体数值**/;
int nCount = 0;
double* dJoints = nullptr;
m_ptrKit->Doc_get_obj_joints(uID, &nCount, &dJoints);
double dDisplayA[6] = {
0.0 }
;
for (int i = 0; i < 6; i++) { dDisplayA[i] = dJoints[i] * (180 / 3.14); }
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dJoints);
```

<2>设定指定对象关节角

HRESULT Doc_set_obj_joints(ULONG i_ulID, DOUBLE *i_dJoints, INT i_nJointsArraySize)	
SDK:10.0.0.7068	
Parameters：	
i_ulID	欲设置关节角的对象ID

HRESULT Doc_set_obj_joints(ULONG i_ulID, DOUBLE *i_dJoints, INT i_nJointsArraySize)	
i_dJoints	关节角数据数组(弧度单位)
i_nJointsArraySize	关节角数据数组大小

C++参考代码段

```
ULONG uID = /**填入具体数值**/;
double dJoints[6] = {
0.1,0.2,0.3,0.4,0.5,0.6}
;
m_ptrKit->Doc_set_obj_joints(uID, dJoints, 6);
```

获取/设置对象关节限位

获取指定对象关节限位

HRESULT Doc_get_obj_links(ULONG i_ulID, INT *o_nLinkCount, DOUBLE **o_dLinks)	
SDK:10.0.0.7068	
Parameter:	
i_ulID	欲获取关节限位的对象ID
o_nLinkCount	传出关节限位数据个数
o_dLinks	传出关节限位数据(度数单位)，按每个轴限位的最小值、最大值排序，需要调用PQAPIFreeArray释放

C++参考代码段

```
ULONG uID = /**填入具体数值**/;
int nCount = 0;
```

```
double* dLinks = nullptr;
m_ptrKit->Doc_get_obj_links(uID, &nCount, &dLinks);
```

设置指定对象关节限位

HRESULT Doc_set_obj_links(ULONG i_ulID, DOUBLE *i_dLinks, INT i_nLinksArraySize)	
SDK:10.0.0.7068	
Parameter:	
i_ulID	欲设置关节限位的对象ID
i_dLinks	关节限位数据数组(度数单位)，按每个轴限位的最大值、最小值排序
i_nLinksArraySize	关节限位数据数组大小

C++参考代码段

```
ULONG uID = uRobotID;
double dLinks[12] = {
90.0,-90.0,120.0,-95.0,360.0,-360.0,180.0,-180.0, 105.0,-105.0,180.0,-180.0}
;
m_ptrKit->Doc_set_obj_links(uID, dLinks, 12);
```

获取/设置对象速度

HRESULT Doc_get_obj_velocity(ULONG i_ulID, DOUBLE *o_dVelocity, DOUBLE *o_dRAD)	
SDK:9.2.0.6340	
Parameter:	
i_ulID	欲获取线速度、角速度的对象ID
o_dVelocity	线速度，单位mm/s

HRESULT Doc_get_obj_velocity(ULONG i_ulID, DOUBLE *o_dVelocity, DOUBLE *o_dRAD)	
o_dRAD	角速度，单位rad/s

获取指定机器人、工件对象在PQKit环境中仿真时的线速度、角速度。

C++参考代码段

```
ULONG uRobotID = /**填入具体数值**/;
double dVelocity = 0.0;
double dRAD = 0.0;
m_ptrKit->Doc_get_obj_velocity(uRobotID, &dVelocity, &dRAD);
```

设置指定机器人、工件对象在PQKit环境中仿真时的线速度、角速度。

HRESULT Doc_set_obj_velocity(ULONG i_ulID, DOUBLE i_dVelocity, DOUBLE i_dRAD)	
SDK:9.2.0.6340	
Parameter:	
i_ulID	欲设置线速度、角速度的对象ID
i_dVelocity	线速度，单位mm/s
i_dRAD	角速度，单位rad/s

C++参考代码段

```
ULONG uRobotID = /**填入具体数值**/;
double dVelocity = 123.0;
double dRAD = 1.450;
m_ptrKit->Doc_set_obj_velocity(uRobotID, dVelocity, dRAD);
```

获取/设置对象位姿

获取指定对象位姿

HRESULT Doc_get_obj_posture(ULONG i_ulID, PQPostureType i_ePostureType, INT *o_nPostureArraySize, ULONG i_ulCoorID, DOUBLE o_dPosture)**	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameters：	
i_ulID	欲获取位姿的对象ID
i_ePostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义
o_nPostureArraySize	返回位姿数据长度
i_ulCoorID	指定位姿参考坐标系，若为世界坐标系传0
o_dPosture	具体位姿数据，需要调用PQAPIFreeArray释放

C++参考代码段

```
ULONG uRobotID = /**填入具体数值**/;
PQPostureType ePostureType = EULERANGLEZYX;
double* dPosture = nullptr;
int nPostureCount = 0;
m_ptrKit->Doc_get_obj_posture(uRobotID, ePostureType, &nPostureCount,0, &dPosture);
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dPosture);
```

设置指定对象位姿

HRESULT Doc_set_obj_posture(ULONG i_ulID, DOUBLE *i_dPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType,ULONG i_ulCoorID)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	

HRESULT Doc_set_obj_posture(ULONG i_ulID, DOUBLE *i_dPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType,ULONG i_ulCoorID)	
Parameters：	
i_ulID	欲设置位姿的对象ID
i_dPosture	位姿数据
i_nPostureArraySize	位姿数据数组大小
i_ePostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义
i_ulCoorID	位姿参考坐标系，若为世界坐标系传0

C++参考代码段

```
ULONG uRobotID = /**填入具体数值**/;
PQPostureType ePostureType = EULERANGLEZYX;
double dPosture[6] = {
100.0,200.0,300.0,90.0,0.0,0.0}
;
int nPostureCount = 6;
m_ptrKit->Doc_set_obj_posture(uRobotID, dPosture, nPostureCount, ePostureType, 0);
```

设置对象颜色

设置指定对象显示的颜色。

HRESULT Doc_set_obj_color(ULONG i_ulID, DOUBLE i_dR, DOUBLE i_dG, DOUBLE i_dB)	
SDK:9.2.0.6340	
Parameters：	

HRESULT Doc_set_obj_color(ULONG i_ulID, DOUBLE i_dR, DOUBLE i_dG, DOUBLE i_dB)	
i_ulID	欲设置颜色的对象ID
i_dR	RGB-R值
i_dG	RGB-G值
i_dB	RGB-B值

C++参考代码段

```
ULONG uRobotID = /**填入具体数值**/;
double dR = 255.0;
double dG = 0.0;
double dB = 0.0;
m_ptrKit->Doc_set_obj_color(uRobotID,dR,dG,dB);
```

获取对象关节个数

获取对象关节个数

HRESULT Doc_get_obj_joint_count(ULONG i_ulID, INT *o_nCount)	
SDK:9.2.0.6340	
Parameter:	
i_ulID	欲获取关节个数对象ID
o_nCount	关节个数
## 矩阵转换	

PQKit提供两种矩阵转换接口，一是旋转矩阵转欧拉角或者四元数，二是欧拉角或四元数转旋转矩阵。

输入4*4矩阵，转换成指定格式的欧拉角或者四元数。

HRESULT PQAPITransPosition(DOUBLE i_dInputPosition[16], PQPostureType i_ePostureType, INT* o_nTargetPositionSize, DOUBLE o_dTargetPosition)**	
SDK:9.2.0.6340(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameters：	
i_dInputPosition	4*4矩阵，定长，大小为16，按行排序
i_ePostureType	指定格式，如欧拉角ZYX、四元数，详见9.2 位姿描述定义
o_nTargetPositionSize	转换后的姿态数据大小
o_dTargetPosition	转换后的姿态数据，例如姿态为欧拉角ZYX，则传出的XYZABC中ABC分别为绕Z绕Y绕X的度数 姿态数据需要调用PQAPIFreeArray释放

C++参考代码段

```
double dInputMatrix[16] = {
0.81915204428899191,-0.57357643635104594,0.0000000000000000,123.0000000000000000, 0.573576436351045
;
PQPostureType ePostureType = EULERANGLEZYX;
int nCount = 0;
double* dEuler = nullptr;
m_ptrKit->PQAPITransPosition(dInputMatrix, ePostureType,&nCount, &dEuler);
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dEuler);
```

输入指定格式位姿，转换成旋转矩阵

HRESULT Math_trans_posture_to_rotationmatrix(DOUBLE* i_dPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, DOUBLE o_dTranslation[16])	
SDK:9.2.0.6340(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameters：	
i_dPosture	位姿数据，例如姿态为欧拉角ZYX，则传入的XYZABC中ABC分别为绕Z绕Y绕X的度数
i_nPostureArraySize	i_dPosture数组大小
i_ePostureType	指定格式，如欧拉角ZYX、四元数，详见9.2位姿描述定义
o_dTranslation	4*4矩阵按行排序输出

C++参考代码段

```
double dEulerZYX[6] = {
0.0,0.0,0.0,11.0,22.0,33.0 }
;
double dMatrix[16] = {
0.0}
;
m_ptrKit->Math_trans_posture_to_rotationmatrix(dEulerZYX,6, EULERANGLEZYX, dMatrix);
```

设置当前激活设备

在PQKit环境中只有机器人、导轨、变位机对象才能被设置成当前激活设备。

HRESULT PQAPISetActiveEngine(ULONG i_uEngineID)	
SDK:9.2.0.6340	

HRESULT PQAPISetActiveEngine(ULONG i_uEngineID)	
Parameters：	
i_uEngineID	指定激活设备的ID
## 获取当前激活设备	

获取当前工程中激活的设备ID。

HRESULT PQAPIGetActiveEngine(ULONG* o_uEngineID)	
SDK:9.2.0.6340	
Parameters：	
o_uEngineID	获取当前激活设备的ID
## 获取对象姿态表达方式	

获取机构定义时设置的姿态表达方式。

HRESULT PQAPIGetObjPositionAttitudeType(ULONG i_ulID, PQPostureType* o_ePostureType)	
SDK:9.2.0.6340(函数本体最低支持版本) 11.1.0.7310 (PQPostureType最低支持版本)	
Parameters：	
i_ulID	指定对象的ID
o_ePostureType	返回指定的对象的姿态表达方式，具体值详见9.2 位姿描述定义
## 设置对象可见性	

设置指定对象隐藏与显示。

HRESULT Doc_set_obj_visibility(ULONG i_ulObjID, BOOL i_bShow)	
SDK:9.2.0.6340	
Parameters：	
i_ulObjID	指定对象的ID
i_bShow	是否显示可见
## 删除对象	

调用Doc_delete_obj接口可以删除指定ID的对象，删除对象需要构建欲删除对象的ID数组。删除支持单独删除和多选删除，即使只删除一个对象也需要构造一个只有一个对象ID的数组。

注：删除轨迹点请参考6.12。

HRESULT Doc_delete_obj(ULONG* i_ulObjIDArray, INT i_nObjIDArraySize)	
SDK:11.0.0.7314	
Parameters：	
i_ulObjIDArray	欲删除对象的id数组
i_nObjIDArraySize	欲删除对象的id数组的大小

C++参考代码段

```
//下面代码示例为删除场景中的第一个零件和第一个机器人
ULONG uRobotID = 0;
GetObjIDByName(PQ_ROBOT, _T(""), uRobotID);
// GetObjIDByName函数体请参考sample源码
ULONG uWPID = 0;
GetObjIDByName(PQ_WORKINGPART, _T(""), uWPID);
ULONG dObjID[2] = {
uRobotID,uWPID}
;
m_ptrKit->Doc_delete_obj(dObjID,2);
```

碰撞检测

Kit支持输入一对对象ID来检测该两物体是否发生碰撞，同时也支持传入两组多个物体进行碰撞检测。

单个物体（两个）碰撞检测。

HRESULT Doc_collide_obj_single(ULONG i_ulObjAID, ULONG i_ulObjBID, BOOL *o_bCollide)	
Parameters：	
i_ulObjAID	指定参与碰撞检测的物体A
i_ulObjBID	指定参与碰撞检测的物体B
o_bCollide	TRUE表示A、B相碰

C++参考代码段

```
void MainWindow::CollideObjSingle() {
ULONG uIDA=/* */;
ULONG uIDB = /* */;
long bCollide = 0;
m_ptrKit->Doc_collide_obj_single(uIDA, uIDB, &bCollide);
}
```

多个物体（两组）碰撞检测。

HRESULT Doc_collide_obj_multiple(ULONG *i_ulObjAIDs, INT i_nACount, ULONG *i_ulObjBIDs, INT i_nBCount, VARIANT *o_collideObjs)	
Parameters：	
i_ulObjAIDs	指定参与碰撞检测的物体组A

HRESULT Doc_collide_obj_multiple(ULONG *i_ulObjAIDs, INT i_nACount, ULONG *i_ulObjBIDs, INT i_nBCount, VARIANT *o_collideObjs)	
i_nACount	组A中对象个数
i_ulObjBIDs	指定参与碰撞检测的物体组B
i_nBCount	组B中对象个数
o_collideObjs	返回一个对象ID数组，为发生碰撞的对象ID集合，两两碰撞的对象ID依次排序。例如两组物体中A1与B1碰撞，A2与B2碰撞，则返回的数组内容为A1.ID，B1.ID，A2.ID，B2.ID

C++参考代码段

```
void MainWindow::CollideObjSingle() {
ULONG uIDA=/* */;
INT nCounta = 1;
ULONG uIDB = /* */;
INT nCountb = 1;
VARIANT vCollideObj;
vCollideObj.parray = NULL;
m_ptrKit->Doc_collide_obj_multiple(&uIDA,nCounta,&uIDB,nCountb, &vCollideObj);
}
```

地面显示/隐藏

通过pq_RunCommand执行RO_CMD_FLOOR命令，并通过varParam参数传入VARIANT_BOOL控制地面可见性。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
Parameters：	

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
bsCommandID	"RO_CMD_FLOOR"
wParam	0或NULL
lParam	0或NULL
bsParam	""
varParam	VARIANT_TRUE显示地面，VARIANT_FALSE 隐藏地面
lResult	0

C++参考代码段

```
void MainWindow::ShowFloor() {
CComVariant varparam=VARIANT_FALSE;
CComBSTR cmd = "RO_CMD_FLOOR";
CComBSTR bsparam = "";
long long result;
m_ptrKit->pq_RunCommand(cmd, NULL, NULL, bsparam, varparam, &result);
}
```

显示所有隐藏对象

通过pq_RunCommand执行RO_CMD_SHOW_ALL_HIDE_OBJ命令，来显示所有已经隐藏的实体对象。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
Parameters：	
i_whModuleID	模块ID，显示所有隐藏对象"RO_CMD_SHOW_ALL_HIDE_OBJ"

C++参考代码段

```
void MainWindow::ShowAllHideObj() {
CComBSTR cmd = "RO_CMD_SHOW_ALL_HIDE_OBJ";
m_ptrKit->Doc_start_module(cmd);
}
```

轻量化robx

通过pq_RunCommand执行RO_CMD_FILE_LITE命令，来轻量化robx工程文件，其中bsParam指定保存轻量化结果robx的绝对路径（例如C:\Users \Desktop\A.robx），varParam参数传入VARIANT_BOOL控制是否深度轻量化。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
Parameters：	
bsCommandID	"RO_CMD_FILE_LITE"
wParam	0或NULL
lParam	0或NULL
bsParam	保存轻量化结果robx的绝对路径
varParam	VARIANT_TRUE深度轻量化
lResult	0

C++参考代码段

```
void MainWindow::LightObj() {
CComBSTR cmd = "RO_CMD_FILE_LITE";
VARIANT varparam;
varparam.boolVal = VARIANT_TRUE;
CComBSTR bsparam = "C:\\Users \\Desktop\\A.robx";
long long result;
```

```
m_ptrKit->pq_RunCommand(cmd, NULL, NULL, bsparam, varparam, &result);
}
```

内存释放

PQKit/二次开发提供2个API用于释放在PQ内部创建的内存数据。

PQAPIFree用于释放普通类型数据

HRESULT PQAPIFree(LONG_PTR *i_ptrData)	
Parameters：	
i_ptrData	需要释放的数据指针

C++参考代码段

```
void MainWindow::GetObjLinks() {
ULONG uID = /**填入具体数值**/;
INT nLinkcount = 0;
double* dLinks;
m_ptrKit->Doc_get_obj_links(uID, &nLinkcount, &dLinks);
m_ptrKit->PQAPIFree((LONG_PTR*)dLinks);
}
```

PQAPIFreeArray用于释放数组类型数据

HRESULT PQAPIFreeArray(LONG_PTR *i_ptrDataArray)	
Parameters：	
i_ptrDataArray	需要释放的数组数据指针

C++示例：

```
void func () {
WCHAR *o_ChTcpName;
INT *o_ITcpLength = 0;
theApp.m_spPQData->Tool_get_tcp(ulToolId, &o_iTcpCount, &o_ChTcpName, &o_ITcpLength);
theApp.m_spPQData->PQAPIFreeArray((LONG_PTR*)o_ChTcpName);
```

```
heApp.m_spPQData->PQAPIFreeArray((LONG_PTR*) o_ITcpLength);
}
```

C#示例：

```
void func () {
IntPtr ptrPosture = IntPtr.Zero;
int nCount = 0;
m_ptrKit.Point_get_posture_batch(uPathID, 0, 82, 1, 0, out nCount, ref ptrPosture);
double[] dPostures = new double[nCount];
Marshal.Copy(ptrPosture, dPostures, 0, nCount);
m_ptrKit.PQAPIFreeArray((long)ptrPosture);
}
```

获取坐标系位姿

获取指定坐标系的位姿数据。

HRESULT Doc_get_coordinate_posture(ULONG i_ulCoorID, PQPostureType i_ePostureType, INT *o_nPostureArraySize, DOUBLE **o_dPosture，ULONG i_ulRobCoorID)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameters：	
i_ulCoorID	指定坐标系的ID
i_ePostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义
o_nPostureArraySize	返回坐标系位姿数据的大小
o_dPosture	坐标系的位姿数据，需要调用PQAPIFreeArray释放

HRESULT Doc_get_coordinate_posture(ULONG i_ulCoorID, PQPostureType i_ePostureType, INT *o_nPostureArraySize, DOUBLE **o_dPosture，ULONG i_ulRobCoorID)	
i_ulRobCoorID	传入相对坐标系ID，若获取坐标系基于世界坐标系下位姿，该值传入0

C++示例：

```
ULONG ulCoorID = 0;
GetObjIDByName(PQ_COORD, _T("USER-1"), ulCoorID);
// int nCount = 0;
double* dPosture = nullptr;
m_ptrKit->Doc_get_coordinate_posture(ulCoorID,EULERANGLEXYZ,&nCount,&dPosture,0);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) dPosture);
```

自定义参数

PQArt中有机器人、轨迹、轨迹点三种数据对象支持设置、获取自定义参数。自定义参数为多组key:value的字符串对，key表示自定义参数的名称，value表示自定义参数的值。

调用Doc_set_custom_parameter设置自定义参数，每调用一次给指定对象添加一组自定义参数，多次调用即可添加多组。

HRESULT Doc_set_custom_parameter(ULONG i_ulObjID, const WCHAR* i_chParaName, const WCHAR* i_chParaValue)	
Parameters：	
i_ulObjID	机器人或者轨迹或者轨迹点的ID
i_chParaName	自定义参数名称
i_chParaValue	自定义参数值

C++示例：

```
void MainWindow::SetCustomParameter() {
ULONG uID = /* */;
CComBSTR chparaName = "";
CComBSTR chparaValue = "";
m_ptrKit->Doc_set_custom_parameter(uID, chparaName, chparaValue);
}
```

调用Doc_get_custom_parameter获取指定对象自定义参数。

HRESULT Doc_get_custom_parameter(ULONG i_ulObjID, INT *o_nParaCount, WCHAR **o_chParaNames, INT **o_nParaNameLengthArray, WCHAR **o_chParaValues, INT **o_nParaValueLengthArray)	
Parameters：	
i_ulObjID	机器人或者轨迹或者轨迹点的ID
o_nParaCount	指定对象下自定义参数的个数
o_chParaNames	指定对象下所有自定义参数名称合并的字符串，字符串需要调用PQAPIFreeArray释放
o_nParaNameLengthArray	每一个自定义参数名称的长度数组，数组需要调用PQAPIFreeArray释放
o_chParaValues	指定对象下所有自定义参数值合并的字符串，字符串需要调用PQAPIFreeArray释放
o_nParaValueLengthArray	每一个自定义参数值的长度数组，数组需要调用PQAPIFreeArray释放

例如，某条轨迹下有2个自定义参数，值为"20"的 "Accelerated Speed"和值为 "ArcWired"的 "PathType"。则o_nParaCount值为2，o_chParaNames为"Accelerated SpeedPathType"，o_nParaNameLengthArray为[17，8]，o_chParaValues为"20ArcWired"，o_nParaValueLengthArray为[2，8]。双指针传参均在PQArt内部分配内存，需要调用PQAPIFreeArray释放。

C++示例：

```
HRESULT CRoHelp::GetCustomParameter() {
ULONG uID = /* */;
INT nParaCount = 0;
WCHAR* chParaName= nullptr;
INT* nParaNameLengthArr= nullptr;
WCHAR* chParaValue= nullptr;
INT* nParaValueLengthArr= nullptr;
m_ptrKit->Doc_get_custom_parameter(uID, &nParaCount, &chParaName, &nParaNameLengthArr, &chParaValue
m_ptrKit ->PQAPIFree((LONG_PTR*)chParaName);
m_ptrKit ->PQAPIFree((LONG_PTR*)nParaNameLengthArr);
m_ptrKit ->PQAPIFree((LONG_PTR*)chParaValue);
m_ptrKit ->PQAPIFree((LONG_PTR*)nParaValueLengthArr);
}
```

旋转物体(绕自身坐标系)

在PQArt中选中物体后，弹出三维球，即为该物体的自身坐标系。可调用Doc_obj_rotate_local_X、Doc_obj_rotate_local_Y、Doc_obj_rotate_local_Z来使指定物体绕自身坐标系指定轴旋转一定角度。

HRESULT Doc_obj_rotate_local_X(ULONG i_ulObjID, DOUBLE i_dRad);	
Parameters：	
i_ulObjID	指定旋转对象
i_dRad	旋转的角度（弧度值）

C++示例：

```
void MainWindow::ObjRotateLocalX() {
ULONG uID = /* */;
double dRad = 2.3;
m_ptrKit->Doc_obj_rotate_local_X(uID, dRad);
}
```

动态碰撞检测(仿真中)

通过Doc_add_sim_collide_data接口可设置仿真过程中哪两组物体进行碰撞检测（设置后需要在仿真面板中勾选碰撞检测项）。

HRESULT Doc_add_sim_collide_data(ULONG* i_vCollisionCheckIDS, INT i_nCheckIDSCount, ULONG* i_vCollisionWithIDS, INT i_nWithIDSCount)	
Parameters：	
vCollisionCheckIDS	第一组碰撞对物体的ID
i_nCheckIDSCount	第一组碰撞对个数
i_vCollisionWithIDS	第二组碰撞对物体的ID
i_nWithIDSCount	第二组碰撞对个数

C++示例：

```
void MainWindow::AddSimCollideData() {
std::vector<ULONG> vCollisionCheckIDS;
ULONG* ptr = vCollisionCheckIDS.data();
INT nCheckIDScount = 0;
std::vector<ULONG> vCollisionWithIDS;
ULONG* ptrtemp = vCollisionWithIDS.data();
INT nWithIDSCount = 0;
```

```
m_ptrKit->Doc_add_sim_collide_data(ptr, nCheckIDScount, ptrtemp, nWithIDSCount);
}
```

通过Doc_clear_sim_collide_data接口可以清空碰撞队设置。

HRESULT Doc_clear_sim_collide_data()

Parameters：

C++示例：

```
void MainWindow::ClearSimCollideData() {
m_ptrKit->Doc_clear_sim_collide_data();
}
```

获取当前选中物体

通过Doc_get_current_selected_obj接口可以获取PQKit中当前已经选中的物体ID。即可知哪些物体在PQKit中已处于被选中状态。

HRESULT Doc_get_current_selected_obj(INT *o_nSelectedCount, ULONG **o_ulSelectedObjIDs)	
Parameters：	
o_nSelectedCount	当前PQKit中已选中的对象个数
o_ulSelectedObjIDs	当前PQKit中已选中的对象ID数组，ID数组需要调用PQAPIFreeArray释放

C#示例：

```
void func () {
IntPtr ptrObjIDs = IntPtr.Zero;
int nCount = 0;
m_ptrKit.Doc_get_current_selected_obj(out nCount, ref ptrObjIDs);
if (ptrObjIDs != IntPtr.Zero) {
int[] nIDs = new int[nCount];
Marshal.Copy(ptrObjIDs, nIDs, 0, nCount);
```

```
m_ptrKit.PQAPIFreeArray((long)ptrObjIDs);
}
}
```

C++示例：

```
void MainWindow::GetCurrentSelectedObj() {
INT nSelectedCount = 0;
ULONG* ulSelectedObjIDs=nullptr;
m_ptrKit->Doc_get_current_selected_obj(&nSelectedCount, &ulSelectedObjIDs);
m_ptrKit->PQAPIFree((LONG_PTR*)ulSelectedObjIDs);
}
```

根据对象ID获取对象类型

HRESULT Doc_get_objtype_by_id(ULONG i_ulObjID, LONG* o_lobjType)	
Parameters：	
i_ulObjID	传入对象ID
o_lobjType	传出对象类型，详情见9.1数据类型定义

C++示例：

```
void MainWindow::GetObjtypeByID() {
ULONG uID = /* */;
LONG ObjType = 0;
m_ptrKit->Doc_get_objtype_by_id(uID, &ObjType);
}
```

批量添加仿真事件

选中要添加事件的轨迹后通过pq_RunCommand执行 RO_CMD_ADDEVENT_BAT命令执行批量添加事件。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
Parameters：	
i_whModuleID	模块ID，如打开文件"RO_CMD_ADDEVENT_BAT"

C++示例：

```
void MainWindow::AddEvent() {
CComBSTR cmd = "RO_CMD_ADDEVENT_BAT";
m_ptrKit->Doc_start_module(cmd);
}
```

新建坐标系

通过pq_RunCommand执行RO_CMD_CREATE_FRAME命令执行新建坐标系。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
Parameters：	
i_whModuleID	模块ID，如打开文件"RO_CMD_CREATE_FRAME"

C++示例：

```
void MainWindow::AddEvent() {
CComBSTR cmd = "RO_CMD_CREATE_FRAME";
m_ptrKit->Doc_start_module(cmd);
}
```

设置标注

HRESULT Doc_insert_annotation(DOUBLE i_dBasePoint[3], DOUBLE i_dTextPoint[3], ULONG i_ulCoordinateID, const WCHAR* i_whText, DOUBLE i_dTextColor[3], DOUBLE i_dBaseColor[3], PQAnnotationType i_eType, LONG i_lPtSize, DOUBLE i_lPtColor[3], DOUBLE i_dFontSize, BOOL i_bFrame, ULONG* o_AnnotationID, ULONG i_uObjID, BOOL i_bUpdate)	
Parameters：	
i_dBasePoint[3]	需要添加标注的起点位置
i_dTextPoint[3]	需要添加标注的文本位置
i_ulCoordinateID	用户坐标系ID
i_whText	标注的文本信息
i_dTextColor[3]	文本颜色
i_dBaseColor[3]	标注线的颜色及文本框颜色
i_eType	选择标注类型，只显示标注文本或者显示标准线及文本
i_lPtSize	标注点大小
i_lPtColor	标注点颜色
i_dFontSize	设置文本字体
i_bFrame	是否显示文本框
o_AnnotationID	输出标注ID
i_uObjID	输入需要绑定的工件ID，若不绑定工件，输入0
i_bUpdate	插入标注后是否更新，如果多次循环调用将i_bUpdate设置为False，在完成所有标注插入后调用

HRESULT Doc_insert_annotation(DOUBLE i_dBasePoint[3], DOUBLE i_dTextPoint[3], ULONG i_ulCoordinateID, const WCHAR* i_whText, DOUBLE i_dTextColor[3], DOUBLE i_dBaseColor[3], PQAnnotationType i_eType, LONG i_lPtSize, DOUBLE i_lPtColor[3], DOUBLE i_dFontSize, BOOL i_bFrame, ULONG* o_AnnotationID, ULONG i_uObjID, BOOL i_bUpdate)	
	Doc_update_view更新一次即可

C++示例：

```
void MainWindow::InsertAnnotation() {
double basePoint[3] = {
}
;
double textPoint[3] = {
}
;
ULONG uCoordinateID = 0;
CComBSTR wText = "HELLO";
double Textcolor[3] = {
}
;
double baseColor[3] = {
}
;
PQAnnotationType atype = PQ_ANNOTATION_LINE;
double dPtSize = 10;
double dPtColor[3] = {
}
;
double fontsize = 15;
BOOL frame = TRUE;
ULONG uID;
ULONG uObjID;
GetObjIDByName(PQ_WORKINGPART, L"A4纸张模板", uObjID);
m_ptrKit->Doc_insert_annotation(basePoint, textPoint, uCoordinateID, wText,Textcolor, baseColor, at
}
```

修改标注

HRESULT Annotation_modify_text(ULONG i_ulAnnotationId, const WCHAR* i_whEditText)	
Parameters：	
i_ulAnnotationId	传入需要修改标注id
i_whEditText	传入需要修改标注文本

C++示例：

```
void MainWindow::AnnotationModifyText() {
ULONG uID = /* */;
CComBSTR wEdittext= "";
m_ptrKit->Annotation_modify_text(uID, &wEdittext);
}
```

获取坐标系

传入坐标系类型，获取当前场景内的所有该类型的坐标系对象。

HRESULT Doc_get_coordinate(PQCoordinateType i_eType, INT* o_nCount, WCHAR** o_whCoordinateNames, INT** o_nCoordinateNameLengthArray, ULONG** o_uCoordinateIDs)	
Parameters：	
i_eType	指定坐标系类型，详见9.8坐标系类型
o_nCount	返回该指定类型坐标系的个数

HRESULT Doc_get_coordinate(PQCoordinateType i_eType, INT* o_nCount, WCHAR** o_whCoordinateNames, INT** o_nCoordinateNameLengthArray, ULONG** o_uCoordinateIDs)	
o_whCoordinateNames	返回该指定类型各坐标系的名字拼接的字符串，字符串需要调用PQAPIFreeArray释放
o_nCoordinateNameLengthArray	返回该指定类型各坐标系名字的长度集合，坐标系名字的长度集合需要调用PQAPIFreeArray释放
o_uCoordinateIDs	返回该指定类型坐标系的ID集合，ID集合需要调用PQAPIFreeArray释放

例如，场景下有2个坐标系，名称分别为Coor0、Coor12，则api调用结果中o_nCount值为2，o_whCoordinateNames值为" Coor0Coor12", o_nCoordinateNameLengthArray的值为{5,6}。

C++示例：

```
void MainWindow::GetCoordinate() {
PQCoordinateType pType;
INT nCount = 2;
WCHAR* whCoordinateName = nullptr;
INT* nCoordinateNameLengthArr = nullptr;
ULONG* uCoordinateIDs = nullptr;
m_ptrKit->Doc_get_coordinate(pType, &nCount, &whCoordinateName, &nCoordinateNameLengthArr, &uCoordi
m_ptrKit->PQAPIFree((LONG_PTR*)whCoordinateName);
m_ptrKit->PQAPIFree((LONG_PTR*)nCoordinateNameLengthArr);
m_ptrKit->PQAPIFree((LONG_PTR*)uCoordinateIDs);
}
```

绘图区画点

在拾取模块执行状态下，可以在绘图区绘制圆点和文字。

HRESULT View_draw_point(DOUBLE i_PtPosition[3], ULONG i_ulCoordinateID, LONG i_lPtSize, LONG i_lPtColor, const WCHAR* i_wsText, LONG i_lTextColor)	
Parameters：	
i_PtPosition	点的位置
i_ulCoordinateID	点是基于哪个坐标系下的
i_lPtSize	点的大小
i_lPtColor	点的颜色
i_wsText	点附带的文本
i_lTextColor	点附带的文本的颜色

C++示例：

```
void MainWindow::ViewDrawPoint() {
double dPosition[3] = {
}
;
ULONG ulCoordinateID = 0;
GetObjIDByName(176, L"坐标系5", ulCoordinateID);
LONG lptSize = 12;
LONG lptColor = RGB(255, 255, 0);
CComBSTR whText;
LONG ltextColor = 15073228;
m_ptrKit->View_draw_point(dPosition, ulCoordinateID, lptSize, lptColor, &whText, ltextColor);
}
```

判断三点共线

判断给出的三个点是否共线

HRESULT Math_is_three_point_collinear([in]DOUBLE* i_dPoint1, [in]DOUBLE* i_dPoint2, [in]DOUBLE* i_dPoint3, [out]BOOL* o_bIsColliner)	
Parameters：	
i_dPoint1	输入点1
i_dPoint2	输入点2
i_dPoint3	输入点3
o_bIsColliner	输出结果

C++示例：

```
void MainWindow::MathThreePointCollinear() {
double dPoint1[3] = {
}
;
double dPoint2[3] = {
}
;
double dPoint3[3] = {
}
;
LONG bIsColliner = 0;
m_ptrKit->Math_is_three_point_collinear(dPoint1, dPoint2, dPoint3, &bIsColliner);
}
```

判断设计环境与真实环境误差是否在范围内

三点校准中，判断设计环境与真实环境的误差值是否在给定的范围内

HRESULT Math_is_calibrate_point_less_distance([in]DOUBLE* i_dSrcPosition, [in]DOUBLE* i_dDesPosition, [in]DOUBLE i_dSafetyValue, [out]BOOL* o_bIsLess)	
Parameters：	
i_dSrcPosition	三点校准中，设计环境的三个点
i_dDesPosition	三点校准中，真实环境的三个点
i_dSafetyValue	输入误差范围
o_bIsLess	输出判断结果

C++示例：

```
void MainWindow::MathCalibratePointLessDistance() {
double dSrcposition[3] = {
}
;
double dDesposition[3] = {
}
;
double dSafevalue = 0.01;
LONG lresult = 0;
m_ptrKit->Math_is_calibrate_point_less_distance(dSrcposition, dDesposition, dSafevalue, &lresult);
}
```

设置地板颜色

根据给定的RGB设置地板颜色

HRESULT Doc_set_floor_color(DOUBLE i_dR, DOUBLE i_dG, DOUBLE i_dB)	
Parameters：	
i_dR	RGB中的R
i_dG	RGB中的G
i_dB	RGB中的B

C++示例：

```
void MainWindow::SetFloorColor() {
m_ptrKit-> Doc_set_floor_color (225,0,0);
}
```

设置零件

根据给定的零件ID设置零件

HRESULT Doc_setup_workingpart(ULONG i_ulWorkingPart)	
Parameters：	
i_ulWorkingPart	设置零件ID

C++示例：

```
void MainWindow::SetUpWorkingPart() {
ULONG uID;
GetObjIDByName(16, L"A1纸张模板", uID);
m_ptrKit->Doc_setup_workingpart(uID);
}
```

获取喷嘴

获取在给定机器人ID下绑定的喷嘴信息

HRESULT Doc_get_nozzle(ULONG i_ulRobotID, INT* o_nSize, WCHAR** o_sNames, ULONG** o_sIDs)	
Parameters：	
i_ulRobotID	机器人ID
o_nSize	喷嘴个数
o_sNames	喷嘴名字
o_sIDs	喷嘴ID

C++示例：

```
void MainWindow::GetNozzle() {
ULONG uRobotID;
GetObjIDByName(32, L"ABB-IRB1410", uRobotID);
int nSize=0;
WCHAR * wchName=nullptr;
ULONG *uNozzleID1=nullptr;
m_ptrKit-> Doc_get_nozzle(uRobotID,&nSize ,& wchName, uNozzleID1);
m_ptrKit->PQAPIFree((LONG_PTR*) wchName);
m_ptrKit->PQAPIFree((LONG_PTR*) uNozzleID1);
}
```

获取喷刷

获取在给定喷嘴ID下绑定的喷刷信息

HRESULT Doc_get_assistor(ULONG i_ulNozzleID, INT* o_nSize, WCHAR** o_sNames, ULONG** o_sIDs)	
Parameters：	
i_ulNozzleID	喷嘴ID

HRESULT Doc_get_assistor(ULONG i_ulNozzleID, INT* o_nSize, WCHAR** o_sNames, ULONG** o_sIDs)	
o_nSize	喷刷个数
o_sNames	喷刷名字
o_sIDs	喷刷ID

C++示例：

```
void MainWindow::Getassistor() {
ULONG uNozzleID1=0;
m_ptrKit->Doc_get_nozzle(uRobotID, L"喷嘴1", uNozzleID1);
int nSize=0;
WCHAR * wchName=nullptr;
ULONG *uAssistorID1=0;
m_ptrKit->Doc_get_assistor(uNozzleID1, &nSize,& wchName, &uAssistorID1);
m_ptrKit->PQAPIFree((LONG_PTR*) wchName);
m_ptrKit->PQAPIFree((LONG_PTR*) uAssistorID1);
}
```

获取框选标注点信息

获取框选中标注点信息

HRESULT Doc_get_selected_annotation_position(INT* o_nPointCount, DOUBLE** o_dBasePoint, DOUBLE** o_dTextPoint，ULONG** o_ulAnnotationIDs)	
Parameters：	
nPointCount	获取标注点个数
o_dBasePoint	获取标注点起点坐标，需要调用PQAPIFreeArray释放

HRESULT Doc_get_selected_annotation_position(INT* o_nPointCount, DOUBLE o_dBasePoint, DOUBLE** o_dTextPoint，ULONG** o_ulAnnotationIDs)**	
o_dTextPoint	获取标注点末端坐标，需要调用PQAPIFreeArray释放
o_ulAnnotationIDs	获取标注ID，需要调用PQAPIFreeArray释放

C++示例：

```
void MainWindow::Getannotationposition() {
INT count=0;
DOUBLE * dBasePoint=nullptr;
DOUBLE * dTextPoint= nullptr;
ULONG*   dAnnotaionID= nullptr;
m_ptrKit->Doc_get_selected_annotation_position(&count,&dBasePoint,&dTextPoint,& dAnnotaionID);
m_ptrKit->PQAPIFree((LONG_PTR*) dBasePoint);
m_ptrKit->PQAPIFree((LONG_PTR*) dTextPoint);
m_ptrKit->PQAPIFree((LONG_PTR*) dAnnotaionID);  }
```

设置机器人多机协同

HRESULT Doc_set_synergism_instruction (ULONG i_ulPathID, ULONG i_ulPathIDSyn, const WCHAR* i_whSystemName)	
Parameters：	
i_ulObjIDs	需要进行轨迹同步的轨迹1(轨迹1与轨迹2所属机器人需要先同属于名为i_whSystemName的协同系统中)
i_nCount	需要进行轨迹同步的轨迹2
i_whSystemName	协同系统名字

HRESULT Doc_set_synergism_instruction (ULONG i_ulPathID, ULONG i_ulPathIDSyn, const WCHAR* i_whSystemName)	
## 设置同步速度	

HRESULT Doc_set_synergism_velocity (ULONG i_ulPathID, ULONG i_ulPathIDSyn)	
Parameters：	
i_ulObjIDs	主动轨迹ID
i_nCount	随动轨迹ID
## 设置同步指令	

HRESULT Doc_synergism_set (ULONG* i_ulObjIDs, INT i_nCount,const WCHAR* i_whSystemName)	
Parameters：	
i_ulObjIDs	设置协同的机构ID数组
i_nCount	机构id数组个数
i_whSystemName	协同系统名称
## 修改坐标系	

HRESULT Doc_set_coordinate_posture(ULONG i_ulCoorID, INT i_nPostureType, DOUBLE* i_dPosture, INT i_nPostureSize, BOOL i_bUpdatePost)	
Parameters：	
i_ulCoorID	坐标系ID

HRESULT Doc_set_coordinate_posture(ULONG i_ulCoorID, INT i_nPostureType, DOUBLE* i_dPosture, INT i_nPostureSize, BOOL i_bUpdatePost)	
i_nPostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义
i_dPosture	修改位姿数据
i_bUpdatePost	是否修改关联工具位置

C++示例：

```
void MainWindow::test() {
ULONG uid;
GetObjIDByName(176, L"坐标系1", uid);
double posture[6] = {
1,1,1,5,2,6 }
;
m_ptrKit->Doc_set_coordinate_posture(uid, 1, posture, 6, 1);
}
```

设置开启/关闭导出三维模型写入ID

开启之后导出实体会在实体中写入面的id信息

HRESULT Doc_set_save_entityId_flag(BOOL i_bIsSave)	
Parameters：	
bIsSave	是否保存,保存传True，不保存传False，全局设置，仅需设置一次,每次重新打开软件不需要重新设置

HRESULT Doc_set_save_entityId_flag(BOOL i_bIsSave)	
## 导出实体模型	

输入实体的id数组，可传入多个id，保存成一个stp文件，只能转实体模型，只能是输入stp输出stp

HRESULT Doc_Export_Model(const WCHAR* i_sFilePath, ULONG* i_uObjIDs,INT i_nObjCount)	
Parameters：	
i_sFilePath	保存stp文件的完整路径，需写清楚文件后缀（.stp）
I_uObjIDs	输入的实体ID数组
I_nObjCount	输入的数组个数
void MainWindow:: Export_Model () { ULONG ulPartID = 0; GetObjIDByName(225, L"模型", ulPartID); ULONG uPartids[1] = { ulPartID }; const WCHAR* Filetem = L"C:\Users\tem\Desktop\测试.step"; m_ptrKit-> Doc_Export_Model (Filetem，uPartids，1); }	
## 绘制直线	
### 1.50.1绘制直线	

在绘图区绘制直线，可设置起点，终点，颜色，线粗细

HRESULT Doc_draw_cylinder(DOUBLE* i_dLineStart, INT i_nStartCount,DOUBLE* i_dLineEnd, INT i_nEndCount, DOUBLE i_radius, DOUBLE* i_dColorRGB, INT i_nColorCount, ULONG i_uCoordinateID,ULONG* o_uCylinderID, BOOL i_bIsUpdate)	
Parameters：	
i_dLineStart	输入线的起点坐标xyz
i_nStartCount	起点位置数组长度，一般为3
i_dLineEnd	输入线的终点坐标xyz
i_nEndCount	终点位置数组长度，一般为3
i_radius	线的粗细，最大为64
i_dColorRGB	线的颜色RGB(0-255)三色数组
i_nColorCount	传入颜色数组的长度，一般为3
i_uCoordinateID	传入位置参考坐标系，若为世界坐标系传入0
o_uCylinderID	传出画线对象id
i_bIsUpdate	是否刷新绘图区，若循环插入多条线，设置成false，最后一条线设置为True刷新，或者所有线生成后调用Doc_update_view刷新一次即可，减少计算时间
void MainWindow:: DrawLine () { double start[3] = {0.0}; double dEnd[3] = {100,100,100}; double dRGB[3] = { 255, 0, 0 }; ULONG i_uCoordinateID = 0; ULONG o_uCylinderID = 0; m_ptrKit ->Doc_draw_cylinder(start,3, dEnd,3,16, dRGB,3,i_uCoordinateID,&o_uCylinderID， false); }	

HRESULT Doc_draw_cylinder(DOUBLE* i_dLineStart, INT i_nStartCount,DOUBLE* i_dLineEnd, INT i_nEndCount, DOUBLE i_radius, DOUBLE* i_dColorRGB, INT i_nColorCount, ULONG i_uCoordinateID,ULONG* o_uCylinderID, BOOL i_bIsUpdate)	
### 1.50.2改变线的颜色	

HRESULT Doc_change_cylinder_color (ULONG i_uCylinderID, DOUBLE* i_dColorRGB, INT i_nColorCount)	
Parameters：	
i_dLineStart	输入线的起点坐标xyz
dColorRGB	线的颜色RGB(0-255)三色数组
nColorCount	传入颜色数组的长度，一般为3
void MainWindow:: ChangeLineColor () { double dRGB[3] = { 255, 0, 0 }; ULONG i_uCylinderID =/*自行获取*/； m_ptrKit -> Doc_change_cylinder_color (i_uCylinderID，dRGB ， 3); }	
### 1.50.3按边绘制直线	

调用Doc_draw_straight_line 接口，传入直线边名称绘制该直线，可由3.12 接口获取或者拾取命令中获取。

HRESULT Doc_draw_straight_line(const WCHAR* i_wsEdgeName, DOUBLE i_dLineRadius, LONG i_lColor, const WCHAR* i_wsDisplayName, ULONG i_uCoordinateID, ULONG* o_ulAccPartID)	
SDK: 11.2.0.7732	

HRESULT Doc_draw_straight_line(const WCHAR* i_wsEdgeName, DOUBLE i_dLineRadius, LONG i_lColor, const WCHAR* i_wsDisplayName, ULONG i_uCoordinateID, ULONG* o_ulAccPartID)	
Parameters：	
i_wsEdgeName	边信息，可由3.12 接口获取或者拾取命令中获取
i_dLineRadius	圆柱半径
i_lColor	指定颜色
i_wsDisplayName	绘制出的圆弧显示的名称
i_uCoordinateID	绘制参考的坐标系，传0即相对于世界坐标系
o_ulAccPartID	传出转化的场景元素ID

C++参考代码段

```
LONG lColor = RGB(255.0, 165.0, 0.0);
ULONG ulLineID = 0;
m_ptrKit->Doc_draw_straight_line(i_wsTargetEdge,3.0,lColor, i_wsTargetEdge, 0, &ulLineID);
```

绘制标准圆弧

PQkit提供两种方式在设计环境中绘制出指定圆弧（圆弧柱体形式），并将其转化成场景元素。

<1>调用Doc_draw_circle 接口，传入圆弧名称绘制圆弧，可由3.12 接口获取或者拾取命令中获取。

HRESULT Doc_draw_circle(const WCHAR* i_wsCircleEdgeName, DOUBLE i_dLineRadius, LONG i_lColor, const WCHAR* i_wsDisplayName, ULONG i_uCoordinateID, ULONG* o_ulAccPartID)	
SDK: 11.1.0.7592	
Parameters：	
i_wsCircleEdgeName	边信息，可由3.12 接口获取或者拾取命令中获取
i_dLineRadius	圆柱半径
i_lColor	指定颜色
i_wsDisplayName	绘制出的圆弧显示的名称
i_uCoordinateID	绘制参考的坐标系，传0即相对于世界坐标系
o_ulAccPartID	传出转化的场景元素ID

C++参考代码段

```
 LONG lColor = RGB(255.0, 165.0, 0.0);
ULONG ulID = 0;
wchar_t whEdge[] = _T("79#38");
m_ptrKit->Doc_draw_circle(whEdge, 3.0, lColor, whEdge, 0, &ulID);
```

<2>调用Doc_draw_circle_with_pt接口传入圆上连续3点绘制圆弧。

HRESULT Doc_draw_circle_with_pt (const DOUBLE i_dCirclePt1[3], const DOUBLE i_dCirclePt2[3], const DOUBLE i_dCirclePt3[3], DOUBLE i_dLineRadius, LONG i_lColor, const WCHAR* i_wsDisplayName, ULONG i_uCoordinateID, ULONG* o_ulAccPartID)	
SDK: 11.1.0.7608	
Parameters：	
i_dCirclePt1	圆上连续三点的第一点
i_dCirclePt2	圆上连续三点的第二点
i_dCirclePt3	圆上连续三点的第三点
i_dLineRadius	圆柱半径
i_lColor	指定颜色
i_wsDisplayName	绘制出的圆弧显示的名称
i_uCoordinateID	绘制参考的坐标系，传0即相对于世界坐标系
o_ulAccPartID	传出转化的场景元素ID
## 动态碰撞检测(非仿真中)	

使用Doc_set_collide_data设置待检测的两组物体，Doc_get_collide_result输入检测距离以及碰撞结果，Doc_clear_collide_data清空检测对象缓存，注意每次使用Doc_set_collide_data设置检测物体前需要使用Doc_clear_collide_data情况后重新设置；

1.51.1设置碰撞物体

HRESULT Doc_set_collide_data (ULONG* i_vCollisionCheckIDS, INT i_nCheckIDSCount, ULONG* i_vCollisionWithIDS, INT i_nWithIDSCount)	
Parameters：	
vCollisionCheckIDS	第一组碰撞对物体的ID
i_nCheckIDSCount	第一组碰撞对个数
i_vCollisionWithIDS	第二组碰撞对物体的ID
i_nWithIDSCount	第二组碰撞对个数
### 1.51.2获取检测结果	

HRESULT Doc_get_collide_result(DOUBLE i_dMInDistance, ,BOOL b_isPreciseDetection ， ULONG o_ulObjAIDs, INT* o_nObjCount);**	
Parameters：	
i_dMInDistance	碰撞检测最小距离（需要物体完全碰撞再检测输入0，需要物体小于一定距离后便检测输入距离，单位mm）
o_ulObjAIDs	发生碰撞物体的ID，两个为一组
b_isPreciseDetection	是否精确检测，传True为精确检测，精准度高耗时长，传False为包围盒检测，计算速度快精准度降低
o_nObjCount	碰撞物体个数

1.51.3 情况碰撞物体缓存

HRESULT Doc_clear_collide_data ();

C++示例：

```
void MainWindow:: CollideDetection() {
std::vector<ULONG> vCollisionCheckIDS;
ULONG* ptr = vCollisionCheckIDS.data();
INT nCheckIDScount = vCollisionCheckIDS.size();
std::vector<ULONG> vCollisionWithIDS;
ULONG* ptrtemp = vCollisionWithIDS.data();
INT nWithIDSCount = vCollisionWithIDS.size();
m_ptrKit-> Doc_clear_collide_data ();
m_ptrKit-> Doc_set_collide_data (ptr, nCheckIDScount, ptrtemp, nWithIDSCount);
ULONG* uObjIOs = nillptr;
INT oCount = 0;
m_ptrKit-> Doc_get_collide_result(0,True, &uObjIOs, &oCount)
m_ptrKit.PQAPIFreeArray((long) uObjIOs);
}
```

根据点和两个向量计算位姿矩阵

依据某一点位置和x和Z两个方向的向量给出对应的位姿矩阵

HRESULT Math_trans_vector_to_posture(DOUBLE i_dPoint[3], DOUBLE i_dXVector[3], DOUBLE i_dZVector[3], PQPostureType i_ePostureType, DOUBLE o_dPosture, INT* o_nPostureArraySize)**	
Parameters：	
i_dPoint	输入点位置的数组(xyz)
i_dXVector	X方向法向
i_dZtVector	Z方向法向
i_ePostureType	输出位姿类型，，如四元数、欧拉角XYZ、欧拉角Z YX等

HRESULT Math_trans_vector_to_posture(DOUBLE i_dPoint[3], DOUBLE i_dXVector[3], DOUBLE i_dZVector[3], PQPostureType i_ePostureType, DOUBLE o_dPosture, INT* o_nPostureArraySize)**	
	等，详见9.2 位姿描述定义
o_dPosture	输出位姿矩阵，需要释放内存
o_nPostureArraySize	输出位姿矩阵个数
## 修改标注显示	

根据给定显示类型，修改标注显示

HRESULT Annotation_modify_display(PQAnnotationDisplayType i_pqAnnotationDisplayType)	
SDK:11.0.1.7407	
Parameters：	
i_pqAnnotationDisplayType	需要修改显示类型，详见9.10标注显示类型

C++示例：

```
m_ptrKitCallback->Annotation_modify_display(PQ_SHOWONLYPOINT);
```

场景文件转零件

PQKit提供了接口可以快速将导入的场景文件转换为零件对象，这里仅仅涉及到数据格式的转换，如需定义零件其它数据如CP、RP点等，需要执行定义零件模块（模块ID为"RO_CMD_CREATE_WORKINGPART"）。

HRESULT Doc_transform_accessory_into_part(ULONG i_ulAccessoryID, ULONG* o_ulPartID, BOOL i_bUpdateView)	
SDK:11.1.0.7425	
Parameters：	
i_ulAccessoryID	导入的场景文件ID
o_ulPartID	转换成的零件ID
i_bUpdateView	是否需要触发PQKit场景更新

C++示例代码：

```
//以下代码展示导入一个stl文件，并将其转化为PQKit的零件对象，然后变位机抓取该零件 wchar_t whPath[] =
ULONG ulAccPartID = 0;
m_ptrKit->Doc_import_accessory_part(whPath, 0, &ulAccPartID);
ULONG ulPartID = 0;
m_ptrKit->Doc_transform_accessory_into_part(ulAccPartID, &ulPartID, TRUE);
ULONG uExternalID = 0;
GetObjIDByName(PQ_ROBOT, _T("变位机或者机器人的名称"), uExternalID);
m_ptrKit->Doc_catch_obj(uExternalID, ulPartID);
```

文档自动保存设置

PQKit自动保存文档功能默认是开启状态，如需关闭或者开启，需要调用Doc_enable_autosave接口。

HRESULT Doc_enable_autosave(BOOL i_bEnable)	
SDK:11.1.0.7430	
Parameters：	
i_bEnable	是否开启文档自动保存功能

HRESULT Doc_enable_autosave(BOOL i_bEnable)	
## 点云校准	

输入固定物体上四个点的xyz与待校准物体上的四个点xyz数组，会将待校准物体位置进行校准

HRESULT Doc_Point_cloud_calibration (ULONG i_uFixID, DOUBLE* i_dFixPosture, INT i_nFixPostureCount, ULONG u_MoveId, DOUBLE* i_dMovePosture, INT i_nMovePostureCount)	
SDK:11.1.0.7431	
Parameters：	
i_uFixID	固定物体id
i_dFixPosture	固定物体上四个点世界坐标系下xyz数组
i_nFixPostureCount	固定物体位置数组长度，一般为12
u_MoveId	移动物体id
i_dMovePosture	移动物体上四个点世界坐标系下xyz数组
i_nMovePostureCount	移动物体位置数组长度，一般为12
## 获取事件模板个数	

输入机器人ID，获取事件模板个数

HRESULT Doc_get_event_template_count(ULONG i_ulRobotID, INT* o_nTemplateCount)	
SDK:11.1.0.7592	
Parameters：	
i_ulRobotID	机器人id
o_nTemplateCount	事件模板个数

C++示例代码：

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"SJPTSROBOT", ulRobotID);
int nEventTemplateLen = 0;
m_ptrKit->Doc_get_event_template_count(ulRobotID, &nEventTemplateLen);
```

获取事件模板名字和内容

输入机器人ID，以及想获取的事件模板的序列，从1开始计数，输出事件模板名字和内容

HRESULT Doc_get_event_template(ULONG i_ulRobotID, INT i_nTemplateNum, WCHAR** o_whTemplateName, WCHAR** o_whTemplateContent)	
SDK:11.1.0.7592	
Parameters：	
i_ulRobotID	机器人id
i_nTemplateNum	想要获取第几个事件模板名字和内容，从1开始计数
o_whTemplateName	获取事件模板的名字

HRESULT Doc_get_event_template(ULONG i_ulRobotID, INT i_nTemplateNum, WCHAR** o_whTemplateName, WCHAR** o_whTemplateContent)	
o_whTemplateContent	获取事件模板的内容

C++示例代码：

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"SJPTSROBOT", ulRobotID);
WCHAR* wsEventTemplate = nullptr;
WCHAR* wsTemplateContext = nullptr;
m_ptrKit->Doc_get_event_template(ulRobotID, 2, &wsEventTemplate, &wsTemplateContext);
```

创建Box场景文件

调用Doc_create_box_accessory_part接口可以创建一个指定长宽高的方形场景文件。

HRESULT Doc_create_box_accessory_part(DOUBLE i_dLength, DOUBLE i_dWidth, DOUBLE i_dHeight, LONG i_lColor, const WCHAR* i_wsDisplayName, ULONG*o_ulPartID)	
SDK:11.2.0.7685	
Parameters：	
i_dLength	方形-长，mm单位
i_dWidth	方形-宽，mm单位
i_dHeight	方形-高，mm单位
i_lColor	指定颜色

HRESULT Doc_create_box_accessory_part(DOUBLE i_dLength, DOUBLE i_dWidth, DOUBLE i_dHeight, LONG i_lColor, const WCHAR* i_wsDisplayName, ULONG*o_ulPartID)	
i_wsDisplayName	显示的名字
o_ulPartID	返回创建的场景文件的ID

C++示例：

```
COLORREF cColor = RGB(220.0, 20.0, 147.0);
wchar_t whName[] = _T("NewPart-1");
ULONG ulPartID = 0;
m_ptrKit->Doc_create_box_accessory_part(30.0,40.0,360.0, cColor, whName,&ulPartID);
```

保存三维区域缩略图

调用Doc_save_scene_image接口可以指定一个jpg图片的绝对路径、长、宽数据来保存当前三维场景的缩略图。如果长、宽都给0的话，会按当前系统分辨率尺寸保存。

HRESULT Doc_save_scene_image(const WCHAR* i_wtImagePath, INT i_nWidth, INT i_nHeight)	
SDK:11.2.0.7788	
Parameters：	
i_wtImagePath	jpg格式文件绝对路径
i_nWidth	图片宽度，像素单位
i_nHeight	图片高度，像素单位

HRESULT Doc_save_scene_image(const WCHAR* i_wtImagePath, INT i_nWidth, INT i_nHeight)	
# 机构	
## 获取机构末端位姿	

获取机器人末端法兰中心点位姿，如安装了工具则获取的是该工具当前使用的TCP处的位姿。

HRESULT Robot_get_end_posture(ULONG i_ulObjID, PQPostureType i_ePostureType, INT *o_nPostureArraySize, DOUBLE **o_dPosture)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameters：	
i_ulObjID	机器人ID
i_ePostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义
o_nPostureArraySize	返回位姿数组长度
o_dPosture	具体位姿数据，需要调用PQAPIFreeArray释放内存

C++示例：

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, _T(""), ulRobotID);
int nCount = 0;
double* dEndPosture = nullptr;
m_ptrKit->Robot_get_end_posture(ulRobotID,EULERANGLEXYZ,&nCount,&dEndPosture);
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dEndPosture);
```

创建外部轴链接

HRESULT PQAPICreateExternalLink(ULONG i_ulEngineID, ULONG i_ulGuideID, INT i_nAngle, BOOL i_bSyncPosition, ULONG i_ulPositionerID)	
Parameters：	
i_ulEngineID	欲创建和导轨或者变位机关链接的机器人ID
i_ulGuideID	导轨ID
i_nAngle	机器人在导轨上的安装角度
i_bSyncPosition	机器人与导轨是否同步位置
i_ulPositionerID	变位机ID

C++示例：

```
void MainWindow::CreateExternalLink() {
ULONG uGuideID = 0;
GetObjIDByName(32, L"ABB-IRBT4004_Mirrored_7m_REV5", uGuideID);
ULONG uEngineID = 0;
GetObjIDByName(32, L"ABB-IRB4600-45", uEngineID);
INT nAngle = 30;
LONG bSyncPosition = 1;
ULONG uPositionerID = 0;
m_ptrKit->PQAPICreateExternalLink(uEngineID, uGuideID, nAngle, bSyncPosition, uPositionerID);
}
```

解除外部轴链接

HRESULT PQAPIDeleteExternalLink(ULONG i_ulEngineID,ULONG i_ulExternalID,BOOL i_bClearAll)	
Parameters：	

HRESULT PQAPIDeleteExternalLink(ULONG i_ulEngineID,ULONG i_ulExternalID,BOOL i_bClearAll)	
i_ulEngineID	欲解除和导轨或者变位机关链接的机器人ID
i_ulExternalID	欲解除的导轨或者变位机关的ID
i_bClearAll	是否清空外部轴链接

C++示例：

```
void MainWindow::DeleteExternalLink() {
ULONG uEngineID = 0;
GetObjIDByName(32, L" ABB-IRB4600-45", uEngineID);
ULONG uExternalID = 0;
GetObjIDByName(32, L" ABB-IRB4600-45", uExternalID);
LONG bClearAll = 1;
m_ptrKit->PQAPIDeleteExternalLink(uEngineID, uExternalID, bClearAll);
}
```

获取机构基坐标系

HRESULT Robot_get_base_coordinate(ULONG i_ulRobotID, ULONG* o_uCoordinateID)	
Parameters：	
i_ulRobotID	指定机构ID
o_uCoordinateID	指定机构的基坐标系ID

C++示例：

```
void MainWindow::GetBaseCoordinate() {
ULONG uRobotID = 0;
GetObjIDByName(32, L" ABB-IRB4600-45", uRobotID);
ULONG uCoordinateID = 0;
```

```
m_ptrKit->Robot_get_base_coordinate(uRootID, &uCoordinateID);
}
```

获取机构下轨迹组

HRESULT Doc_get_pathgroup_name(ULONG i_ulRobotID, VARIANT* o_varNameArray)	
Parameters：	
i_ulRobotID	指定机构ID
o_varNameArray	机构下轨迹组名称数组

C++示例：

```
void MainWindow::GetPathgroupName() {
ULONG uRobotID = 0;
GetObjIDByName(32, L" ABB-IRB4600-45", uRobotID);
VARIANT varName;
varName.parray = NULL;
m_ptrKit->Doc_get_pathgroup_name(uRobotID, &varName);
}
```

定义机构

通过pq_RunCommand执行RO_CMD_DEFINE_MECHANISM命令，来定义机构。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
Parameters：	
i_whModuleID	模块ID，如打开文件"RO_CMD_DEFINE_MECHANISM"

C++参考代码段

```
void MainWindow::DefineMechaism() {
CComBSTR strCMD = "RO_CMD_DEFINE_MECHANISM";
m_ptrKit->Doc_start_module(strCMD);
}
```

导入自定义机构

<1>执行RO_CMD_DEFROBOT_SETUP模块，手动选择

<2>调用接口Doc_insert_custom_define_robot传入robr文件的绝对路径，将该自定义机器人导入场景，返回机器人ID。

HRESULT Doc_insert_custom_define_robot(const WCHAR* i_wcRobrPath, ULONG* o_ulRobotID)	
SDK: 11.1.0.7310	
Parameters：	
i_wcRobrPath	robr文件的绝对路径
o_ulRobotID	返回机器人ID

C++参考代码段

```
ULONG ulRobotID = 0;
wchar_t whPath[] = L"D:\\Temp\\ER20_2300_HI_A6.robr";
m_ptrKit->Doc_insert_custom_define_robot(whPath, &ulRobotID);
```

定义状态机

通过pq_RunCommand执行RO_CMD_DEFINE_MECHSTATE命令，来定义机构。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
Parameters：	

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
i_whModuleID	模块ID，如打开文件"RO_CMD_DEFINE_MECHSTATE"

C++参考代码段

```
void MainWindow::DefineMechstate() {
CComBSTR strCMD = "RO_CMD_DEFINE_MECHSTATE";
m_ptrKit->Doc_start_module(strCMD);
}
```

导入自定义状态机

通过pq_RunCommand执行RO_CMD_IMPORT_MECHSTATE命令， bsParam传入自定义状态机文件的绝对路径来导入自定义状态机。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
Parameters：	
bsCommandID	"RO_CMD_IMPORT_MECHSTATE"
wParam	0或NULL
lParam	0或NULL
bsParam	导入的自定义零件的绝对路径
varParam	Empty
lResult	0

C++参考代码段

```
void MainWindow::ImportMechstate() {
CComBSTR strCMD = "RO_CMD_IMPORT_MECHSTATE";
```

```
CComBSTR bsparam = ""//绝对路径;
long long lresult;
m_ptrKit->pq_RunCommand(cmd, NULL, NULL, bsparam, CCOMVariant(), &lresult);
}
```

获取机器人装载工具

HRESULT Robot_get_tool(ULONG i_ulRobotID, ULONG *o_ulToolID)	
Parameters：	
ulRobotID	指定机器人ID
o_ulToolID	返回机器人所用工具的ID

C++参考代码段

```
void MainWindow::GetTool() {
ULONG uRobotID = 0;
GetObjIDByName(32, L"", uRobotID);
ULONG uToolID = 0;
m_ptrKit->Robot_get_tool(uRobotID, &uToolID);
}
```

获取机器人轴数

HRESULT Robot_get_joints_count(ULONG i_ulRobotID, INT* o_nJointsCount)	
Parameters：	
ulRobotID	指定机器人ID
o_nJointsCount	返回机器人轴数

C++参考代码段

```
ULONG uRobotID = 0;
int nCount = 0;
m_ptrKit->Robot_get_tool(uRobotID, & nCount);
```

获取机器人类型

HRESULT Robot_get_model_num (ULONG i_ulRobotID, WCHAR o_wsRobotType)**	
Parameters：	
ulRobotID	指定机器人ID
o_wsRobotType	返回机器人的类型（定义机构时决定） 需要调用PQAPIFreeArray释放

C++参考代码段

```
ULONG uRobotID = 0;
WCHAR* wsType = nullptr;
m_ptrKit-> Robot_get_model_num (uRobotID, &wsType);
m_ptrKit. PQAPIFreeArray ((LONG_PTR*) wsType);
```

获取机器人正运动学求解结果

HRESULT Robot_get_forward_kinematics (ULONG i_ulRobotID, DOUBLE* i_dRobotJoint, INT i_nJointCount, INT i_nPostureType, ULONG i_ulCoordinateID, BOOL i_bIsTcpPosture, INT* o_nPostureArraySize, DOUBLE o_dPosture)**	
Parameters：	
ulRobotID	指定机器人ID
i_dRobotJoint	传入机器人关节角，均为弧度值（不带外部轴信息）
i_nJointCount	关节角个数

HRESULT Robot_get_forward_kinematics (ULONG i_ulRobotID, DOUBLE* i_dRobotJoint, INT i_nJointCount, INT i_nPostureType, ULONG i_ulCoordinateID, BOOL i_bIsTcpPosture, INT* o_nPostureArraySize, DOUBLE** o_dPosture)	
i_nPostureType	所需要传出末端的位姿的姿态类型，可参考9.2
i_ulCoordinateID	指定位姿的参考坐标系，若为世界坐标系传0
i_bIsTcpPosture	传出姿态是否为tcp位姿，若是传TRUE，传FALSE获取的为法兰位姿态
o_nPostureArraySize	传出位姿数组长度
o_dPosture	传出的位姿态数组，需要调用PQAPIFree释放

C++参考代码段

```
ULONG uRobotID = 0;
double dJoint[6] = {
0,0,0,0,0,1.59 }
;
INT o_nPostureArraySize = 0;
DOUBLE* o_dPosture = nullptr;
ptrKit ->Robot_get_forward_kinematics(uRobotID, dJoint, 6, 1, 0,1, &o_nPostureArraySize, &o_dPostur
PQAPIFree((LONG_PTR*) o_dPosture);
```

获取机器人品牌

HRESULT Robot_get_brand (ULONG i_ulRobotID, WCHAR** o_chRobotBrand)	
Parameters：	
ulRobotID	指定机器人ID
o_chRobotBrand	传出机器人品牌，需要调用PQAPIFreeArray释放

C++参考代码段

```
ULONG uRobotID = 0;
WCHAR* o_chRobotBrand = null;
ptrKit -> Robot_get_brand (uRobotID, &o_chRobotBrand);
m_ptrKit->PQAPIFreeArray ((LONG_PTR*) o_chRobotBrand);
```

获取机器人类型

该接口用于细分机器人是导轨，变位机，还是机器人

HRESULT Robot_get_type (ULONG i_ulRobotID, PQRobotType* o_eRobotType)	
Parameters：	
ulRobotID	指定机器人ID
o_eRobotType	传出机器人类型，详情参考9.9

C++参考代码段

```
ULONG uRobotID = 0;
PQRobotType o_eRobotType = PQ_Robot;
ptrKit -> Robot_get_type (uRobotID, &eRobotType);
```

初始化config结构体

2.16.1初始化struct_PQ6RConfig

HRESULT Doc_init_6Rconfig (struct_PQ6RConfig* o_eAxisCfg)

Sdk：11.1.0.7438

Parameters：

2.16.2初始化 struct_PQ3R1PConfig

HRESULT Doc_init_3R1Pconfig (struct_PQ3R1PConfig * o_eAxisCfg)

Sdk：11.1.0.7438

Parameters：

获取机器人轴配置

2.17.1 6r机器人获取轴配置

HRESULT Robot_6R_get_axis_config（ULONG i_ulRobotID, PQ6RConfig* o_eAxisCfg)	
Sdk：11.1.0.7438	
Parameters：	
i_ulRobotID	机器人id
o_eAxisCfg	传出轴配置信息

C++参考代码段

```
ULONG uRobotID = 0;
PQ6RConfig o_eAxisCfg;
ptrKit -> Robot_6R_get_axis_config (uRobotID, & o_eAxisCfg);
```

2.17.2 3R1P机器人获取轴配置

HRESULT Robot_3R1P_get_axis_config（ULONG i_ulRobotID, PQ3R1PConfig* o_eAxisCfg）	
Sdk：11.1.0.7438	
Parameters：	
i_ulRobotID	机器人id
o_eAxisCfg	传出轴配置信息

C++参考代码段

```
ULONG uRobotID = 0;
PQ3R1PConfig o_eAxisCfg;
ptrKit -> Robot_3R1P_get_axis_config (uRobotID, & o_eAxisCfg);
```

获取奇异点浮动

HRESULT Robot_get_floating（ULONG i_ulRobotID, DOUBLE* o_dFloating）	
Sdk：11.1.0.7438	
Parameters：	
i_ulRobotID	机器人id
o_dFloating	奇异点浮动值

C++参考代码段

```
ULONG uRobotID = 0;
DOUBLE o_dFloating;
ptrKit -> Robot_get_floating (uRobotID, &o_dFloating);
```

获取机器人指定轴正解

HRESULT Robot_get_forward_kinematics_by_jointIndex（ULONG i_ulRobotID, DOUBLE* i_dRobotJoint, INT i_nJointCount, INT i_nJointIndex, PQPostureType i_ePostureType, ULONG i_ulCoordinateID, INT* o_nPostureArraySize, DOUBLE o_dPosture）**	
Sdk：11.1.0.7684	
Parameters：	
i_ulRobotID	机器人id
i_dRobotJoint	机器人待求解关节角，均为弧度值
i_nJointCount	关节角数组个数
i_nJointIndex	需要求解末端位置的关节索引，0为Base，-1为末端
i_ePostureType	需要传出位姿的
i_ulCoordinateID	传出位姿参考坐标系的的id，世界坐标系为0
o_nPostureArraySize	传出位姿数组大小
o_dPosture	传出位姿数组

C++参考代码段

```
ULONG uRobotID = 0;
DOUBLE i_dRobotJoint={
0.17,0,0.157,0.079,0,0}
;
PQPostureType  i_ePostureType = EULERANGLEXYZ;
INT o_nPostureArraySize = 0;
DOUBLE * o_dPosture = nullptr;
ptrKit -> Robot_get_forward_kinematics_by_jointIndex (uRobotID, i_dRobotJoint,
6,4, i_ePostureType,0, &o_nPostureArraySize, &o_nPostureArraySize);
m_ptrKit->PQAPIFree((LONG_PTR*) o_dPosture);
```

获取机器人链接物体

HRESULT Robot_get_link_Obj(ULONG i_ulRobotID, ULONG o_uObjIds, INT* o_nObjArraySize)**	
Sdk：11.1.0.7830	
Parameters：	
i_ulRobotID	机器人id
o_uObjIds	机器人链接物体的id数组
o_nObjArraySize	链接物体的id数组长度

C++参考代码段

```
ULONG uRobotID = 0;
ULONG * o_uObjIds = nullptr;
INT o_nObjArraySize = 0;
ptrKit -> Robot_get_link_Obj (uRobotID, &o_uObjIds, &o_nObjArraySize);
m_ptrKit->PQAPIFree((LONG_PTR*) o_dPosture);
```

获取抓取指定物体的id

HRESULT Doc_get_drived_obj(ULONG i_ulID, ULONG* o_nCatchID)	
Sdk：11.1.0.7830	
Parameters：	
i_ulID	传入物体id
o_nCatchID	抓取着i_ulID的物体id

C++参考代码段

```
ULONG i_ulID = 0;
ULONG  o_nCatchID = 0;
ptrKit -> Doc_get_drived_obj (i_ulID, & o_nCatchID);
```

获取机器人额定关节速度

调用Robot_get_rated_joint_velocity可以获取机器人各关节的额定关节速度(角度/秒)。

HRESULT Robot_get_rated_joint_velocity(ULONG i_ulRobotID, INT* o_nJointCount, DOUBLE o_dJointVelocity)**	
SDK:11.2.0.7865	
Parameters：	
i_ulRobotID	机器人ID
o_nJointCount	传出关节速度数组大小
o_dJointVelocity	传出关节速度数组，需要调用PQAPIFreeArray释放
# 工件	
## 创建长方体工件	

HRESULT PQAPICreateBoxPart(DOUBLE i_dLength, DOUBLE i_dWidth, DOUBLE i_dHeight, DOUBLE i_dR, DOUBLE i_dG, DOUBLE i_dB, BSTR i_PartName, ULONG* o_PartID)	
Parameters：	
i_dLength	工件长度
i_dWidth	工件宽度
i_dHeight	工件高度
i_dR	工件颜色-R值
i_dG	工件颜色-G值
i_dB	工件颜色-B值
i_PartName	工件名称
o_PartID	返回工件ID

C++参考代码段

```
void MainWindow::CreateBoxPart() {
double dLength = 2.2;
double dWidth = 3.5;
double dHeight = 4.14;
double dr = 255;
double db = 255;
double dg = 0;
CComBSTR partName = "";
ULONG uID = 0;
```

```
m_ptrKit->PQAPICreateBoxPart(dLength, dHeight, dWidth, dr, dg, db, partName, &uID);
}
```

三点法校准工件

HRESULT PQAPICalibratePart(ULONG i_ulPartID, DOUBLE* i_dSrcPosition, DOUBLE* i_dDesPosition, ULONG i_uCoordinateID，INT i_nTolerancesType, DOUBLE i_dTolerancesValue)	
Parameters：	
i_ulPartID	欲校准工件ID
i_dSrcPosition	欲校准工件上选择不共线的3个点位姿
i_dDesPosition	真实环境中i_dSrcPosition 参数指定的3个点各自对应的点位姿
i_uCoordinateID	校准参考坐标系，如指定坐标系，则i_dSrcPosition、i_dDesPosition均需为该坐标系下位姿。如位姿均为世界坐标系下位姿，则指定为0即可。
i_nTolerancesType	选择误差分配方式，0为平均分配，1为以第一个点为基准
i_dTolerancesValue	输入误差值

三点校准的使用流程如下：

<1>获取欲校准的工件的ID。

<2>调用拾取实体元素模块（见7.6）进入拾取状态，每拾取到一个点，即可在回调函数中调用Part_get_vertex_position即可获取该点的位置。

<3>指定第二步中选中的3个点分别对应的真实环境中的点的xyz。

<4>调用PQAPICalibratePart即可实现校准。

C++参考代码段

```
void MainWindow::CalibratePart() {
ULONG uID = 0;
GetObjIDByName(16, L"", uID);
double dSrcPosition[18] = {
}
;
double dDesPosition[18] = {
}
;
ULONG uCoordinateID = 0;
INT nTolerancesType = 1;
double dTolenceValue = 0.5;
m_ptrKit->PQAPICalibratePart(uID, dSrcPosition, dDesPosition, uCoordinateID, nTolerancesType, dTole
}
```

获取工件顶点个数

HRESULT PQAPIGetWorkPartVertexCount(ULONG i_ulPartID, LONG *o_nCount)	
SDK:9.2.0.6472	
Parameters：	
i_ulPartID	工件ID
o_nCount	工件顶点个数
## 获取工件顶点位置	

PQKit提供了两种方式可以获取指定零件的顶点数据。

<1>PQAPIGetWorkPartVertex,需要先调用PQAPIGetWorkPartVertexCount指定零件下的顶点个数，然后再构建一个顶点个数乘3大小的数组，传给接口获取顶点的位置数据。

HRESULT PQAPIGetWorkPartVertex(ULONG i_ulPartID,ULONG i_uCoordinateID,INT i_nCount, DOUBLE* i_dSrcPosition)	
SDK:9.2.0.6472	
Parameters：	
i_ulPartID	工件ID
i_uCoordinateID	参考坐标系
i_nCount	顶点个数
i_dSrcPosition	顶点位置数组

C++参考代码段

```
ULONG ulPartID = /*自行获取*/;
LONG lCount = 0;
m_ptrKit->PQAPIGetWorkPartVertexCount(ulPartID, &lCount);
double* dPosition = new double[lCount * 3];
m_ptrKit->PQAPIGetWorkPartVertex(ulPartID, 0, lCount, dPosition);
…… delete[] dPosition;
dPosition = nullptr;
```

<2> Part_get_vertex_position_ex,可以一次性传出指定零件下顶点个数及顶点的位置。

HRESULT Part_get_vertex_position_ex(ULONG i_ulPartID, ULONG i_ulCoordinateID,INT* o_nVertexCount, DOUBLE o_dVertexPosition)**	
SDK:11.2.0.7788	
Parameters：	
i_ulPartID	工件ID

HRESULT Part_get_vertex_position_ex(ULONG i_ulPartID, ULONG i_ulCoordinateID,INT* o_nVertexCount, DOUBLE o_dVertexPosition)**	
i_uCoordinateID	参考坐标系
o_nVertexCount	顶点个数
o_dVertexPosition	顶点位置数组,需要调用PQAPIFreeArray释放

C++参考代码段

```
ULONG ulPartID = /*自行获取*/;
double* dPosition = nullptr;
int nCount = 0;
m_ptrKit->Part_get_vertex_position_ex(ulPartID,0,&nCount, &dPosition);
……
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dPosition);
```

定义零件

通过pq_RunCommand执行RO_CMD_CREATE_WORKINGPART命令，来定义零件。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
Parameters：	
i_whModuleID	模块ID，如打开文件"RO_CMD_CREATE_WORKINGPART"

C++参考代码段

```
void MainWindow::CreateWorkingPart() {
CComBSTR strCMD = "RO_CMD_CREATE_WORKINGPART";
m_ptrKit->Doc_start_module(strCMD);
}
```

导入自定义零件

<1>执行RO_CMD_IMPORT_PART模块，手动选择

<2>调用接口Doc_insert_custom_define_part传入robp文件的绝对路径，将该零件导入场景，返回零件ID。

HRESULT Doc_insert_custom_define_part(const WCHAR* i_wcRobpPath, ULONG* o_ulPartID)	
SDK: 11.1.0.7310	
Parameters：	
i_wcRobpPath	robp文件的绝对路径
o_ulPartID	返回零件ID

C++参考代码段

```
ULONG ulPartID = 0;
wchar_t whRobpPath[] = L"D:\\Temp\\扫描-1.robp";
m_ptrKit->Doc_insert_custom_define_part(whRobpPath,&ulPartID);
```

定义底座

通过pq_RunCommand执行RO_CMD_CREATE_SEAT命令，来定义底座。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
Parameters：	
i_whModuleID	模块ID，如打开文件"RO_CMD_CREATE_SEAT"

C++参考代码段

```
void MainWindow::CreateSeat() {
CComBSTR strCMD = "RO_CMD_CREATE_SEAT";
m_ptrKit->Doc_start_module(strCMD);
}
```

导入自定义底座

通过pq_RunCommand执行RO_CMD_IMPORT_SEAT命令，bsParam传入自定义底座文件的绝对路径来导入自定义底座。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
Parameters：	
bsCommandID	"RO_CMD_IMPORT_SEAT"
wParam	0或NULL
lParam	0或NULL
bsParam	自定义底座文件的绝对路径
varParam	Empty
lResult	0

C++参考代码段

```
void MainWindow::ImportSeat() {
CComBSTR cmd = " RO_CMD_IMPORT_SEAT ";
CComBSTR bsparam = ""//绝对路径;
long long lresult;
m_ptrKit->pq_RunCommand(cmd, NULL, NULL, bsparam, CCOMVariant(), &lresult);
}
```

判断边是否在面上

此接口用来判断边与面的关系，当且仅当线在面上，且此线不属于面时返回TRUE。根据拾取接口获取的名字（WCHAR*）判断

HRESULT Part_check_edge_in_face_by_name (const WCHAR* i_wcharFaceName, WCHAR i_wcharEdgeName , DOUBLE i_dProjectionLengthThreshold，BOOL* o_bIn)**	
Parameters：	
i_wcharFaceName	传入面的Name
i_wcharEdgeName	传入边的Name
i_dProjectionLengthThreshold	edge在face上投影线段长度与edge长度之间的阈值
o_bIn	传出判断结果

C++参考代码段

```
void MainWindow::CheakEdgeInFace() {
WCHAR* i_wcharFaceName =L“1#9”;
WCHAR* i_wcharEdgeName =L”1#8”;
LONG bIn = 0;
m_ptrKit->Part_check_edge_in_face(i_wcharFaceName, i_wcharEdgeName,0.00, &bIn);
}
```

获取线的两端点位置与切向

根据拾取接口获取的名字（WCHAR*）

HRESULT Part_get_extreme_point_of_edge_by_name (const WCHAR* i_wcharEdgeName, double o_dStartPoint, INT* o_nStartPointSize, double** o_dStartTanVector, INT* o_nStartVectorSize, double** o_dEndPoint, INT* o_nEndPointSize, double** o_dEndTanVector, INT* o_nEndVectorSize)**	
Parameters：	

HRESULT Part_get_extreme_point_of_edge_by_name (const WCHAR* i_wcharEdgeName, double o_dStartPoint, INT* o_nStartPointSize, double** o_dStartTanVector, INT* o_nStartVectorSize, double** o_dEndPoint, INT* o_nEndPointSize, double** o_dEndTanVector, INT* o_nEndVectorSize)**	
i_wEdgeName	输入边的Name
o_dStartPoint	起点位姿数组
o_nStartPointSize	起点位姿数组的长度
o_dStartTanVector	起点切向量数组
o_nStartVectorSize	起点切向量数组长度
o_dEndPoint	终点位姿数组
o_nEndPointSize	终点位姿数组的长度
o_dEndTanVector	终点切向量数组
o_nEndVectorSize	终点切向数组的长度

C++参考代码段

```
void MainWindow::GetExtremePointOfEdge() {
WCHAR* i_wEdgeName =L”1#9”
INT o_nStartPointCount = 0;
INT o_nEndPointCount = 0;
double* o_dStartPoint = nullptr;
double* o_dEndPoint = nullptr;
INT o_nStartPointSize=0;
double* o_dStartTanVector = nullptr;
INT o_nStartVectorSize = 0;
INT o_nEndPointSize = 0;
double* o_dEndTanVector = nullptr;
INT o_nEndVectorSize = 0;
m_ptrKit->Part_get_extreme_point_of_edge(_wcharLineName, &o_dStartPoint, &o_nStartPointCount, &o_dS
m_ptrKit->PQAPIFree((LONG_PTR*) o_dStartPoint;
m_ptrKit->PQAPIFree((LONG_PTR*) o_dEndPoint);
m_ptrKit->PQAPIFree((LONG_PTR*) o_dStartTanVector);
m_ptrKit->PQAPIFree((LONG_PTR*) o_dEndTanVector); }
```

获取零件上所有的面

传出名字,与拾取一致（WCHAR*）,每个Name用逗号“，”分割，需自行拆分

HRESULT Part_get_face_by_name (ULONG i_uPartID, WCHAR o_wFaceNames)**	
Parameters：	
i_uPartID	传入零件的ID
o_wFaceNames	传出面的Nane字符串，需要调用PQAPIFree释放，每个Name用逗号“，”分割，需自行拆分

C++参考代码段

```
void MainWindow::GetFace() {
ULONG uFaceID = /* */;
INT nCount = 0;
```

```
WCHAR* o_wFaceNames =nullptr;
m_ptrKit-> Part_get_face_by_name (uFaceID, & o_wFaceNames);
m_ptrKit->PQAPIFree((LONG_PTR*) o_wFaceNames);
}
```

获取零件上所有的边

传入零件的id,传出零件上所有面的name

HRESULT Part_get_edge _by_name (ULONG i_uPartID, eLoopType i_EedgeType, , WCHAR o_wcharEdgeNames)**	
Parameters：	
i_uPartID	传入零件的ID
i_EedgeType	需要获取边的类型，E_OUTLOOP为外环边，E_INNERLOOP为内环边
o_wcharEdgeNames	传出边的Name字符串，每个Name用逗号“，”分割，边的ID数组需要调用PQAPIFree释放

C++参考代码段

```
 void MainWindow::GetEdge() {
ULONG uID = 0;
GetObjIDByName(16, L"", uID);
eLoopType eType;
INT nCount;
WCHAR* o_wcharEdgeNames =nullptr;
m_ptrKit->Part_get_edge(uID, eType, & o_wcharEdgeNames);
m_ptrKit->PQAPIFree((LONG_PTR*) o_wcharEdgeNames);
}
```

获取面的法向量

传入面的名称（可由3.11接口获取），与面上某一点的世界坐标系位置xyz，传出面上该点法向量，若传入点的xyz不在面上会自动计算距离此点最近的面上的点的法向量。

HRESULT Part_get_face_normal_by_name(const WCHAR* i_wcharFacePartName, DOUBLE i_PointX, DOUBLE i_PointY, DOUBLE i_PointZ, DOUBLE* o_dx, DOUBLE* o_dy, DOUBLE* o_dz)	
Parameters：	
i_wcharFacePartNames	传入面的名称
i_PointX	传入参考点坐标x值
i_PointY	传入参考点坐标y值
i_PointZ	传入参考点坐标z值
o_dx	法向量x方向分量
o_dy	法向量y方向分量
o_dz	法向量z方向分量

C++参考代码段

```
wchar_t whFaceName[] = _T("42#1");
double i_dPointX = 1.2;
```

```
double i_dPointY = 5.24;
double i_dPointZ = 13.2;
double dX = 0.0;
double dY = 0.0;
double dZ = 0.0;
m_ptrKit->Part_get_face_normal(whFaceName,i_dPointX,i_dPointY,i_dPointZ,&dX,&dY,&dZ);
```

获取包含边的所有面

传入传出为线和面的Name（WCHAR*）

HRESULT Part_get_face_of_edge_by_name (const WCHAR* i_wcharEdgeName, WCHAR o_wFaceNames)**	
Parameters：	
i_wcharEdgeName	传入边的Name
o_wFaceNames	传出获取到所有面的Name，每个Name用逗号“，”分割，需要调用PQAPIFreeArray释放

C++参考代码段

```
void MainWindow::GetFaceOfEdge() {
WCHAR* i_wcharEdgeName = L”1#9”;
WCHAR* o_wFaceNames =nullptr;
m_ptrKit->Part_get_face_of_edge(i_wcharEdgeName, & o_wFaceNames);
m_ptrKit->PQAPIFree((LONG_PTR*) o_wFaceNames);
}
```

获取顶点坐标

获取指定对象顶点处的位置，顶点ID由8.5回调函数中传出。8.5的拾取回调中本身也会返回所选中的顶点位置，只是是基于世界坐标系下的描述，本函数增加了一个坐标系指定。

HRESULT Part_get_vertex_position(ULONG i_ulPartID, const WCHAR* i_whVertextName, ULONG i_ulCoordinateID, DOUBLE o_dPosition[3])	
SDK:11.1.0.7442	
Parameters：	
i_ulPartID	指定工件的id
i_whVertextName	顶点ID，即8.5回调函数中传出的实体ID
i_ulCoordinateID	坐标系ID，指定获取基于哪个坐标系下的位置
o_dPosition	指定顶点在指定坐标系下的位置
## 获取射线与曲面交点	

获取自定义射线与指定曲面交点，传出交点坐标和法向。

HRESULT Part_get_ray_surface_intersetion(const WCHAR* i_wSurfaceName, DOUBLE i_dPointCoor[3], DOUBLE i_dPointVec[3], DOUBLE ** o_dIntersetionPoint, INT* o_nArrySize)	
Parameters：	
i_wSurfaceName	指定曲面的实体名（通过7.6.1 PQKit拾取实体回调获取）
i_dPointCoor[3]	自定义射线起点
i_dPointVec[3]	自定义射线方向
o_dIntersetionPoint	计算出的交点坐标和方向，传出格式为x,y,z,dx,dy,dz，交

HRESULT Part_get_ray_surface_intersetion(const WCHAR* i_wSurfaceName, DOUBLE i_dPointCoor[3], DOUBLE i_dPointVec[3], DOUBLE ** o_dIntersetionPoint, INT* o_nArrySize)	
	点的位姿需要调用PQAPIFreeArray释放
o_nArrySize	传出数组个数

C++参考代码段

```
void MainWindow::GetRaySurfaceIntersection() {
CComBSTR whSurfaceName = "";
double dPosition[3] = {
}
;
double dPointVec[3] = {
}
;
double* dIntersetionpoint=nullptr;
int nArrsize = 2;
m_ptrKit->Part_get_ray_surface_intersetion(whSurfaceName, dPosition, dPointVec, &dIntersetionpoint,
m_ptrKit->PQAPIFree((LONG_PTR*)dIntersetionpoint);
}
```

判断点在曲面上

判断定义点在给定阈值范围内是否在指定曲面上

HRESULT Part_cheak_point_on_surface(const WCHAR* i_wSurfaceName, DOUBLE i_dPointCoor[3], DOUBLE i_dTol, BOOL* o_bPointonSuraface)	
Parameters：	
i_wSurfaceName	指定曲面的实体名（通过7.6.1 PQKit拾取实体回调获取）

HRESULT Part_cheak_point_on_surface(const WCHAR* i_wSurfaceName, DOUBLE i_dPointCoor[3], DOUBLE i_dTol, BOOL* o_bPointonSuraface)	
i_dPointCoor[3]	自定义点
i_dTol	阈值
o_bPointonSuraface	传出判断结果

C++参考代码段

```
void MainWindow::CheakPointOnSurface() {
CComBSTR whSurfaceName = "";
double dPosition[3] = {
}
;
double dTol = 2.3;
LONG bPointOnSurface = 0;
m_ptrKit->Part_cheak_point_on_surface(whSurfaceName, dPosition, dTol, &bPointOnSurface);
}
```

零件\场景文件输出

调用Part_export接口可以将零件或者场景文件导出为指定格式文件。通过PQModelType来指定导出的格式。

HRESULT Part_export(ULONG i_ulObjID, ULONG i_ulObjVisualID, WCHAR* i_bsFileName，PQModelType eModelType)	
Parameters：	
i_ulObjID	指定输出对象的位姿数据时参考的机器人的ID。如果指定了机器人ID则基于该指定机器人Base坐标系输出位姿数据；若该值为0，则基于世界坐标系输出位姿数据

HRESULT Part_export(ULONG i_ulObjID, ULONG i_ulObjVisualID, WCHAR* i_bsFileName，PQModelType eModelType)	
i_ulObjVisualID	零件或者装配件的ID
i_bsFileName	点云文件保存路径，如“D:\VisualPLY”,注意无需文件名后缀
eModelType	导出格式类型： PQ_MODEL_STANFORD_TRIANGLE_MESH_PLY 点云 PQ_MODEL_STL STL

C++参考代码段

```
void MainWindow::ExportPLY() {
ULONG ulRobotID = 123;
//示例值，仅作演示
ULONG ulPartID = 456;
//示例值，仅作演示
CComBSTR bsPath = "D:\\Temp";
m_ptrKit->Part_export(ulRobotID,ulPartID, bsPath, PQ_MODEL_STL);
}
```

点云重建后转化为stl文件

3.19.1将点云对象转为stl场景

输入点云对象ID，输出转换后stl对象的ID。

HRESULT Doc_pointcloud_gp3reconstruction (ULONG i_ulObjID, PQGP3ReconParm* i_sGp3ReconPram, ULONG* o_ulMeshObjID)	
Parameters：	
i_ulObjID	输入点云物体的id

HRESULT Doc_pointcloud_gp3reconstruction (ULONG i_ulObjID, PQGP3ReconParm* i_sGp3ReconPram, ULONG* o_ulMeshObjID)	
i_sGp3ReconPram	点云重建算法结构体参数
o_ulMeshObjID	输出stl格式物体的id

后置参数结构体参数含义如下：

Parameters：	
dKRadius	用于确定三角测量中使用的k个最近邻的球体半径，单位为mm
dMu	最近邻距离乘数
iMaxNearestNeighborsNum	搜索的最近邻的最大数量
uKSearch	用于特征估计的k个最近邻的数量

PQGP3ReconParm结构体对象可使用接口Kit_init_gp3reconstructionparm_struct进行初始化。

C++参考代码段

```
void MainWindow:: pointcloudtostl() {
ULONG uIDPointCloud = 0;
GetObjIDByName(225, L"test", uIDPointCloud);
PQGP3ReconParm Gp3ReconPram;
m_ptrKit -> Kit_init_gp3reconstructionparm_struct (&Gp3ReconPram);
Gp3ReconPram.m_dKRadius = 5;
Gp3ReconPram.m_dMu = 5;
Gp3ReconPram.m_iMaxNearestNeighborsNum = 4;
Gp3ReconPram.m_uKSearch = 5;
Ulong uIDStl=0;
m_ptrKit -> Doc_pointcloud_gp3reconstruction(uIDPointCloud, &Gp3ReconPram ,& uIDStl);
}
```

3.19.2将点云对象转为stl保存至本地

HRESULT Doc_save_pointcloud_gp3reconstruction_as_stl (ULONG i_ulObjID, PQGP3ReconParm* i_sGp3ReconPram, LPWSTR i_wsFileName)	
Parameters：	
i_ulObjID	输入点云物体的id
i_sGp3ReconPram	点云重建算法结构体参数
i_wsFileName	保存的文件路径，后缀名需要为.stl

后置参数结构体参数含义同上

C++参考代码段

```
void MainWindow:: pointcloudtostl() {
ULONG uIDPointCloud = 0;
GetObjIDByName(225, L"test", uIDPointCloud);
PQGP3ReconParm Gp3ReconPram;
m_ptrKit -> Kit_init_gp3reconstructionparm_struct (&Gp3ReconPram);
Gp3ReconPram.m_dKRadius = 5;
Gp3ReconPram.m_dMu = 5;
Gp3ReconPram.m_iMaxNearestNeighborsNum = 4;
Gp3ReconPram.m_uKSearch = 5;
m_ptrKit -> Doc_save_pointcloud_gp3reconstruction_as_stl (uIDPointCloud, &Gp3ReconPram ,       (LPWS
}
```

机构抓取工件

机构抓取工件，绑定工件，输入机构对象ID和工件对象的ID

HRESULT Doc_catch_obj(ULONG i_uEngineID, ULONG i_ulPartID)	
Parameters：	

HRESULT Doc_catch_obj(ULONG i_uEngineID, ULONG i_ulPartID)	
i_uEngineID	机器人的id
i_ulPartID	机器人抓取的工件id
void MainWindow:: catchObject() { ULONG uIDRobot = 0; GetObjIDByName(32, L"", uIDRobot); ULONG uIDCatchedObj =0; ULONG uIDTool=0; m_ptrKit ->Robot_get_tool(uIDRobot, & uIDTool); m_ptrKit ->Doc_catch_obj(uIDTool, uIDCatchedObj); }	
## 机构放开工件	

机构放开工件，取消绑定工件，输入机构对象ID和工件对象的ID

HRESULT Doc_release_obj(ULONG i_uEngineID, ULONG i_ulPartID)	
Parameters：	
i_uEngineID	机器人的id
i_ulPartID	机器人抓取的工件id
void MainWindow:: releaseObject() { ULONG uIDRobot = 0; GetObjIDByName(32, L"", uIDRobot); ULONG uIDCatchedObj =0; ULONG uIDTool=0; m_ptrKit ->Robot_get_tool(uIDRobot, & uIDTool); m_ptrKit ->Doc_release_obj(uIDTool, uIDCatchedObj); }	
## 点云提取边界	

点云提取边界，输入点云对象ID,并设置边界提取算法参数值，输出边界曲线对象ID和数量。之后可通过Doc_create_path_from_curve接口拾取曲线对象生成机器人运动轨迹

HRESULT Doc_create_curve_from_pointcloud(ULONG i_ulObjID,PQEdgeExtraType i_eEdgeExtraType, PQEdgeParm* i_sEdgeParm, ULONG o_ulEdgeObjAIDs, INT* o_nObjCount)**	
Parameters：	
i_ulObjID	输入点云物体的id
i_eEdgeExtraType	边界提取算法类型，此处设置为0
i_sEdgeParm	边界提取算法参数结构体
o_ulEdgeObjAIDs	输出边界曲线ID数组，数据在PQKit内部分配内存，需要调用PQAPIFreeArray释放
o_nObjCount	输出边界曲线个数

边界提取算法结构体参数含义如下，可使用Kit_init_pointcloud_edgeparm_struct进行初始化：

Parameters：	
sASEdgeParm	AShape边界提取算法参数，此处仅设置sASEdgeParm即可 sASEdgeParm.dAlpha:提取边界半径Alpha值; sASEdgeParm.dDistanceThreshold:分割平面最大距离阈值； sASEdgeParm.iMaxIterations：分割平面最大迭代次数 sASEdgeParm.dRadiusSearch:边界点分类搜索半径； sASEdgeParm.iMinClusterSize:边界点分类有效所包含最小点数； sASEdgeParm.iMaxClusterSize: 边界点分类有效所包含最大点数； sASEdgeParm.fSmoothnessThreshold:点云分类法向之间的最大允许差值； sASEdgeParm.fCurvatureThreshold:点云分类的曲率阈值； sASEdgeParm.uiNumberOfNeighbours:分类搜索邻域数； sASEdgeParm.fResidualThreshold:同点云类最大距离
sBoundaryParm	曲率边界提取算法参数 sBoundaryParm.dRadiusSearch:法向量搜索半径

C++参考代码段

```
void MainWindow::getpointcloudedge() {
ULONG uObjID = 0;
GetObjIDByName(225, L"test", uObjID);
struct_PQEdgePram sEdgeParm;
m_ptrKit ->Kit_init_pointcloud_edgeparm_struct(&sEdgeParm);
sEdgeParm.sASEdgeParm.dAlpha = 10;
sEdgeParm.sASEdgeParm.dDistanceThreshold = 25;
sEdgeParm.sASEdgeParm.iMinClusterSize = 100;
int num = 0;
ULONG* ulEdgeObjAIDs = nullptr;
m_ptrKit ->Doc_create_curve_from_pointcloud (uObjID, PQ_ASHAPE, &sEdgeParm, ulEdgeObjAIDs,        &n
m_ptrKit->PQAPIFreeArray((LONG_PTR*)ulEdgeObjAIDs);
return;
}
```

零件仿真

PQKit提供了一系列接口支持用户指定零件和路径数据进行零件的单独仿真。

整体使用流程需要先设置零件对象和路径数据，支持设置多零件；然后初始化仿真器；再启动仿真。在仿真未退出状态下可以指定零件上的某个路径点序号，来仿真到该点。

<1>设置仿真的零件和路径点

HRESULT Doc_custom_simulate_set_data(ULONG i_ulObjID, INT i_nPointCount, DOUBLE* i_dPointPosture, PQPostureType i_ePostureType)	
SDK:11.1.0.7442	
Parameter:	
i_ulObjID	仿真的零件ID
i_nPointCount	仿真的路径点的个数
i_dPointPosture	仿真的路径点位姿数据，以欧拉角为例，每

HRESULT Doc_custom_simulate_set_data(ULONG i_ulObjID, INT i_nPointCount, DOUBLE* i_dPointPosture, PQPostureType i_ePostureType)	
	个点的XYZABC按序排列即可
i_ePostureType	指定姿态表示方式：欧拉角、四元素，详见9.2位姿描述定义

C++参考代码段

```
ULONG ulPartA = 0;
GetObjIDByName(PQ_WORKINGPART, _T("工作台"), ulPartA);
//名称以实际项目为准 int nAPtCount = 20;
double* dPointDataA = new double[nAPtCount * 6];
for (int i = 0; i < nAPtCount; i++) { dPointDataA[0 + 6 * i] = 895.0000; dPointDataA[1 + 6 * i] = -
m_ptrKit->Doc_custom_simulate_set_data(ulPartA, nAPtCount, dPointDataA, EULERANGLEXYZ);
ULONG ulPartB = 0;
GetObjIDByName(PQ_WORKINGPART, _T("扫描件"), ulPartB);
//名称以实际项目为准 int nBPtCount = 200;
double* dPointDataB = new double[nBPtCount*6];
for (int i = 0; i < nBPtCount; i++) { dPointDataB[0 + 6 * i] = 0.0000; dPointDataB[1 + 6 * i] = 0.0
m_ptrKit->Doc_custom_simulate_set_data(ulPartB, nBPtCount, dPointDataB, EULERANGLEXYZ);
m_ptrKit->Doc_custom_simulate_initialize(FALSE, TRUE);
delete[] dPointDataA;
dPointDataA = nullptr;
delete[] dPointDataB;
dPointDataA = nullptr;
```

<2>初始化仿真器

HRESULT Doc_custom_simulate_initialize(BOOL i_bShowSimControlDlg, BOOL i_bEnableOutput)	
SDK:11.1.0.7442	
Parameter:	

HRESULT Doc_custom_simulate_initialize(BOOL i_bShowSimControlDlg, BOOL i_bEnableOutput)	
i_bShowSimControlDlg	是否显示仿真控制面板
i_bEnableOutput	是否显示仿真日志输出窗口

<3>仿真启动、暂停、停止

仿真启动：Doc_custom_simulate_start

仿真暂停：Doc_custom_simulate_stop

仿真停止：Doc_custom_simulate_end

<4>仿真到某一点

HRESULT Doc_custom_simulate_simulate_to_point(ULONG i_ulObjID, INT i_nPointIndex)	
SDK:11.1.0.7442	
Parameter:	
i_ulObjID	要仿真到的路径点对应的对象
i_nPointIndex	要仿真到的路径点在这个对象的路径点序列中的序号，从0计数

C++参考代码段

```
ULONG ulPartB = 0;
GetObjIDByName(PQ_WORKINGPART, _T("扫描件"), ulPartB);
m_ptrKit->Doc_custom_simulate_simulate_to_point(ulPartB,10);
```

获取边几何类型

调用Part_get_edge_type接口获取指定边的类型。

HRESULT Part_get_edge_type(const WCHAR* i_wsEdgeName, PQEdgeType* o_eEd geType)	
SDK:11.1.0.7519	
Parameter:	
i_wsEdgeName	边信息，可由 3.12 接口获取或者拾取命令中获取
o_eEdgeType	边类型，参见 9.10
## 获取圆弧边半径	

调用 Part_get_circle_radius 接口获取指定圆弧的半径。

HRESULT Part_get_circle_radius(const WCHAR* i_wsEdgeName, double* o_dRa dius)	
SDK:11.1.0.7519	
Parameter:	
i_wsEdgeName	边信息，可由 3.12 接口获取或者拾取命令中获取
o_dRadius	圆弧半径
## 获取直线边长	

HRESULT Part_get_straight_line_length(const WCHAR* i_wsEdgeName, double * o_dLength)	
SDK:11.1.0.7519	
Parameter:	

HRESULT Part_get_straight_line_length(const WCHAR* i_wsEdgeName, double * o_dLength)	
i_wsEdgeName	边信息，可由 3.12 接口获取或者拾取命令中获取
o_dLength	直线段长度
## 几何边离散位置点	

调用Part_get_discrete_point_position 接口在指定圆弧或者直线上按条件离散出点位数据，不包含点位位姿姿态。

HRESULT Part_get_discrete_point_position(const ULONG i_ulPartID, const WCHAR* i_wsEdgeName, PQDiscreteType i_eDiscreteType, INT i_nPointCount, DOUBLE i_dStep, DOUBLE i_dInterferenceH, DOUBLE i_dInterferenceT, INT*o_nReturnPtCount, DOUBLE o_dPtPosition, DOUBLE** o_dPtTangent)**	
SDK:11.1.0.7519	
Parameter:	
i_ulPartID	边所在的工件的 ID
i_wsEdgeName	边信息，可由 3.12 接口获取或者拾取命令中获取
i_eDiscreteType	离散类型，由枚举 PQDiscreteType 控制 PQ_DISCRETE_BY_STEP 按步长离散 PQ_DISCRETE_BY_NUM按个数离散 PQ_DISCRETE_AUTO 自动离散
i_nPointCount	按个数离散的个数
i_dStep	按步长离散的步长

HRESULT Part_get_discrete_point_position(const ULONG i_ulPartID, const WCHAR* i_wsEdgeName, PQDiscreteType i_eDiscreteType, INT i_nPointCount, DOUBLE i_dStep, DOUBLE i_dInterferenceH, DOUBLE i_dInterferenceT, INTo*_nReturnPtCount, DOUBLE o_dPtPosition, DOUBLE** o_dPtTangent)**	
i_dInterferenceH	起点内偏长度
i_dInterferenceT	终点内偏长度
o_nReturnPtCount	返回离散的点个数
o_dPtPosition	返回离散的点的位置数据，按 x、y、z 顺序排序。需要调用 PQAPIFree Array 释放内存
o_dPtTangent	返回离散点的切向量，按 x、y、z 顺序排序。需要调用 PQAPIFreeArr ay 释放内存

C++参考代码段

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, _T("Panason_TM_2000"), ulRobotID);
ULONG uPartID = 0;
GetObjIDByName(PQ_ACCESSORY_PART, _T("TestAcc"), uPartID);
// wchar_t* wsEdgeNames = nullptr;
m_ptrKit->Part_get_edge_by_name(uPartID, E_OUTLOOP, &wsEdgeNames);
std::wstring t_wsEdges = wsEdgeNames;
std::vector<std::wstring> t_vEdges;
//splitString(t_wsEdges, t_vEdges);
自行按逗号切片
m_ptrKit->PQAPIFreeArray((LONG_PTR*)wsEdgeNames);
// std::wstring whTargetEdge;
for (auto t_whEdge : t_vEdges) {
wchar_t* whTempEdge = const_cast<wchar_t*>(t_whEdge.c_str());
PQEdgeType eType = PQ_CURVE_STRAIGNT;
m_ptrKit->Part_get_edge_type(whTempEdge, &eType);
if (PQ_CURVE_CIRCLE == eType) {
whTargetEdge = t_whEdge;
```

```
break;
}
}
int nPtCount = 0;
double* dPosition = nullptr;
double* dTangent = nullptr;
m_ptrKit->Part_get_discrete_point_position(uPartID, const_cast<wchar_t*>(whTargetEdge.c_ str()), PQ
double* dPtPosture = new double[nPtCount * 6];
for (int i = 0; i < nPtCount; i++) { dPtPosture[i * 6] = dPosition[i * 3]; dPtPosture[i * 6 + 1] =
ULONG uPathID = 0;
m_ptrKit->Path_insert_from_point_with_sameproperty(ulRobotID, nPtCount, dPtPosture, EULE RANGLEXYZ,
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dPosition);
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dTangent);
```

获取标准圆弧信息

调用Part_get_circle_edge_info 接口可以获取指定标准圆弧的相关信息，包含半径、圆心位置、起始角、终止角、圆所在平面法向量、X轴向量。

HRESULT Part_get_circle_edge_info(const WCHAR* i_wsEdgeName, DOUBLE* o_dRadius, DOUBLE* o_dCircleCenterX, DOUBLE* o_dCircleCenterY, DOUBLE* o_dCircleCenterZ, DOUBLE* o_dCircleStartAngle, DOUBLE* o_dCircleEndAngle, DOUBLE* o_dNormalX, DOUBLE* o_dNormalY, DOUBLE* o_dNormalZ, DOUBLE* o_dXNormalX, DOUBLE* o_dXNormalY, DOUBLE* o_dXNormalZ)	
SDK: 11.1.0.7592	
Parameters：	
i_wsEdgeName	边信息，可由 3.12 接口获取或者拾取命令中获取
o_dRadius	圆弧半径

HRESULT Part_get_circle_edge_info(const WCHAR* i_wsEdgeName, DOUBLE* o_dRadius, DOUBLE* o_dCircleCenterX, DOUBLE* o_dCircleCenterY, DOUBLE* o_dCircleCenterZ, DOUBLE* o_dCircleStartAngle, DOUBLE* o_dCircleEndAngle, DOUBLE* o_dNormalX, DOUBLE* o_dNormalY, DOUBLE* o_dNormalZ, DOUBLE* o_dXNormalX, DOUBLE* o_dXNormalY, DOUBLE* o_dXNormalZ)	
o_dCircleCenterX	圆心位置X
o_dCircleCenterY	圆心位置Y
o_dCircleCenterZ	圆心位置Z
o_dCircleStartAngle	起始角，角度单位
o_dCircleEndAngle	终止角，角度单位
o_dNormalX	法向量X
o_dNormalY	法向量Y
o_dNormalZ	法向量Z
o_dXNormalX	X轴向量X
o_dXNormalY	X轴向量Y
o_dXNormalZ	X轴向量Z

C++参考代码段

```
wchar_t whEdge[] = _T("79#38");
double dRadius = 0.0;
double dCircleCenterX = 0.0;
double dCircleCenterY = 0.0;
double dCircleCenterZ = 0.0;
double dCircleStartAngle = 0.0;
double dCircleEndAngle = 0.0;
double dNormalX = 0.0;
double dNormalY = 0.0;
double dNormalZ = 0.0;
```

```
double dXNormalX = 0.0;
double dXNormalY = 0.0;
double dXNormalZ = 0.0;
m_ptrKit->Part_get_circle_edge_info(whEdge, &dRadius, &dCircleCenterX, &dCircleCenterY, &dCircleCen
```

装配场景

输入同级的场景ID，能将其装配在一起

HRESULT Part_assembly(ULONG* i_ulAccPartID, INT i_nAccPartSize,const WCHAR* i_wsAccPartName)	
SDK: 11.1.0.7666	
Parameters：	
i_ulAccPartID	需要装配在一起的场景ID
i_nAccPartSize	需要装配场景的个数
i_wsAccPartName	装配名字

C++参考代码段

```
 ULONG ulAssPartID = 0;
GetObjIDByName(PQ_ACCESSORY_PART, L"HBM_XJ03_04_OP10_DR_NC_igs001", ulAssPartID);
ULONG ulAssPartID2 = 0;
GetObjIDByName(PQ_ACCESSORY_PART, L"PartFrame", ulAssPartID2);
ULONG ulTotalPartID[2]={
ulAssPartID2,ulAssPartID}
;
CComBSTR bsName = "装配1";
m_ptrKit->Part_assembly(ulTotalPartID, 2, bsName);
```

获取工件包围盒

调用Part_get_boundingbox_point 接口，传入场景或工件ID可以获取该对象的包围盒极值点、顶点数据。

HRESULT Part_get_boundingbox_point(const ULONG i_ulPartID, DOUBLE o_dMin[3], DOUBLE o_dMax[3], DOUBLE o_dVertex[24])	
SDK: 11.2.0.7732	
Parameters：	
i_ulPartID	场景或者工件ID
o_dMin	极小值点
o_dMax	极大值点
o_dVertex	包含极值点的24个顶点数据，目视平面坐标系的方向，按右上后、右上前、左上后、左上前、左下后、左下前、右下后、右下前顺序排序

C++参考代码段

```
double dMin[3] = {
0.0 }
;
double dMax[3] = {
0.0 }
;
double dVertex[24] = {
0.0 }
;
m_ptrKit->Part_get_boundingbox_point(ulPartID,dMin,dMax,dVertex);
```

获取面几何类型

调用Part_get_face_type 接口可以获取指定面的类型。

HRESULT Part_get_face_type(const WCHAR* i_wchFaceName, PQFaceType* o_eFaceType)	
SDK: 11.2.0.7788	
Parameters：	
i_wchFaceName	边信息，可由 3.11 接口获取或者拾取命令中获取
o_eFaceType	面类型，参加9.11
## 工件插入pos点	

调用Part_insert_point_with_posture给指定工件插入轨迹。

HRESULT Part_insert_point_with_posture(ULONG i_uPart, INT i_nPtCount, DOUBLE* i_dPosition, PQPostureType i_ePostureType, DOUBLE i_dVelocity, DOUBLE i_dpalvel, INT* i_nApproach, const WCHAR* i_PathName, const WCHAR* i_GroupName, ULONG i_uCoordinateID, BOOL i_bIsUpdate, ULONG* i_uPathID)	
SDK: 11.2.0.7788	
Parameters：	
i_uPart	零件id
i_nPtCount	插入点的个数
i_dPosition	插入轨迹点位姿数组
i_ePostureType	指定i_dPosition的姿态表达方式，如四元数、欧拉角XYZ、欧拉角ZYX等等，详见9.2 位姿描述定义
i_dVelocity	轨迹速度

HRESULT Part_insert_point_with_posture(ULONG i_uPart, INT i_nPtCount, DOUBLE* i_dPosition, PQPostureType i_ePostureType, DOUBLE i_dVelocity, DOUBLE i_dpalvel, INT* i_nApproach, const WCHAR* i_PathName, const WCHAR* i_GroupName, ULONG i_uCoordinateID, BOOL i_bIsUpdate, ULONG* i_uPathID)	
i_dpalvel	轨迹角速度
i_nApproach	轨迹逼近方式数组，精准到点为-1
i_PathName	轨迹名字
i_GroupName	轨迹点名字
i_uCoordinateID	参考坐标系id，世界坐标系为0
i_bIsUpdate	插入轨迹后是否执行update
i_uPathID	传出轨迹id

C++参考代码段

```
ULONG uPartID = 0;
double i_dPosition[6] = {
0.0 }
;
PQPostureType i_ePostureType = EULERANGLEXYZ;
DOUBLE i_dVelocity = 30;
DOUBLE i_dpalvel = 0.1;
INT i_nApproach[1] = {
-1 }
;
ULONG uPathID = 0;
m_ptrKit->Part_insert_point_with_posture(uPartID,1,i_dPosition,i_ePostureType, i_dVelocity, i_dpalv
return S_OK;
```

工具

获取/设置TCP位姿

4.1.1获取工具TCP位姿

HRESULT Tool_get_tcp_posture(ULONG i_ulObjID, const WCHAR* i_chTcpName,PQPostureType i_ePostureType, INT *o_nPostureCount, DOUBLE **o_dTcpPosture)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameter:	
i_ulObjID	欲获取TCP的工具ID
i_chTcpName	指定TCP名称，如果不指定，则获取当前工具在用TCP
i_ePostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义
o_nPostureCount	返回TCP数据的数组长度
o_dTcpPosture	TCP位姿数据,需要调用PQAPIFreeArray释放内存

C++参考代码段

```
ULONG ulToolID = 0;
GetObjIDByName(PQ_TOOL, _T(""), ulToolID);
int nCount = 0;
double* dTcpPosture = nullptr;
wchar_t wcTcpName[] = _T("");
m_ptrKit->Tool_get_tcp_posture(ulToolID, wcTcpName, EULERANGLEXYZ, &nCount, &dTcpPosture);
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dTcpPosture);
```

4.1.2设置工具TCP位姿

HRESULT Tool_set_tcp_posture(ULONG i_ulObjID, const WCHAR* i_chTcpName,DOUBLE *i_dTcpPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQPostureType 最低支持版本)	
Parameter:	
i_ulObjID	欲指定TCP的工具ID
i_chTcpName	指定TCP名称，如果不指定，则设置当前工具在用TCP
i_dTcpPosture	TCP数据数组
i_nPostureArraySize	TCP数据数组大小
i_ePostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义

C++参考代码段

```
ULONG ulToolID = 0;
GetObjIDByName(PQ_TOOL, _T(""), ulToolID);
double dTcpPosture[] = {
90.0,-45.0,60.0,45.0,30.0,60.0}
;
wchar_t wcTcpName[] = _T("TCP");
m_ptrKit->Tool_set_tcp_posture(ulToolID, wcTcpName, dTcpPosture, 6, EULERANGLEZYX);
```

定义工具

PQKit支持2种形式来定义工具，一是执行定义工具模块进行界面操作定义工具；二是通过Doc_define_custom_fl_tool来无界面定义FL工具。

<1>通过Doc_start_module执行RO_CMD_CREATE_TOOL命令，来定义工具。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
SDK:9.2.0.6420 (函数本体最低支持版本)	
Parameters：	
i_whModuleID	模块ID，如打开文件"RO_CMD_CREATE_TOOL"

C++参考代码段

```
CComBSTR strCMD = "RO_CMD_CREATE_TOOL";
m_ptrKit->Doc_start_module(strCMD);
```

<2>调用Doc_define_custom_fl_tool接口定义FL工具。使用该接口定义FL功能需要先将三维模型导入成场景并获取场景ID，导入成场景可参考Doc_import_accessory_part接口。

HRESULT Doc_define_custom_fl_tool(const ULONG i_ulPartID, DOUBLE* i_dFLPosture,INT i_nFLPostureSize, PQPostureType i_eFLPostureType, DOUBLE* i_dTCPPosture, INT i_nTCPPostureSize, PQPostureType i_eTCPPostureType, const WCHAR* i_wsSavePath, ULONG* o_ulToolID)	
SDK:11.2.0.7544	
Parameters：	
i_ulPartID	要定义成FL工具的场景对象ID
i_dFLPosture	FL点位姿数组，该点决定了FL工具吸附到机器人FL盘上的位姿

HRESULT Doc_define_custom_fl_tool(const ULONG i_ulPartID, DOUBLE* i_dFLPosture,INT i_nFLPostureSize, PQPostureType i_eFLPostureType, DOUBLE* i_dTCPPosture, INT i_nTCPPostureSize, PQPostureType i_eTCPPostureType, const WCHAR* i_wsSavePath, ULONG* o_ulToolID)	
i_nFLPostureSize	FL点位姿数组大小
i_eFLPostureType	FL点位姿姿态表示：欧拉角、四元素，详见9.2 位姿描述定义
i_dTCPPosture	TCP点位姿数组，如不指定TCP位姿数据的话，将默认按全0处理
i_nTCPPostureSize	TCP点位姿数组大小
i_eTCPPostureType	TCP点位姿姿态表示
i_wsSavePath	保存成robt工具的目录，如不指定则不保存文件
o_ulToolID	返回定义的FL工具ID
## 导入自定义工具	

<1>执行RO_CMD_IMPORT_TOOL模块，手动选择

<2>调用Doc_insert_custom_define_tool接口传入robt文件的绝对路径，将该工具导入场景，返回工具ID。

HRESULT Doc_insert_custom_define_tool(const WCHAR* i_wcRobtPath, BOOL i_bReplaceFLWithoutSelectTool, BOOL i_bShowProgress, ULONG* o_ulToolID)	
SDK:11.1.0.7352	
Parameters：	
i_wcRobtPath	robt文件的绝对路径
i_bReplaceFLWithoutSelectTool	当前未选中工具情况下是否替换FL工具，如果选中了FL工具，则该参数无论TRUE或者FALSE都将替换FL工具
i_bShowProgress	是否显示导入进度
o_ulToolID	返回工具ID

C++参考代码段

```
ULONG ulToolID = 0;
wchar_t whPath[] = L"D:\\Temp\\Model\\TwoKong-FL.robt";
m_ptrKit->Doc_insert_custom_define_tool(whPath, 1, 0, &ulToolID);
```

快换工具安装/卸载

4.4.1改变状态-无轨迹

通过pq_RunCommand执行RO_CMD_CATCH_TOOL_CUSTOM命令来执行快换工具的安装，执行RO_CMD_RELESE_TOOL_CUSTOM命令来执行快换工具的卸载，其中wParam传入快换工具的ID。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:9.2.0. 6420 (函数本体最低支持版本)	
Parameters：	
bsCommandID	"RO_CMD_CATCH_TOOL_CUSTOM"或者"RO_CMD_RELESE_TOOL_CUSTOM"
wParam	快换工具ID
lParam	0或NULL
bsParam	""
varParam	Empty
lResult	0

C++参考代码段

```
void MainWindow::CatchToolCustom() {
CComBSTR cmd = " RO_CMD_CATCH_TOOL_CUSTOM ";
long long wParam;
CComBSTR bsparam = "";
long long lresult;
m_ptrKit->pq_RunCommand(cmd, wParam, NULL, bsparam, CCOMVariant(), &lresult);
}
```

4.4.2生成轨迹

通过pq_RunCommand执行RO_CMD_CATCH_TOOL命令来执行快换工具的安装，执行RO_CMD_RELESE_TOOL命令来执行快换工具的卸载，其中wParam传入快换工具的ID。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:9.2.0.6420 (函数本体最低支持版本)	
Parameters：	
bsCommandID	"RO_CMD_CATCH_TOOL"或者"RO_CMD_RELESE_TOOL"
wParam	快换工具ID
lParam	0或NULL
bsParam	""
varParam	Empty
lResult	0

C++参考代码段

```
void MainWindow::CatchTool() {
CComBSTR cmd = "RO_CMD_CATCH_TOOL";
long long wParam;
CComBSTR bsparam = "";
long long lresult;
m_ptrKit->pq_RunCommand(cmd, wParam, NULL, bsparam, CCOMVariant(), &lresult);
}
```

获取工具所有TCP

HRESULT Tool_get_tcp(ULONG i_ulID, INT *o_nTcpCount , WCHAR **o_chTcpNames, INT **o_nTcpNameLengthArray)	
SDK:9.2.0.6420 (函数本体最低支持版本)	
Parameter:	

HRESULT Tool_get_tcp(ULONG i_ulID, INT *o_nTcpCount , WCHAR **o_chTcpNames, INT **o_nTcpNameLengthArray)	
i_ulID	指定工具ID
nTcpCount	返回工具下TCP的数量
o_chTcpNames	返回TCP名字的字符串，多个TCP字符串会拼接成一个字符串，需根据每个tcp名称长度的数组进行分割，o_chTcpNames数据在PQKit内部分配内存，需要调用PQAPIFreeArray释放
o_nTcpNameLengthArray	返回每个tcp名称长度的数组，o_nTcpNameLengthArray数据在PQKit内部分配内存，需要调用PQAPIFreeArray释放

例如，工具tool1下有2个tcp，分别为TCP0、TCP123，则api调用结果中nTcpCount值为2，o_chTcpNames值为"TCP0TCP123", o_nTcpNameLengthArray的值为{4,6}。

C++参考代码段

```
ULONG uID = 0;
INT nTcpCount = 0;
WCHAR* whTcpName= nullptr;
INT* nTCPNameLengthArr;
m_ptrKit->Tool_get_tcp(uID, &nTcpCount, &whTcpName, &nTCPNameLengthArr);
m_ptrKit->PQAPIFree((LONG_PTR*)whTcpName);
m_ptrKit->PQAPIFree((LONG_PTR*)nTCPNameLengthArr);
```

轨迹

对于PQKit仍然可以通过弹出轨迹生成面板来生成轨迹，并增加了轨迹生成的回调。详见7.3节生成轨迹。

轨迹点生成轨迹

PQKit提供由轨迹点数据生成轨迹的能力。通过接口一次可传入一个或者多个轨迹点的位姿、点指令、线速度、速度百分比、该点处机器人逼近方式（过渡方式）等参数生成PQKit的轨迹对象，并返回轨迹ID。其中Path_insert_from_point允许每个点的点指令、线速度、速度百分比、逼近方式参数各不相同。如一批插入的轨迹点的点指令、线速度、速度百分比、逼近方式参数一样，建议使用Path_insert_from_point_with_sameproperty接口。该接口较Path_insert_from_point有明显性能提升。

5.1.1 Path_insert_from_point

HRESULT Path_insert_from_point(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, PQPostureType i_ePostureType, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, ULONG* o_PathID,BOOL i_bIsUpdate)	
SDK:9.2.0.6420(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameters：	
i_ulRobotID	指定生成的轨迹归属于哪个机器人
i_nPtCount	指定轨迹点个数
i_dPosition	轨迹点数据集合，按序排列。
i_ePostureType	指定i_dPosition的姿态表达方式，如四元数、欧拉角XYZ、欧拉角ZYX等等，详见9.2 位姿描述定义
i_eInstruct	轨迹点运动指令，按序排列,详见9.3 轨迹点指令描述定义
i_dVelocity	轨迹点速度，按序排列

HRESULT Path_insert_from_point(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, PQPostureType i_ePostureType, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, ULONG* o_PathID,BOOL i_bIsUpdate)	
i_dSpeedPercent	轨迹点速度百分比，按序排列
i_nApproach	轨迹点轨迹逼近，按序排列
i_PathName	生成的轨迹名称
i_GroupName	生成的轨迹所属轨迹组名称
i_uCoordinateID	传入的点的位姿的参考坐标系，如点位姿基于世界坐标系则传0
i_bToolEndPosture	表示输入的位姿是否是工具末端位姿
o_PathID	返回生成的轨迹ID
i_bIsUpdate	插入点后是否更新，如果多次循环调用将i_bIsUpdate设置为False，在完成所有轨迹点插入后调用Doc_update_view更新一次即可

注：

a.i_dPosition：如轨迹点姿态指定为欧拉角EULERANGLEZYX，指定了3个点的数据，那此处应传入X1、Y1、Z1、A1、B1、C1、X2、Y2、Z2、A2、B2、C2、X3、Y3、Z3、A3、B3、C3。其中A就是绕Z轴旋转的角度（度数）、B就是绕Y轴旋转的角度（度数）、C就是绕X轴旋转的角度（度数）。

b. i_nApproach:-1表示精确到点

c. i_bToolEndPosture:如果传入的是工具末端姿态，就将i_bToolEndPosture置TRUE。

C++参考代码段

```
ULONG uRobotID = 0;
GetObjIDByName(PQ_ROBOT, _T(""), uRobotID);
// double dPosition[12] = {
989.0000,281.0000,452.0000,0.0000,0.0000,90.0000, 989.0000,181.0000,452.0000,0.0000,0.0000,90.0000
;
PQPostureType ePosType = EULERANGLEXYZ;
PQPointInstruction nInstruction[2] = {
PQ_LINE,PQ_LINE }
;
double dVel[2] = {
200.0,200.0 }
;
double dSpeedP[2] = {
100.0,100.0 }
;
int nApproach[2] = {
-1,-1 }
;
wchar_t whPathName[] = _T("Test_Path");
wchar_t whPathGroupName[] = _T("Test_Group");
ULONG uPathID = 0;
m_ptrKit->Path_insert_from_point(uRobotID, 2, dPosition, ePosType, nInstruction, dVel, dSpeedP, nAp
```

5.1.2 Path_insert_from_point_with_sameproperty

HRESULT Path_insert_from_point_with_sameproperty(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, PQPostureType i_ePostureType, PQPointInstruction i_eInstruct, DOUBLE i_dVelocity, DOUBLE i_dSpeedPercent, INT i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, BOOL i_bIsUpdate, ULONG* o_PathID)	
SDK: 10.0.0.7068(函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	

HRESULT Path_insert_from_point_with_sameproperty(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, PQPostureType i_ePostureType, PQPointInstruction i_eInstruct, DOUBLE i_dVelocity, DOUBLE i_dSpeedPercent, INT i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, BOOL i_bIsUpdate, ULONG* o_PathID)	
Parameters：	
i_ulRobotID	指定生成的轨迹归属于哪个机器人
i_nPtCount	指定轨迹点个数
i_dPosition	轨迹点数据集合，按序排列。
i_ePostureType	指定i_dPosition的姿态表达方式，如四元数、欧拉角XYZ、欧拉角ZYX等等，详见9.2 位姿描述定义
i_eInstruct	轨迹点运动指令，所有点一致
i_dVelocity	轨迹点速度，所有点一致
i_dSpeedPercent	轨迹点速度百分比，所有点一致
i_nApproach	轨迹点轨迹逼近，所有点一致
i_PathName	生成的轨迹名称

HRESULT Path_insert_from_point_with_sameproperty(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, PQPostureType i_ePostureType, PQPointInstruction i_eInstruct, DOUBLE i_dVelocity, DOUBLE i_dSpeedPercent, INT i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, BOOL i_bIsUpdate, ULONG* o_PathID)	
i_GroupName	生成的轨迹所属轨迹组名称
i_uCoordinateID	传入的点的位姿的参考坐标系，如点位姿基于世界坐标系则传0
i_bToolEndPosture	表示输入的位姿是否是工具末端位姿
i_bIsUpdate	表示是否更新显示
o_PathID	返回生成的轨迹ID

C++参考代码段

```
ULONG uRobotID = 0;
GetObjIDByName(PQ_ROBOT, _T(""), uRobotID);
// double dPosition[12] = {
989.0000,281.0000,452.0000,0.0000,0.0000,90.0000, 989.0000,181.0000,452.0000,0.0000,0.0000,90.0000
;
PQPostureType ePosType = EULERANGLEXYZ;
PQPointInstruction nInstruction = PQ_LINE ;
double dVel = 200.0;
double dSpeedP = 100.0;
int nApproach = -1;
wchar_t whPathName[] = _T("Test_Path");
wchar_t whPathGroupName[] = _T("Test_Group");
```

```
ULONG uPathID = 0;
m_ptrKit->Path_insert_from_point_with_sameproperty(uRobotID, 2, dPosition, ePosType, nInstruction,
```

5.1.3 Path_insert_from_point_with_external_joint带外部轴插入轨迹

HRESULT Path_insert_from_point_with_external_joint(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, ULONG* i_ExtJointIDs, INT i_nExtJointCount, DOUBLE* i_dExtValue, INT i_nExtValueCount, PQPostureType i_ePostureType, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, ULONG* o_PathID, BOOL i_bIsUpdate)	
SDK: 11.2.0.7721 (函数本体最低支持版本) 11.1.0.7310(PQPostureType最低支持版本)	
Parameters：	
i_ulRobotID	指定生成的轨迹归属于哪个机器人
i_nPtCount	指定轨迹点个数
i_dPosition	轨迹点数据集合，按序排列。
i_ExtJointIDs	外部轴关节id数组
i_nExtJointCount	外部轴关节id数组长度
i_dExtValue	外部轴对应位置数组，旋转轴单位为弧度，传入顺序为：外部轴1第一个点所有轴数据、外部轴2第1个点所有轴数据...外部轴1第2个点所有轴数据、外部轴2第2个点所有轴数据...以此类推，

HRESULT Path_insert_from_point_with_external_joint(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, ULONG* i_ExtJointIDs, INT i_nExtJointCount, DOUBLE* i_dExtValue, INT i_nExtValueCount, PQPostureType i_ePostureType, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, ULONG* o_PathID, BOOL i_bIsUpdate)	
	外部轴得顺序与传入i_ExtJointIDs的顺序保持一致。
i_ePostureType	指定i_dPosition的姿态表达方式，如四元数、欧拉角XYZ、欧拉角ZYX等等，详见9.2 位姿描述定义
i_eInstruct	轨迹点运动指令，按序排列, 详见9.3 轨迹点指令描述定义
i_dVelocity	轨迹点速度，按序排列
i_dSpeedPercent	轨迹点速度百分比，按序排列
i_nApproach	轨迹点轨迹逼近，按序排列
i_PathName	生成的轨迹名称
i_GroupName	生成的轨迹所属轨迹组名称
i_uCoordinateID	传入的点的位姿的参考坐标系，如点位姿基于世界坐标系则传0

HRESULT Path_insert_from_point_with_external_joint(ULONG i_ulRobotID, INT i_nPtCount, DOUBLE* i_dPosition, ULONG* i_ExtJointIDs, INT i_nExtJointCount, DOUBLE* i_dExtValue, INT i_nExtValueCount, PQPostureType i_ePostureType, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, BOOL i_bToolEndPosture, ULONG* o_PathID, BOOL i_bIsUpdate)	
i_bToolEndPosture	表示输入的位姿是否是工具末端位姿
o_PathID	返回生成的轨迹ID
i_bIsUpdate	插入点后是否更新，如果多次循环调用将i_bIsUpdate设置为False，在完成所有轨迹点插入后调用Doc_update_view更新一次即可
i_ePostureType	指定i_dPosition的姿态表达方式，如四元数、欧拉角XYZ、欧拉角ZYX等等，详见9.2 位姿描述定义

C++参考代码段

```
ULONG uRobotID = 0;
ULONG uGuideID = 0;
ULONG uMesID = 0;
GetObjIDByName(m_ptrKit, PQ_ROBOT, L"机器人", uRobotID);
GetObjIDByName(m_ptrKit, PQ_ROBOT, L"导轨", uGuideID);
GetObjIDByName(m_ptrKit, PQ_ROBOT, L"变位机", uMesID);
ULONG i_ExtJointIDs[2] = {
uGuideID ,uMesID }
```

```
;
double dPosition[12] = {
-2271.837025,-535.76009,1337.220742,13.295030,-43.655341,-15.332022,-2218.059215,-1083.124334,1336.
;
PQPostureType ePosType = EULERANGLEZYX;
PQPointInstruction nInstruction[2] = {
PQ_LINE,PQ_LINE }
;
double dVel[2] = {
200.0,200.0 }
;
double dSpeedP[2] = {
100.0,100.0 }
;
DOUBLE i_dExtValue[8] = {
800.89 ,0,0.00, 47* M_PI /180,1195.06 ,0,0.00, 47 * M_PI / 180 }
;
int nApproach[2] = {
-1,-1 }
;
wchar_t whPathName[] = _T("Test_Path");
wchar_t whPathGroupName[] = _T("Test_Group");
ULONG uPathID = 0;
m_ptrKit->Path_insert_from_point_with_external_joint(uRobotID, 2, dPosition, i_ExtJointID  s,2, i_d
```

生成AbsJ轨迹

在指定轨迹之前手动添加一个ABSJoint过渡轨迹。使用此API需要机器人和导轨、变位机已经存在抓取或者链接关系。注：本API可以支持多个点的数据接收处理，但是如果传入多个点的数据，将会生成多个absJ轨迹。

HRESULT PQAPIAddAbsJointPath(ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRJointsCount,DOUBLE* i_dGuideJoints, INT i_nGJointsCount, DOUBLE* i_dPositionerJoints, INT i_nPJointsCount,DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, INT i_nPointCount, ULONG i_uPathID, ULONG* o_uPathID)	
SDK:10.0.0.7068 (函数本体最低支持版本)11.1.0.7310(函数enum 传参最低支持版本)	
Parameters：	
i_ulRobotID	轨迹归属机器人ID
i_dRobotJoints	机器人关节角(弧度)
i_nRJointsCount	机器人关节角数组大小
i_dGuideJoints	导轨关节角(旋转轴传弧度，平移轴传长度)
i_nGJointsCount	导轨关节角数组大小
i_dPositionerJoints	变位机关节角(旋转轴传弧度，平移轴传长度)
i_nPJointsCount	变位机关节角数组大小
i_dVelocity	轨迹点速度
i_dSpeedPercent	轨迹点速度百分比

HRESULT PQAPIAddAbsJointPath(ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRJointsCount,DOUBLE* i_dGuideJoints, INT i_nGJointsCount, DOUBLE* i_dPositionerJoints, INT i_nPJointsCount,DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, INT i_nPointCount, ULONG i_uPathID, ULONG* o_uPathID)	
i_nApproach	轨迹点逼近方式
i_nPointCount	轨迹点个数
i_uPathID	预在其前插入ABSJoint过渡轨迹的轨迹ID
o_uPathID	生成的ABSJoint轨迹的轨迹ID

C++参考代码段

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"ROBOT1", ulRobotID);
DOUBLE dRobotJoints[6] = {
50,47,109,164,85,-185}
;
INT nRJointsCount = 6;
DOUBLE dGuideJoints[1] = {
0}
;
INT nGJointsCount = 1;
DOUBLE dPositionerJoints[1] = {
0}
;
INT nPJointsCount = 1;
DOUBLE dVelocity[1] = {
50}
;
DOUBLE dSpeedPercent[1] = {
50}
```

```
;
INT nApproach[1] = {
50}
;
INT PointCount = 1;
ULONG uiPathID ;
GetObjIDByName(PQ_PATH, L"过渡点82", uiPathID);
ULONG uoPathID = 0;
m_ptrKit->PQAPIAddAbsJointPath(ulRobotID, dRobotJoints, nRJointsCount, dGuideJoints, nGJointsCount,
}
```

生成过渡轨迹

在两条轨迹之间自动添加过渡轨迹

HRESULT PQAPIAddTransitPath(ULONG i_uPathAID, ULONG i_uPathBID, ULONG* o_PathID)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_uPathAID	轨迹A ID
i_uPathBID	轨迹B ID
o_PathID	生成的过渡轨迹ID

C++参考代码段

```
ULONG uPathAID = 0;
GetObjIDByName(PQ_PATH, L"", uPathAID);
ULONG uPathBID = 0;
GetObjIDByName(PQ_PATH, L"", uPathBID);
ULONG PathID = 0;
m_ptrKit->PQAPIAddTransitPath(uPathAID, uPathBID, &PathID);
```

生成避障轨迹

5.4.1初始化优化参数

HRESULT Kit_init_motion_plan_struct(PQMotionPlanParameter* i_PQMotionPlanParameter)	
SDK: 11.1.0.7330(函数本体最低支持版本)	
Parameters：	
i_PQMotionPlanParameter	需要赋值的优化参数结构体
### 5.4.2生成避障轨迹	

在两条轨迹间生成避障轨迹，需要先设置碰撞对。依次指定这2条连续轨迹ID，i_ulStartPathID对应轨迹在前，i_ulEndPathID对应轨迹id在后。

HRESULT Path_motion_plan(ULONG i_ulStartPathID, ULONG i_ulEndPathID, PQMotionPlanParameter i_PQMotionPlanParameter, ULONG* i_ulNewPathID)	
SDK: 11.1.0.7330(函数本体最低支持版本)	
Parameters：	
i_ulStartPathID	第一条轨迹id
i_ulEndPathID	第二条轨迹id
i_PQMotionPlanParameter	求解参数,包括 DOUBLE dMaxDistance：采样步长，默认0.5 DOUBLE dMaxTime：最大求解时间，若计算超过这个时间则认定求解失败，默认10s（不包含碰撞初始化时间） PQMotionOptimization eMotionOptimization：优化方式，包括PQ_NO_OPTIMIZATION 不做优化；PQ_SIMPLIFY简化路径；PQ_MERGEPOINTS = 合并相邻

HRESULT Path_motion_plan(ULONG i_ulStartPathID, ULONG i_ulEndPathID, PQMotionPlanParameter i_PQMotionPlanParameter, ULONG* i_ulNewPathID)	
	点，默认PQ_SIMPLIFY INT nPointCount 插入轨迹点数，进行平滑处理，一般与PQMotionOptimization配合使用，再进行轨迹优化可能会存在轨迹点与点之间碰撞，根据原始采样结果进行插补会进行相应改善，默认为10（10个点包含起点与终点，生成的新轨迹会自动省略掉起点终点避免重复）
i_ulNewPathID	传出新生成轨迹id

C++参考代码段

```
void MainWindow:: SampleFun () {
ULONG uPath1 = 0;
GetObjIDByName(PQ_PATH, L"", uPath1);
ULONG uPath2 = 0;
GetObjIDByName(PQ_PATH, L"", uPath2);
struct_PQMotionPlanParameter i_PQMotionPlanParameter;
m_ptrKit-> Kit_init_motion_plan_struct(&i_PQMotionPlanParameter);
i_PQMotionPlanParameter. dMaxDistance=0.01;
i_PQMotionPlanParameter. dMaxTime =30;
i_PQMotionPlanParameter. eMotionOptimization = PQ_SIMPLIFY;
i_PQMotionPlanParameter. nPointCount = 10;
ULONG uNewpathID = 0;
m_ptrKit-> Path_motion_plan (uPath1, uPath2, i_PQMotionPlanParameter, &uNewpathID);
}
```

更改轨迹组顺序

轨迹组顺序的更改目前只支持同一机器人下的更改，不支持不同机器人的轨迹组调序。

HRESULT PQAPIModifyPathGroupIndex(ULONG i_ulRobotID, BSTR i_bsSrcName, BSTR i_bsTarName, BOOL i_bBefore)	
Parameters：	
i_ulRobotID	轨迹组归属机器人
i_bsSrcName	想要调整顺序的轨迹组
i_bsTarName	调整目标轨迹组
i_bBefore	想要放置于目标轨迹组前还是后

C++参考代码段

```
void MainWindow::ModifyPathGroupIndex() {
ULONG ulRobotID = 0;
GetObjIDByName(32, L"", ulRobotID);
CComBSTR bsSrcName="";
CComBSTR bsTarName="";
LONG bBefore = 0;
m_ptrKit->PQAPIModifyPathGroupIndex(ulRobotID, bsSrcName, bsTarName, bBefore);
}
```

更改轨迹顺序

将i_ulSrcPathID指定的轨迹移至i_ulTarPathID指定的轨迹前还是后(取决于i_bAhead)。

HRESULT Path_change_order(ULONG i_ulSrcPathID, ULONG i_ulTarPathID, BOOL i_bAhead)	
Parameters：	
i_ulSrcPathID	轨迹组归属机器人

HRESULT Path_change_order(ULONG i_ulSrcPathID, ULONG i_ulTarPathID, BOOL i_bAhead)	
i_ulTarPathID	想要调整顺序的轨迹组
i_bAhead	调整目标轨迹组

C++参考代码段

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobotID);
CComBSTR bsSrcName="";
CComBSTR bsTarName="";
LONG bBefore = 0;
m_ptrKit->PQAPIModifyPathGroupIndex(ulRobotID, bsSrcName, bsTarName, bBefore);
```

设置/获取轨迹轴配置

5.6.1 6R机器人设置轨迹轴配置

HRESULT PQAPISetAxisConfig(ULONG i_uPathID, struct_PQ6RConfig i_eAxisCfg)	
11.1.0.7438 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
i_eAxisCfg	6R机器人呢轴配置信息

C++参考代码段

```
ULONG ulTarPathID = 0;
GetObjIDByName(PQ_PATH, L"", i_uPathID);
struct_PQ6RConfig i_eAxisCfg ;
m_ptrKit ->Doc_init_6Rconfig(&i_eAxisCfg);
m_ptrKit-> PQAPISetAxisConfig (i_uPathID, i_eAxisCfg);
```

5.6.2 3R1P机器人获取轨迹轴配置

HRESULT PQAPIGetAxisConfig6R(ULONG i_uPathID, struct_PQ3R1PConfig* o_eAxisCfg)	
11.1.0.7438 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
o_eAxisCfg	6R机器人轴配置信息

C++参考代码段

```
ULONG ulTarPathID = 0;
GetObjIDByName(PQ_PATH, L"", i_uPathID);
struct_PQ3R1PConfig i_eAxisCfg ;
m_ptrKit ->Doc_init_3R1Pconfig(&i_eAxisCfg);
m_ptrKit-> PQAPIGetAxisConfig3R1P (i_uPathID, &i_eAxisCfg);
```

导入轨迹无界面

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
bsCommandID	"RO_CMD_IMPORT_PATH"
wParam	指定轨迹关联的坐标系ID
lParam	0或NULL
bsParam	""

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
varParam	欲导入轨迹文件的绝对路径
lResult	0

C++参考代码段

```
CComBSTR cmd = " RO_CMD_IMPORT_PATH";
CComBSTR bsparam = "";
std::string strFilePath = "C:\\Users\\Public\\Desktop\\ ";
CComVariant varParam(strFilePath.c_str());
long long lresult;
m_ptrKit->pq_RunCommand(cmd, NULL, NULL, bsparam, varparam, &lresult);
```

C#参考代码段

```
string strFilePath = "D:\\ra_dev\\ra_diskR\\Runtime\\zh-cn\\Path\\圆形轨迹_开环. aptsource";
string strCMD = "RO_CMD_IMPORT_PATH";
string strParam = "";
// wParam指定轨迹关联的坐标系 ulong wParam = uCoorID;
/**填入具体数值**/ long lParam = 0;
object varParam = strFilePath as object;
long lResult = 0;
iRPC.pq_RunCommand(strCMD, wParam, lParam, strParam, varParam, out lResult);
```

轨迹添加Z轴旋转最小

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
bsCommandID	"RO_PATH_ZAXISANGLEMIN"
wParam	欲添加Z轴旋转最小的轨迹ID
lParam	0或NULL
bsParam	""
varParam	Empty
lResult	0

C++参考代码段

```
CComBSTR cmd = "RO_PATH_ZAXISANGLEMIN";
ULONG ulPathID=0;
GetObjIDByName (PQ_PATH,L"", ulPathID);
long long  wparam = ulPathID;
CComBSTR bsparam = "";
long long lresult;
m_ptrKit->pq_RunCommand(cmd, wparam, NULL, bsparam, CCOMVariant(), &lresult);
```

C#参考代码段

```
string strCMD = "RO_PATH_ZAXISANGLEMIN";
string strParam = "";
// wParam指定轨迹 ulong wParam = uID;
/**填入具体数值**/ long lParam = 0;
object varParam = null;
long lResult = 0;
iRPC.pq_RunCommand(strCMD, wParam, lParam, strParam, varParam, out lResult);
```

轨迹点倾角

通过varParam依次指定前倾角、侧倾角。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
bsCommandID	"RO_CMD_POINTDIPANGLE"
wParam	欲指定倾角的轨迹ID
lParam	0或NULL
bsParam	""
varParam	依次指定前倾角、侧倾角的2元素数组
lResult	0

C++参考代码段

```
CComBSTR cmd = "RO_CMD_POINTDIPANGLE";
ULONG ulPathID=0;
GetObjIDByName (PQ_PATH,L"", ulPathID);
long long  wparam = ulPathID;
CComBSTR bsparam = "";
double angle[2] ={
}
;
VARIANT pvarArray;
UtilCreateVariantFromArray(2, angle, VT_R8, &pvarArray);
long long lresult;
m_ptrKit->pq_RunCommand(cmd, wparam, NULL, bsparam, pvarArray, &lresult);
```

C#参考代码段

```
string strCMD = "RO_CMD_POINTDIPANGLE";
string strParam = "";
// wParam指定轨迹 ulong wParam = uID;
/**填入具体数值**/ long lParam = 0;
```

```
double[] dPara = new double[2] {
10.00, 20.00}
;
object varParam = dPara as object;
long lResult = 0;
iRPC.pq_RunCommand(strCMD, wParam, lParam, strParam, varParam, out lResult);
```

轨迹往复

HRESULT Path_feature_set_round(ULONG i_ulPathID, INT i_nTime)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
i_nTime	往复次数

C++参考代码段

```
ULONG uID = 0;
GetObjIDByName(PQ_PATH, L"", uID);
INT nTime = 1;
m_ptrKit->Path_feature_set_round(uID, nTime);
```

生成出入刀点

HRESULT Path_add_entry_point(ULONG i_ulPathID, DOUBLE i_dInOffset, DOUBLE i_dOutOffset, PQPointInstruction i_eInInstruction, PQPointInstruction i_eOutInstruction)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
i_dInOffset	入刀点偏移量
i_dOutOffset	出刀点偏移量

HRESULT Path_add_entry_point(ULONG i_ulPathID, DOUBLE i_dInOffset, DOUBLE i_dOutOffset, PQPointInstruction i_eInInstruction, PQPointInstruction i_eOutInstruction)	
i_eInInstruction	入刀点指令，仅限直线和关节角(弧度)
i_eOutInstruction	出刀点指令，仅限直线和关节角(弧度)

C++参考代码段

```
ULONG uID = 0;
GetObjIDByName(PQ_PATH, L"", uID);
double inoffset = 2.3;
double outoffset = 2.4;
PQPointInstruction instruction = PQ_LINE;
PQPointInstruction outinstruction = PQ_LINE;
m_ptrKit->Path_add_entry_point(uID, inoffset, outoffset, i_eInInstruction, i_eOutInstruction);
```

获取外部轴关节角

HRESULT PQAPIGetExternalJointsFromPoints(ULONG i_ulPointID, ULONG i_uExternalID, VARIANT* o_varJointsArray)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_ulPointID	轨迹点ID
i_uExternalID	导轨或者变位机的ID
o_varJointsArray	导轨或者变位机关节角(弧度)

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
VARIANT varIDarr;
m_ptrKit->PQAPIGetPointsID(ulPathID, &varIDarr);
SAFEARRAY* psa = varIDarr.parray;
if (!psa || psa->cDims != 1)
return ;
ULONG Size = psa->rgsabound[0].cElements;
if (Size == 0) {
return ;
}
ULONG ulPointID = 0;
LONG lIdx = 1;
SafeArrayGetElement(psa, &lIdx, &ulPointID);
ULONG uExternalID = 0;
GetObjIDByName(80, L"", uExternalID);
VARIANT varJointsArray;
m_ptrKit->PQAPIGetExternalJointsFromPoints(ulPointID, uExternalID, &varJointsArray);
```

轨迹修改外部轴关节角

原PQAPIModifyExternalAxleBatch弃用。

HRESULT Path_modify_external_axis(ULONG i_ulPathID, ULONG* i_ulGuideID, INT i_nGuideCount, DOUBLE* i_dGuideData, INT i_nGuideDataCount)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
i_ulGuideID	导轨或者变位机等外部轴ID
i_nGuideCount	导轨或者变位机等外部轴个数

HRESULT Path_modify_external_axis(ULONG i_ulPathID, ULONG* i_ulGuideID, INT i_nGuideCount, DOUBLE* i_dGuideData, INT i_nGuideDataCount)	
i_dGuideData	外部轴关节角数据(旋转轴传弧度)
i_nGuideDataCount	外部轴关节角数据个数

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
ULONG ulGuideID=1;
GetObjIDByName(PQ_PATH, L"", ulGuideID);
INT nGuideCount = 1;
DOUBLE dGuideData[3]={
1000.00, 500.00, 1000.00}
;
INT nGuideDataCount = 3;
m_ptrKit->Path_modify_external_axis(ulPathID, &ulGuideID, nGuideCount, &dGuideData, nGuideDataCount
```

删除轨迹组

删除指定机器人下指定名称的轨迹分组

HRESULT PQAPIDeletePathGroup(ULONG i_ulRobotID,BSTR bsName)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_ulRobotID	机器人ID
bsName	轨迹分组名称

C++参考代码段

```
ULONG uID = 0;
GetObjIDByName(PQ_ROBOT, L"", uID);
CComBSTR bsName = "";
m_ptrKit->PQAPIDeletePathGroup(uID, bsName);
```

删除所有轨迹

删除指定机器人下所有轨迹

HRESULT PQAPIDeletePathGroupAll(ULONG i_ulRobotID)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_ulRobotID	轨迹归属机器人ID

C++参考代码段

```
ULONG uID = 0;
GetObjIDByName(PQ_ROBOT, L"", uID);
m_ptrKit->PQAPIDeletePathGroupAll (uID);
```

轨迹关联坐标系

给轨迹设置关联坐标系，设置关联坐标系后，该关联坐标系移动轨迹随动。

HRESULT PQAPICreatePathCoordinateRelation(ULONG i_ulPathID, ULONG i_ulCoordinateID)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
i_ulCoordinateID	指定轨迹关联的坐标系的ID

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
ULONG ulCoordinateID = 0;
GetObjIDByName(PQ_COORD, L"", ulCoordinateID);
m_ptrKit->PQAPICreatePathCoordinateRelation(ulPathID, ulCoordinateID);
```

设定轨迹后置使用坐标系

HRESULT Path_Set_Post_Coordinate (ULONG i_ulPathID, ULONG i_ulCoorID)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
i_ulCoorID	指定轨迹后置所用的坐标系的ID

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
ULONG ulCoorID = 0;
GetObjIDByName(PQ_COORD, L"", ulCoorID);
m_ptrKit->Path_set_post_coordinate(ulPathID, ulCoorID);
```

获取机构下轨迹

获取指定机构下所有轨迹

HRESULT PQAPIGetPathFromRobot(ULONG i_ulRobotID, BSTR* o_sNames, BSTR* o_sIDs)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_ulRobotID	机器人ID

HRESULT PQAPIGetPathFromRobot(ULONG i_ulRobotID, BSTR* o_sNames, BSTR* o_sIDs)	
o_sNames	所有轨迹对象的名字，以“#”分割
o_sIDs	所有轨迹对象的ID，以“#”分割

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
CComBSTR sName="";
CComBSTR sIDs="";
m_ptrKit->PQAPIGetPathFromRobot(ulPathID, &sName, &sIDs);
```

获取轨迹状态

获取轨迹可达性状态

HRESULT PQAPIGetPathStatus(ULONG i_ulPathID, INT* o_nStatus)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
o_nStatus	轨迹可达性状态，具体值详见9.6轨迹可达性状态定义

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT  nStatus;
m_ptrKit->PQAPIGetPathStatus(ulPathID, &nStatus);
```

轨迹逆运动学求解方式

设置轨迹逆运动学求解方式

HRESULT PQAPISetPathInverType(ULONG i_uPathID, INT i_nType)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
i_nType	轨迹逆运动学求解方式，具体值详见9.5 轨迹逆运动学求解方式定义

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT  nType =0;
m_ptrKit->PQAPISetPathInverType(ulPathID, nType);
```

轨迹优化

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
bsCommandID	"RO_CMD_SMOOTH_PATH"
wParam	欲进行优化的轨迹ID
lParam	0或NULL
bsParam	""

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
varParam	Empty
lResult	0

C#参考代码段

```
string strCMD = "RO_CMD_SMOOTH_PATH";
string strParam = "";
ulong wParam = (ulong)uPathID;
/**填入具体数值**/ long lParam = 0;
object varParam = null;
long lResult = 0;
iRPC.pq_RunCommand(strCMD, wParam, lParam, strParam, varParam, out lResult);
```

C++参考代码段

```
CComBSTR cmd = "RO_CMD_SMOOTH_PATH";
ULONG uPathID = 0;
GetObjIDByName(PQ_PATH, L"", uPathID);
long long  wparam = uPathID;
CComBSTR bsparam = "";
long long lresult;
m_ptrKit->pq_RunCommand(cmd, wparam, NULL, bsparam, CCOMVariant(), &lresult);
```

获取轨迹组下轨迹

HRESULT Path_get_group_path(ULONG i_ulRobotID, BSTR i_GroupName, VARIANT* o_sNames, VARIANT* o_sIDs)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulRobotID	轨迹组归属的机构ID

HRESULT Path_get_group_path(ULONG i_ulRobotID, BSTR i_GroupName, VARIANT* o_sNames, VARIANT* o_sIDs)	
i_GroupName	轨迹组名称
o_sNames	轨迹组下轨迹名称数组
o_sIDs	轨迹组下轨迹ID数组

C++参考代码段

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobotID);
CComBSTR GroupName="";
VARIANT sNames;
sNames.parray = NULL;
VARIANT sIDs;
sIDs.parray = NULL;
m_ptrKit->Path_get_group_path(ulRobotID, GroupName, &sNames, &sIDs);
```

获取轨迹中轨迹点个数

HRESULT Path_get_point_count(ULONG i_ulPathID, INT* o_nCount)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
o_nCount	轨迹下轨迹点个数

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nCount = 0;
m_ptrKit->Path_get_point_count(ulPathID,&nCount);
```

添加 / 取消工件关联

添加工件关联，通过执行RO_CMD_REMOVE_ADDREFWORKINGPART命令来实现添加工件关联。通过varParam参数传递轨迹ID和欲关联的工件ID，轨迹ID在前，工件ID在后依次排序，支持传入多个轨迹ID，但是工件ID只能是一个。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:9.2.0.6340 (函数本体最低支持版本)	
Parameters：	
bsCommandID	"RO_CMD_REMOVE_ADDREFWORKINGPART"
wParam	0或NULL
lParam	0或NULL
bsParam	""
varParam	欲添加工件关联的轨迹ID和工件ID的数组
lResult	0

C#参考代码段

```
string strCMD = "RO_CMD_REMOVE_ADDREFWORKINGPART";
string strParam = "";
ulong wParam = 0;
long lParam = 0;
//uObj为构造的欲给轨迹添加工件关联的轨迹和工件的ID数组 object varParam = uObj as object;
long lResult = 0;
iRPC.pq_RunCommand(strCMD, wParam, lParam, strParam, varParam, out lResult);
```

C++参考代码段

```
CComBSTR cmd = "RO_CMD_REMOVE_ADDREFWORKINGPART";
CComBSTR bsparam = "";
```

```
ULONG uIDs[2] = {
}
;
VARIANT pvarArray;
UtilCreateVariantFromArray (2, uIDs, VT_UI4, &pvarArray);
long long lresult;
m_ptrKit->pq_RunCommand(cmd, NULL, NULL, bsparam, pvarArray, &lresult);
```

取消工件关联，通过执行RO_CMD_REMOVE_REFWORKINGPART命令来实现取消工件关联。通过varParam参数传递轨迹ID，支持传入多个轨迹ID。示例代码同上。

关节角生成轨迹

PQKit提供两种由关节角数据生成轨迹点的接口方法。

1、Path_insert_from_joint

调用Path_insert_from_joint接口可生成指定机器人、导轨、变位机关节角数据的轨迹。并且机器人、导轨、变位机会运动到指定的关节角处。默认情况下插入的轨迹点会关联机器人Base坐标系，即轨迹点会随着机器人位置变化而变化；如果不需要轨迹点跟随机器人随动，需要通过i_uCoordinateID参数传入一个用户坐标系ID进去，通常为大地坐标系等，这样生成的轨迹点就不会跟着机器人随动。

HRESULT Path_insert_from_joint(ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRobotJointsSize, DOUBLE* i_dGuideJoints, INT i_nGuideJointsSize, DOUBLE* i_dPositionerJoints, INT i_nPositionerJointsSize, INT i_nPointCount, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, ULONG* o_PathID,BOOL i_bIsUpdate)	
SDK:9.2.0.6340 (函数本体最低支持版本) 11.1.0.7310(函数enum 传参最低支持版本)	
Parameters：	
i_ulRobotID	指定生成的轨迹归属于哪个机器人

HRESULT Path_insert_from_joint(ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRobotJointsSize, DOUBLE* i_dGuideJoints, INT i_nGuideJointsSize, DOUBLE* i_dPositionerJoints, INT i_nPositionerJointsSize, INT i_nPointCount, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, ULONG* o_PathID,BOOL i_bIsUpdate)	
i_dRobotJoints	各个点位机器人的关节角数据（旋转轴传弧度，平移轴传距离）
i_nRobotJointsSize	关节角数据数组大小
i_dGuideJoints	关联的导轨关节角数据（旋转轴传弧度，平移轴传距离）
i_nGuideJointsSize	导轨数据数组大小
i_dPositionerJoints	关联的变位机关节角数据（旋转轴传弧度，平移轴传距离）
i_nPositionerJointsSize	变位机数据数组大小
i_nPointCount	输入的轨迹点个数
i_eInstruct	各个点处的轨迹点指令（PQ_JOINT：关节角 PQ_ABSJOINT：AbsJ）
i_dVelocity	轨迹点速度，按序排列
i_dSpeedPercent	轨迹点速度百分比，按序排列
i_nApproach	轨迹点轨迹逼近，按序排列

HRESULT Path_insert_from_joint(ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRobotJointsSize, DOUBLE* i_dGuideJoints, INT i_nGuideJointsSize, DOUBLE* i_dPositionerJoints, INT i_nPositionerJointsSize, INT i_nPointCount, PQPointInstruction* i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, ULONG* o_PathID,BOOL i_bIsUpdate)	
i_PathName	生成的轨迹名称
i_GroupName	生成的轨迹所属轨迹组名称
i_uCoordinateID	参考坐标系ID
o_PathID	返回生成的轨迹ID
i_bIsUpdate	插入点后是否更新，如果多次循环调用将i_bIsUpdate设置为False，在完成所有轨迹点插入后调用Doc_update_view更新一次即可

C++参考代码段

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"ABB-IRB4600-45", ulRobotID);
DOUBLE dRobotJoints[12] = {
0,0,0,0,5,0,-0.471,24.025,1.947 ,-3.492,21.855,-27.068 }
;
INT nRobotJointsSize = 12;
/*没有变位机和导轨nPositionerJointsSize与nGuideJointsSize设置为0即可 dPositionerJoints, dGuideJoint
DOUBLE dGuideJoints[2] = {
1000,800}
;
INT nGuideJointsSize = 2;
DOUBLE dPositionerJoints[1] = {
0.0}
```

```
;
INT nPositionerJointsSize = 0;
INT nPointCount = 2;
INT eInstruct[1] = {
PQ_LINE}
;
DOUBLE dVelocity[2] = {
200 ,150}
;
DOUBLE dSpeedPercent[2] = {
50,50}
;
INT nApproach[2] = {
-1,-1}
;
std::wstring GroupName1 = L"Rob2_A_";
std::wstring myWString = std::to_wstring(nGuideJointsSize);
std::wstring GroupName = GroupName1 + myWString;
CComBSTR PathName = L"Home点_2";
ULONG uCoordinateID = 0;
ULONG PathID = 0;
m_ptrKit->Path_insert_from_joint(ulRobotID, dRobotJoints, nRobotJointsSize, dGuideJoints, nGuideJoi
CComBSTR(GroupName.c_str()), uCoordinateID, &PathID);
```

2、Path_insert_from_joint_without_movement

机器人不移动，只考虑导轨，不考虑变位机，调用Path_insert_from_joint_without_movement接口可生成指定机器人、导轨关节角数据的轨迹。

HRESULT Path_insert_from_joint_without_movement (ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRobotJointsSize, DOUBLE* i_dGuideJoints, INT i_nGuideJointsSize, INT i_nPointCount, PQPointInstruction * i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, ULONG* o_PathID,BOOL i_bIsUpdate)	
SDK:10.0.0.7068 (函数本体最低支持版本) 11.1.0.7310(函数enum 传参最低支持版本)	

HRESULT Path_insert_from_joint_without_movement (ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRobotJointsSize, DOUBLE* i_dGuideJoints, INT i_nGuideJointsSize, INT i_nPointCount, PQPointInstruction * i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, ULONG* o_PathID,BOOL i_bIsUpdate)	
Parameters：	
i_ulRobotID	指定生成的轨迹归属于哪个机器人
i_dRobotJoints	各个点位机器人的关节角数据（旋转轴传弧度，平移轴传距离）
i_nRobotJointsSize	关节角数据数组大小
i_dGuideJoints	关联的导轨关节角数据（旋转轴传弧度，平移轴传距离）
i_nGuideJointsSize	导轨数据数组大小
i_nPointCount	输入的轨迹点个数
i_eInstruct	各个点处的轨迹点指令（PQ_JOINT：关节角 PQ_ABSJOINT：AbsJ）
i_dVelocity	轨迹点速度，按序排列
i_dSpeedPercent	轨迹点速度百分比，按序排列
i_nApproach	轨迹点轨迹逼近，按序排列
i_PathName	生成的轨迹名称

HRESULT Path_insert_from_joint_without_movement (ULONG i_ulRobotID, DOUBLE* i_dRobotJoints, INT i_nRobotJointsSize, DOUBLE* i_dGuideJoints, INT i_nGuideJointsSize, INT i_nPointCount, PQPointInstruction * i_eInstruct, DOUBLE* i_dVelocity, DOUBLE* i_dSpeedPercent, INT* i_nApproach, BSTR i_PathName, BSTR i_GroupName, ULONG i_uCoordinateID, ULONG* o_PathID,BOOL i_bIsUpdate)	
i_GroupName	生成的轨迹所属轨迹组名称
i_uCoordinateID	参考坐标系ID
o_PathID	返回生成的轨迹ID
i_bIsUpdate	插入点后是否更新，如果多次循环调用将i_bIsUpdate设置为False，在完成所有轨迹点插入后调用Doc_update_view更新一次即可

C++参考代码段

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"ABB-IRB4600-45", ulRobotID);
DOUBLE dRobotJoints[12] = {
0,0,0,0,5,0,-0.471,24.025,1.947 ,-3.492,21.855,-27.068 }
;
INT nRobotJointsSize = 12;
/*没有导轨nGuideJointsSize设置为0即可dGuideJoints不能传NULL*/
DOUBLE dGuideJoints[2] = {
1000,800}
;
INT nGuideJointsSize = 2;
INT nPointCount = 2;
INT eInstruct[1] = {
PQ_JOINT }
```

```
;
DOUBLE dVelocity[2] = {
200 ,150}
;
DOUBLE dSpeedPercent[2] = {
50,50}
;
INT nApproach[2] = {
-1,-1}
;
std::wstring GroupName1 = L"Rob2_A_";
std::wstring myWString = std::to_wstring(nGuideJointsSize);
std::wstring GroupName = GroupName1 + myWString;
CComBSTR PathName = L"Home点_2";
ULONG uCoordinateID = 0;
ULONG PathID = 0;
m_ptrKit-> Path_insert_from_joint_without_movement (ulRobotID, dRobotJoints, nRobotJointsSize, dGui
CComBSTR(GroupName.c_str()),uCoordinateID, &PathID);
```

获取轨迹生成Face数据

返回指定轨迹生成时基于的Face TopoDS数据。

HRESULT Path_get_generate_face(ULONG i_ulPathID, INT *o_nFaceCount, LONG_PTR **o_lpFacePtr)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
o_nFaceCount	Face数据个数
o_lpFacePtr	Face数据集合，LONG_PTR数组内存在PQKit内部分配，需要调用PQAPIFree释放

C++参考代码

```
void CPQMFCSampleDlg:: SampleFun() {
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT nFaceCount = 0;
LONG_PTR* lpFacePtr = NULL;
m_ptrKit->Path_get_generate_face(ulPathID, &nFaceCount, &lpFacePtr);
for (int i = 0; i < nFaceCount; i++) { LPVOID lpData = (LPVOID)lpFacePtr[i]; }
m_ptrKit->PQAPIFree(lpFacePtr);
}
```

获取轨迹生成Shape数据

返回指定轨迹生成时基于的Shape TopoDS数据。

HRESULT Path_get_generate_shape(ULONG i_ulPathID, INT *o_nShapeCount, LONG_PTR **o_lpShapePtr)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
o_nShapeCount	轨迹点ID集合
o_lpShapePtr	Shape数据集合，LONG_PTR数组内存内存在PQKit内部分配，需要调用PQAPIFree释放

返回的数据转TopoDS数据参考：

TopoDS_Shape SampleFun (LPVOID lp) { Handle_TopoDS_TShape tHandle((TopoDS_TShape*)lp); TopoDS_Shape shape; shape.TShape(tHandle); return shape; }

C++参考代码

```
void MainWindow:: SampleFun () {
ULONG uID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT nShapeCount = 0;
LONG_PTR* lpShapePtr;
```

```
m_ptrKit->Path_get_generate_shape(uID, &nShapeCount, &lpShapePtr);
m_ptrKit->PQAPIFree(lpShapePtr);
}
```

获取/设置TCP

获取轨迹所用TCP名字

HRESULT Path_get_tcp(ULONG i_ulID, WCHAR **o_chTcpName)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulID	传入的轨迹ID
o_chTcpName	返回TCP名字,TCP名字需要调用PQAPIFree释放

C++参考代码

```
void MainWindow:: SampleFun() {
ULONG uID = 0;
GetObjIDByName(PQ_PATH, L"", uID);
WCHAR* chTCPName= nullptr;
m_ptrKit->Path_get_tcp(uID, &chTCPName);
m_ptrKit->PQAPIFree((LONG_PTR*)chTCPName);
}
```

轨迹设置TCP

HRESULT Path_set_tcp(ULONG i_ulID, const WCHAR* i_chTcpName)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulID	传入的轨迹ID
i_chTcpName	返回的TCP名字

C++参考代码

```
void MainWindow:: SampleFun () {
ULONG uID = 0;
GetObjIDByName(PQ_PATH, L"", uID);
WCHAR* chTCPName;
m_ptrKit->Path_set_tcp(uID, chTCPName);
}
```

获取轨迹速度

HRESULT Path_get_velocity(ULONG i_ulPathID, DOUBLE * o_dInitWorkingVel, DOUBLE * o_dJointWorkingVel, DOUBLE * o_dJointSpeedPercent)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
o_dInitWorkingVel	传出的轨迹线速度
o_dJointWorkingVel	传出轨迹角速度
o_dJointSpeedPercent	传出轨迹速度百分比

C++参考代码

```
void MainWindow:: SampleFun() {
ULONG uID = 0;
GetObjIDByName(PQ_PATH, L"", uID);
DOUBLE dInitWorkingVel = 0;
DOUBLE dJointWorkingVel = 0;
DOUBLE dJointSpeedPercent = 0;
m_ptrKit->Path_get_velocity(uID, &dJointWorkingVel, &dInitWorkingVel, &dJointSpeedPercent);
}
```

批量设置速度

HRESULT Path_set_velocity_batch(ULONG *i_ulPathIDs, INT i_nPathIDArraySize, double i_dInitWorkingVel, double i_dJointWorkingVel, double i_dJointSpeedPercent)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulPathIDs	轨迹ID数组
I_nPathIDArraySize	传入轨迹数组的元素个数
i_dInitWorkingVel	欲设置的轨迹线速度
i_dJointWorkingVel	欲设置轨迹角速度
i_dJointSpeedPercent	欲设置轨迹速度百分比

C++参考代码

```
void MainWindow :: SampleFun () {
ULONG uID[1] = {
}
;
INT nPathIDArraySize = 1;
double dInitWorkingVel = 2.3;
double dJointWorkingVel = 1.5;
double dJointSpeedPercent = 100;
```

```
m_ptrKit->Path_set_velocity_batch(&uID, nPathIDArraySize, dInitWorkingVel, dJointWorkingVel, dJoint
}
```

获取机器人所有逆解

5.31.1 获取6R机器人所有逆解

HRESULT Robot_Get_All_Inverse_Kinematics (ULONG i_ulRobotID, DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE i_pJointValues, INT* o_nJointValuesCount, struct_PQ6RConfig** o_eAxisCfg, INT* o_nAxisCfgCount)**	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulRobotID	机器人id
i_EndPosture	待求解的末端位姿（机器人BASE坐标系）
i_nPostureArraySize	传入位姿的数组大小
i_ePostureType	传入位姿类型，详情参考9.2
b_IsEndPosture	传入位姿是否为工具末端位姿态，是传True，若是法兰位姿False(均是以机器人当前所在位置为准)
i_pJointValues	输出所有逆解的数组需要调用

HRESULT Robot_Get_All_Inverse_Kinematics (ULONG i_ulRobotID, DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE i_pJointValues, INT* o_nJointValuesCount, struct_PQ6RConfig** o_eAxisCfg, INT* o_nAxisCfgCount)**	
	PQAPIFree释放
o_nJointValuesCount	输出逆解数组的长度
o_eAxisCfg	输出所有逆解的对应的轴配置需要调用PQAPIFree释放
o_nAxisCfgCount	输出逆解的对应的轴配置数组长度
void MainWindow :: SampleFun () { ULONG o_uRobotID1 =0;*//自行获取//* double i_EndPosture[6] = { 432.205368 ,1286.399953 ,983.319324 ,23.536156,19.901450, 36.340421 }; double* i_pJointValues = NULL; INT o_nJointValuesCountl = 0; struct_PQ6RConfig * i_exisCfg = NULL; INT o_nAxisCfgCount = 0; PQPostureType i_ePostureType = EULERANGLEXYZ; m_ptrKit-> Robot_Get_All_Inverse_Kinematics (o_uRobotID1, i_EndPosture, 6, i_ePostureType, True,&i_pJointValues, &o_nJointValuesCountl, &i_eAxisCfg, &o_nAxisCfgCount); }	
### 5.31.2获取3R1P机器人所有逆解	

HRESULT Robot3R1p_Get_All_Inverse_Kinematics (ULONG i_ulRobotID, DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE i_pJointValues, INT* o_nJointValuesCount, struct_PQ3R1PConfig** o_eAxisCfg, INT* o_nAxisCfgCount)**	
SDK:10.0.0.7068 (函数本体最低支持版本)	

HRESULT Robot3R1p_Get_All_Inverse_Kinematics (ULONG i_ulRobotID, DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE i_pJointValues, INT* o_nJointValuesCount, struct_PQ3R1PConfig** o_eAxisCfg, INT* o_nAxisCfgCount)**	
Parameters：	
i_ulRobotID	机器人id
i_EndPosture	待求解的末端位姿（机器人BASE坐标系）
i_nPostureArraySize	传入位姿的数组大小
i_ePostureType	传入位姿类型，详情参考9.2
b_IsEndPosture	传入位姿是否为工具末端位姿态，是传True，若是法兰位姿False(均是以机器人当前所在位置为准)
i_pJointValues	输出所有逆解的数组需要调用PQAPIFree释放
o_nJointValuesCount	输出逆解数组的长度
o_nAxisCfg	输出所有逆解的对应的轴配置需要调用PQAPIFree释放

HRESULT Robot3R1p_Get_All_Inverse_Kinematics (ULONG i_ulRobotID, DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE i_pJointValues, INT* o_nJointValuesCount, struct_PQ3R1PConfig** o_eAxisCfg, INT* o_nAxisCfgCount)**	
o_nAxisCfgCount	输出逆解的对应的轴配置数组长度
void MainWindow :: SampleFun () { ULONG o_uRobotID1 =0;*//自行获取//* double i_EndPosture[6] = { 432.205368 ,1286.399953 ,983.319324 ,23.536156,19.901450, 36.340421 }; double* i_pJointValues = NULL; INT o_nJointValuesCountl = 0; INT* i_nAxisCfg = NULL; INT o_nAxisCfgCount = 0; struct_PQ3R1PConfig * i_exisCfg = NULL; PQPostureType i_ePostureType = EULERANGLEXYZ; m_ptrKit-> Robot_Get_All_Inverse_Kinematics (o_uRobotID1, i_EndPosture, 6, i_ePostureType, True,&i_pJointValues, &o_nJointValuesCountl, &i_nAxisCfg, &o_nAxisCfgCount); }	
## 修改轨迹粗细	

HRESULT Path_modify_line_width(ULONG i_ulPathID, DOUBLE i_dLineWidth)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
i_dLineWidth	设置轨迹粗细值

C++参考代码

```
void MainWindow :: SampleFun () {
ULONG uID;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
```

```
m_ptrKit->Path_modify_line_width(uID,9);
}
```

轨迹是否显示名字

HRESULT Path_show_name(ULONG i_ulPathID, BOOL i_bShow)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
i_bShow	是否显示名字

C++参考代码

```
void MainWindow :: SampleFun () {
ULONG uID;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
m_ptrKit->Path_show_name(uID,true);
}
```

修改轨迹颜色

HRESULT Path_modify_line_color(ULONG i_ulPathID, LONG i_lColor)	
SDK:10.0.0.7068 (函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
i_lColor	修改轨迹颜色

C++参考代码

```
void MainWindow :: SampleFun () {
ULONG uID;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
m_ptrKit->Path_modify_line_color(uID,RGB(255,255,255));
```

```
}
```

带外部轴机器人逆运动学求解

5.35.1 6R机器人求解

HRESULT Robot_inverse_kinematics_with_external_axis (ULONG i_ulRobotID, INT i_nCalInveType,DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE* io_pJointValues, INT o_nJointValuesCount, DOUBLE* io_ExtJointRadValues, INT i_nExtJointRadValuesCount, DOUBLE* io_GuideRadValues, INT i_nGuideValuesCount, DOUBLE i_dfloating, struct_PQ6RConfig i_eAxisCfg, PQERoPathPtState* o_ePtStatus)	
11.1.0.7438 (函数本体最低支持版本)	
Parameters：	
i_ulRobotID	机器人id
i_nCalInveType	联动类型，参考9.5
i_EndPosture	待求解末端位姿数组，为世界坐标系下位姿
i_nPostureArraySize	位姿数组长度
i_ePostureType	位姿类型，参考9.2
b_IsEndPosture	传入位姿是否为工具末端位姿态，是传True，

HRESULT Robot_inverse_kinematics_with_external_axis (ULONG i_ulRobotID, INT i_nCalInveType,DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE* io_pJointValues, INT o_nJointValuesCount, DOUBLE* io_ExtJointRadValues, INT i_nExtJointRadValuesCount, DOUBLE* io_GuideRadValues, INT i_nGuideValuesCount, DOUBLE i_dfloating, struct_PQ6RConfig i_eAxisCfg, PQERoPathPtState* o_ePtStatus)	
	若是世界坐标位姿(轨迹点位姿态)传False
io_pJointValues	传入上一组关节角（弧度），传出求解后关节角
o_nJointValuesCount	关节角数组长度
io_ExtJointRadValues	传入上一组变位机数据，传出求解变位机数据
i_nExtJointRadValuesCount	变位机数组长度
io_GuideRadValues	传入上一组导轨数据，传出求解导轨数据
i_nGuideValuesCount	导轨位置长度

HRESULT Robot_inverse_kinematics_with_external_axis (ULONG i_ulRobotID, INT i_nCalInveType,DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE* io_pJointValues, INT o_nJointValuesCount, DOUBLE* io_ExtJointRadValues, INT i_nExtJointRadValuesCount, DOUBLE* io_GuideRadValues, INT i_nGuideValuesCount, DOUBLE i_dfloating, struct_PQ6RConfig i_eAxisCfg, PQERoPathPtState* o_ePtStatus)	
i_dfloating	奇异点浮动值
i_eAxisCfg	轴配置信息，详见9.7 轴配置定义
o_ePtStatus	可达性状态，参考9.6

C++参考代码

```
void MainWindow :: SampleFun() {
ULONG uRobotID =  0;
/*自行获取*/ double io_pJointValues[6] = {
0.026092672317315262,1.3997540600998919,-0.0069813170076473        696,-0.45177847687885409,0.86789
;
double io_pJointValues2[2] = {
-460.388,628.763 }
;
double i_endPosture[6] = {
-2300.867701,-742.884852,2043.589165 ,-25.650032 ,-3.026638    ,179.878100 }
;
struct_PQ6RConfig i_eAxisCfg ;
m_ptrKit ->Doc_init_6Rconfig(&i_eAxisCfg);
PQPostureType i_ePostureType = EULERANGLEXYZ;
PQERoPathPtState o_ePtStatus = PQ_PPS_BE_REACH;
m_ptrKit ->Robot_inverse_kinematics_with_external_axis(uRobotID, 2, i_endPosture,6,    i_ePostureTy
}
```

5.35.2 3R1P机器人求解

HRESULT Robot3R1P_inverse_kinematics_with_external_axis (ULONG i_ulRobotID, INT i_nCalInveType,DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE* io_pJointValues, INT o_nJointValuesCount, DOUBLE* io_ExtJointRadValues, INT i_nExtJointRadValuesCount, DOUBLE* io_GuideRadValues, INT i_nGuideValuesCount, DOUBLE i_dfloating, struct_PQ3R1PConfig i_eAxisCfg, PQERoPathPtState* o_ePtStatus)	
11.1.0.7438 (函数本体最低支持版本)	
Parameters：	
i_ulRobotID	机器人id
i_nCalInveType	联动类型，参考9.5
i_EndPosture	待求解末端位姿数组，为世界坐标系下位姿
i_nPostureArraySize	位姿数组长度
i_ePostureType	位姿类型，参考9.2
b_IsEndPosture	传入位姿是否为工具末端位姿态，是传True，若是世界坐标位姿(轨迹点位姿态)传False
io_pJointValues	传入上一组关节角（弧

HRESULT Robot3R1P_inverse_kinematics_with_external_axis (ULONG i_ulRobotID, INT i_nCalInveType,DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE* io_pJointValues, INT o_nJointValuesCount, DOUBLE* io_ExtJointRadValues, INT i_nExtJointRadValuesCount, DOUBLE* io_GuideRadValues, INT i_nGuideValuesCount, DOUBLE i_dfloating, struct_PQ3R1PConfig i_eAxisCfg, PQERoPathPtState* o_ePtStatus)	
	度），传出求解后关节角
o_nJointValuesCount	关节角数组长度
io_ExtJointRadValues	传入上一组变位机数据，传出求解变位机数据
i_nExtJointRadValuesCount	变位机数组长度
io_GuideRadValues	传入上一组导轨数据，传出求解导轨数据
i_nGuideValuesCount	导轨位置长度
i_dfloating	奇异点浮动值
i_eAxisCfg	轴配置信息，详见9.7轴配置定义

HRESULT Robot3R1P_inverse_kinematics_with_external_axis (ULONG i_ulRobotID, INT i_nCalInveType,DOUBLE* i_EndPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType, BOOL b_IsEndPosture, DOUBLE* io_pJointValues, INT o_nJointValuesCount, DOUBLE* io_ExtJointRadValues, INT i_nExtJointRadValuesCount, DOUBLE* io_GuideRadValues, INT i_nGuideValuesCount, DOUBLE i_dfloating, struct_PQ3R1PConfig i_eAxisCfg, PQERoPathPtState* o_ePtStatus)	
o_ePtStatus	可达性状态，参考9.6

C++参考代码

```
 void MainWindow :: SampleFun() {
ULONG uRobotID =  0;
/*自行获取*/ double io_pJointValues[4] = {
0.026092672317315262,1.3997540600998919,-0.0069813170076473        696,-0.45177847687885409 }
;
double io_pJointValues2[2] = {
-460.388,628.763 }
;
double i_endPosture[6] = {
-2300.867701,-742.884852,2043.589165 ,-25.650032 ,-3.026638    ,179.878100 }
;
struct_PQ3R1PConfig i_eAxisCfg ;
m_ptrKit ->Doc_init_3R1Pconfig(&i_eAxisCfg);
PQPostureType i_ePostureType = EULERANGLEXYZ;
PQERoPathPtState o_ePtStatus = PQ_PPS_BE_REACH;
m_ptrKit ->Robot3R1P_inverse_kinematics_with_external_axis(uRobotID, 2, i_endPosture,4,     i_ePostu
}
```

轨迹新增多层多道特征

5.36.1轨迹新增多层多道特征

HRESULT Path_add_multi_layer_weld_feature(const ULONG i_ulPathID, INT i_nLayerNum, BOOL i_bAutoLoop, DOUBLE i_dBackPathZOffset, DOUBLE i_dTangentialOffset, DOUBLE i_dTangentialLeftOffset, DOUBLE i_dTangentialRightOffset, ULONG* o_ulFeatureID)	
SDK:9.2.0.7314(函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
i_nLayerNum	层数
i_bAutoLoop	是否自动生成回路
i_dBackPathZOffset	返回路径Z向偏移，单位mm
i_dTangentialOffset	沿着轨迹切向偏移，单位mm
i_dTangentialLeftOffset	沿着轨迹切向左方偏移，单位mm
i_dTangentialRightOffset	沿着轨迹切向右方偏移，单位mm
o_ulFeatureID	返回添加的多层多道特征ID

C++参考代码

```
void MainWindow :: SampleFun () {
ULONG uPathID = 0;
GetObjIDByName(PQ_PATH, _T("轨迹1"), uPathID);
int i_nLayerNum = 2;
bool i_bAutoLoop = true;
double i_dBackPathZOffset = 1;
double i_dTangentialOffset = 1;
double i_dTangentialLeftOffset = 1;
double i_dTangentialRightOffset = 1;
ULONG o_ulFeatureID = 0;
m_ptrKit->Path_add_multi_layer_weld_feature(uPathID, i_nLayerNum, i_bAutoLoop,                i_dBackP
}
```

5.36.2编译多层多道特征参数

HRESULT Path_modify_multi_layer_weld_feature(const ULONG i_ulFeatureID, INT i_nLayerNum, BOOL i_bAutoLoop, DOUBLE i_dBackPathZOffset, DOUBLE i_dTangentialOffset, DOUBLE i_dTangentialLeftOffset, DOUBLE i_dTangentialRightOffset)	
SDK:9.2.0.7314(函数本体最低支持版本)	
Parameters：	
i_ulFeatureID	多层多道特征ID
i_nLayerNum	层数
i_bAutoLoop	是否自动生成回路
i_dBackPathZOffset	返回路径Z向偏移，单位mm

HRESULT Path_modify_multi_layer_weld_feature(const ULONG i_ulFeatureID, INT i_nLayerNum, BOOL i_bAutoLoop, DOUBLE i_dBackPathZOffset, DOUBLE i_dTangentialOffset, DOUBLE i_dTangentialLeftOffset, DOUBLE i_dTangentialRightOffset)	
i_dTangentialOffset	沿着轨迹切向偏移，单位mm
i_dTangentialLeftOffset	沿着轨迹切向左方偏移，单位mm
i_dTangentialRightOffset	沿着轨迹切向右方偏移，单位mm

C++参考代码

```
void MainWindow :: SampleFun () {
ULONG i_ulFeatureID = 0;
/*自行获取*/ int i_nLayerNum = 2;
bool i_bAutoLoop = true;
double i_dBackPathZOffset = 1;
double i_dTangentialOffset = 1;
double i_dTangentialLeftOffset = 1;
double i_dTangentialRightOffset = 1;
ULONG o_ulFeatureID = 0;
m_ptrKit-> Path_modify_multi_layer_weld_feature (i_ulFeatureID, i_nLayerNum,                i_bAutoL
}
```

5.36.3删除多层多道特征

HRESULT Path_delete_feature(const ULONG i_ulFeatureID)	
SDK:9.2.0.7314(函数本体最低支持版本)	
Parameters：	
i_ulFeatureID	多层多道特征ID

C++参考代码

```
void MainWindow :: SampleFun () {
ULONG i_ulFeatureID = 0;
/*自行获取*/
m_ptrKit-> Path_delete_feature(i_ulFeatureID);
}
```

设置轨迹(轨迹点)联动方式全局额外控制参数

需要先设置外部轴联动方式，不能为常规

HRESULT Path_set_linkage_global_additional_control_data (ULONG* i_uObjectID,INT i_PathCount, ULONG i_ExtJointID, INT i_nExtJointIndex, DOUBLE i_ExtJointValue, BOOL i_bIsFixed, DOUBLE i_dDetectionValue, BOOL i_bIsMutationnDetection, DOUBLE i_dMInJointValue, DOUBLE i_dMaxJointValue)	
SDK:9.2.0.7314(函数本体最低支持版本)	
Parameters：	
uObjectID	指定要设置联动求解全局额外控制参数的轨迹或轨迹点id
i_PathCount	传入的轨迹/轨迹点个数

HRESULT Path_set_linkage_global_additional_control_data (ULONG* i_uObjectID,INT i_PathCount, ULONG i_ExtJointID, INT i_nExtJointIndex, DOUBLE i_ExtJointValue, BOOL i_bIsFixed, DOUBLE i_dDetectionValue, BOOL i_bIsMutationnDetection, DOUBLE i_dMInJointValue, DOUBLE i_dMaxJointValue)	
i_ExtJointID	外部轴/导轨的id
i_nExtJointIndex	外部轴/导轨的索引，如一个外部轴有多个关节，设置第几个关节
i_ExtJointValue	外部轴数值
i_bIsFixed	是否跟随，是设置位True，否设置为False
i_dDetectionValue	突变检测数值
i_bIsMutationnDetection	是否进行突变检测，是传True，否传False
i_dMInJointValue	最小关节值（弧度值）
i_dMaxJointValue	最大关节值（弧度值）

C++参考代码段

```
void MainWindow:: SampleFun () {
ULONG uID;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
ULONG uPathId[1] = {
uID }
;
INT i_PathCount=0;
ULONG i_ExtJointID=0
INT i_nExtJointIndex = 0;
//第几个关节传几
DOUBLE i_ExtJointValue=0;
BOOL i_bIsFixed = False;
DOUBLE i_dDetectionValue=0;
BOOL i_bIsMutationnDetection=False;
DOUBLE i_dMInJointValue=-1.57; //传弧度
DOUBLE i_dMaxJointValue = 1.57; //传弧度
m_ptrKit-> Path_insert_from_joint_without_movement (uPathId, i_PathCount, i_ExtJointID          i_
}
```

轨迹旋转

轨迹绕自身坐标系旋转指定角度（传角度值）

HRESULT Path_Rotate (ULONG* i_ulPathIDs, INT i_nPathIDArraySize, DOUBLE i_dX, DOUBLE i_dY, DOUBLE i_dZ)	
SDK:10.0.0.7068(函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
o_nPointsCount	轨迹点个数
i_dX	绕x轴旋转的角度
i_dY	绕y轴旋转的角度

HRESULT Path_Rotate (ULONG* i_ulPathIDs, INT i_nPathIDArraySize, DOUBLE i_dX, DOUBLE i_dY, DOUBLE i_dZ)	
i_dZ	绕z轴旋转的角度

C++参考代码段

```
void MainWindow:: SampleFun () {
ULONG uID;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
ULONG uPathId[1] = {
uID }
;
INT i_PathCount=1;
m_ptrKit-> Path_Rotate (uPathId, i_PathCount, 0,0,90);
}
```

轨迹平移

轨迹绕自身坐标系平移指定长度（单位mm）

HRESULT Path_Translation (ULONG* i_ulPathIDs, INT i_nPathIDArraySize, DOUBLE i_dX, DOUBLE i_dY, DOUBLE i_dZ)	
SDK:10.0.0.7068(函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
o_nPointsCount	轨迹点个数
i_dX	绕x轴旋转的角度
i_dY	绕y轴旋转的角度

HRESULT Path_Translation (ULONG* i_ulPathIDs, INT i_nPathIDArraySize, DOUBLE i_dX, DOUBLE i_dY, DOUBLE i_dZ)	
i_dZ	绕z轴旋转的角度

C++参考代码段

```
void MainWindow:: SampleFun () {
ULONG uID;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
ULONG uPathId[1] = {
uID }
;
INT i_PathCount=1;
m_ptrKit-> Path_Translation (uPathId, i_PathCount, 0,0,20);
}
```

轨迹反向

HRESULT Path_ Path_Reverse (ULONG* i_ulPathIDs, INT i_nPathIDArraySize, DOUBLE i_dX, DOUBLE i_dY, DOUBLE i_dZ)	
SDK:10.0.0.7068(函数本体最低支持版本)	
Parameters：	
i_ulPathID	轨迹ID
o_nPointsCount	轨迹点个数

C++参考代码段

```
void MainWindow:: SampleFun () {
ULONG uID;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
ULONG uPathId[1] = {
uID }
;
```

```
INT i_PathCount=1;
m_ptrKit-> Path_Reverse (uPathId, i_PathCount);
}
```

添加旋转加工轨迹

调用Doc_add_rotray_path接口可以添加旋转加工轨迹。

HRESULT Doc_add_rotray_path(ULONG i_ulRobot, PQRotrayProcessParameter* i_stProcessParameter, PQRotrayAreaParameter* i_stAreaParameter, PQRotrayCoordinateParameter* i_stCoordinateParameter, PQRotrayConnectParameter* i_stConnectParameter, ULONG* o_ulPathID)	
SDK:9.2.0.7314(函数本体最低支持版本)	
Parameters：	
i_ulRobot	指定轨迹所属机器人的ID
i_stProcessParameter	加工参数： 通过struct_PQRotrayProcessParameter结构体来指定加工参数
i_stAreaParameter	区域参数 通过struct_PQRotrayAreaParameter结构体来指定区域参数
i_stCoordinateParameter	坐标系参数 通过struct_PQRotrayCoordinateParameter结构体来指定坐标系参数
i_stConnectParameter	连接参数 通过struct_PQRotrayConnectParameter结构体来指定连接参数

HRESULT Doc_add_rotray_path(ULONG i_ulRobot, PQRotrayProcessParameter* i_stProcessParameter, PQRotrayAreaParameter* i_stAreaParameter, PQRotrayCoordinateParameter* i_stCoordinateParameter, PQRotrayConnectParameter* i_stConnectParameter, ULONG* o_ulPathID)	
o_ulPathID	返回生成的轨迹的ID

struct_PQRotrayProcessParameter：

ulObjID	**加工曲面所属的零件的ID**
nOutputType	输出类型，赋值范围0 1 2 0 拟合圆弧与点分布 1 多边形与点分布 2 高曲面之类
nMaxStepOverDistance	行距-最大行距
dCutTolerance	弦长比(加工精度)

struct_PQRotrayAreaParameter：

nRotAxis	**旋转轴，赋值范围0 1 2 0 X轴 1 Y轴 2 Z轴**
dStartAngle	角度范围-起始角
dEndAngle	角度范围-终止角
dStartHeight	轴向范围-起始值
dEndHeight	轴向范围-终止值

struct_PQRotrayCoordinateParameter：

ulCoordinateID	**指定坐标系ID**

目前接口仅支持指定坐标系ID，区域参数结构体中的nRotAxis指定的旋转轴也是指定的这里ID对应的坐标系的轴。如需指定世界坐标系，ulCoordinateID赋值0即可。

struct_PQRotrayConnectParameter：

nEntryType	**接近方式，赋值范围0 1 2 3 0 从安全距离接近 1 从快速移动距离接近 2 从慢速移动距离接近 3 直接接近**
nExitType	返回方式，赋值范围0 1 2 3 4 0 返回安全距离 1 返回到快速移动距离 2 返回到慢速移动距离 3 沿柱面中轴返回到安全距离 4 直接返回

C++参考代码

```
void SampleFunc() {
ULONG uRobotID = 0;
GetObjIDByName(PQ_ROBOT, _T(""), uRobotID);
//名称自行填入
ULONG uWPID = 0;
GetObjIDByName(PQ_WORKINGPART, _T(""), uWPID);
ULONG uCoorID = 0;
GetObjIDByName(PQ_COORD, _T(""), uCoorID);
struct_PQRotrayCoordinateParameter pqRCP;
pqRCP.ulCoordinateID = uCoorID;
struct_PQRotrayProcessParameter pqRPP;
pqRPP.ulObjID = uWPID;
pqRPP.nMaxStepOverDistance = 50;
pqRPP.nOutputType = 1;
pqRPP.dCutTolerance = 0.1;
struct_PQRotrayAreaParameter pqRAP;
pqRAP.nRotAxis = 2;
pqRAP.dStartAngle = 0.0;
pqRAP.dEndAngle = 360.0;
pqRAP.dStartHeight = 5.0;
pqRAP.dEndHeight = 150.0;
struct_PQRotrayConnectParameter pRCCP;
pRCCP.nEntryType = 0;
pRCCP.nExitType = 0;
ULONG ulNewPathID = 0;
m_ptrKit->Doc_add_rotray_path(uRobotID, &pqRPP, &pqRAP, &pqRCP, &pRCCP, &ulNewPathID);
return;
}
```

机器人获取轴配置

5.42.1 3R1P机器人获取轴配置

HRESULT Robot3R1P_get_axis_config (ULONG i_uPathID, struct_PQ3R1PConfig* o_eAxisCfg)	
11.1.0.7438 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
o_eAxisCfg	3R1P机器人轴配置信息

C++参考代码段

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobotID);
struct_PQ3R1PConfig i_eAxisCfg ;
m_ptrKit ->Doc_init_3R1Pconfig(&i_eAxisCfg);
m_ptrKit-> PQAPIGetAxisConfig (ulRobotID, &i_eAxisCfg);
```

轨迹注释/取消注释

调用Path_comment接口注释指定轨迹。

HRESULT Path_comment(ULONG *i_ulPathIDs, INT i_nPathIDArraySize)	
SDK:11.2.0.7544	
Parameters：	
i_ulPathIDs	想要注释的轨迹ID数组
i_nPathIDArraySize	想要注释的轨迹ID数组大小

调用Path_uncomment接口取消指定轨迹注释状态。

HRESULT Path_uncomment(ULONG *i_ulPathIDs, INT i_nPathIDArraySize)	
SDK:11.2.0.7544	
Parameters：	
i_ulPathIDs	想要取消注释的轨迹ID数组
i_nPathIDArraySize	想要取消注释的轨迹ID数组大小
## 获取截面线截面法向	

HRESULT Path_get_sectionLine_face_normal (ULONG i_uPathID, DOUBLE** o_FaceNormal,INT* o_nFaceNormalCount)	
11.2.0.7769 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
o_FaceNormal	传出截面线的法向数组
o_nFaceNormalCount	传出数组法向个数

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
double* dNormals = nullptr;
INT nCountFace = 0;
m_ptrKit->Path_get_sectionLine_face_normal(ulPathID, &dNormals, &nCountFace);
```

提取曲线特征生成轨迹

调用Doc_create_path_from_curve接口可以提取多组曲线特征生成轨迹

HRESULT Doc_create_path_from_curve(ULONG* i_uCurveID, INT i_nCurveIDCount, PQCurveShapeParm i_sCurveShapeParm, ULONG i_ulRobotID, ULONG o_ulPathIDs, INT* o_nPathCount)**	
Parameters：	
i_ulRobotID	指定轨迹所属机器人的ID
i_uCurveID	曲线特征ID数组
i_nCurveIDCount	曲线特征ID数组长度
i_sCurveShapeParm	生成曲线轨迹参数 通过struct_PQPFGCurveShapeParm结构体来指定曲线轨迹参数
o_ulPathIDs	返回生成的轨迹组的ID数组
o_nPathCount	返回生成的轨迹组的长度

轨迹生成算法结构体参数struct_PQPFGCurveShapeParm含义如下，可使用Kit_init_curveshape_pram_struct进行初始化：

eCurveSelectType	**曲线类型，赋值范围0 1 2 0 曲线 1 连续线 2 边**
bRev	是否反转
sPathPara	轨迹参数 通过struct_PQPFEPathParameterInfor结构体来指定生成轨迹参数
sPathTransit	出刀入刀设置 通过struct_PQPFEPathTransitInfor结构体来指定生成轨迹出入刀参数
bCurvesMergePath	是否合并（一条曲线生成的一条轨迹）
iCoordinate	机器人轨迹后置坐标系序号
epfPostProc	轨迹后处理操作，默认为0，赋值范围： 0 无 1 z轴旋转最小
nToolType	生成轨迹工具序号
nTCPType	生成轨迹工具TCP序号

目前接口仅支持指定曲线类型为0曲线时生成轨迹。机器人轨迹后置坐标系iCoordinate，轨迹工具序号nToolType，轨迹工具 TCP序号nTCPType默认为-1，默认不另外绑定坐标系和工具TCP。

struct_PQPFEPathParameterInfor：

dInterferenceH	起点内偏，单位mm
dInterferenceT	终点内偏，单位mm
epsPtStepType	轨迹点数控制方式，赋值范围0 1 2 0 按步长方式 1 按点个数方式 2 自动计算
dPointNumer	轨迹点数
dPointStep	点步长，单位mm
bMinPtCnt	是否仅为直线生成首末点
epjType	连接点设置，赋值范围0 1 2 0 不考虑连接点 1 连接处生成一个点 2 连接处生成两个点
bCircleMinPtCnt	是否仅为圆弧生成3个点
dCircleApproximation	拟合圆弧容差（mm）
bGenCirc4Spline	是否为样条生成圆弧点
dCirc4SplineAng	圆周角阈值（°）
bGenLine4Spline	是否稀疏直线
dLine4SplineAng	共线角度阈值（°）
bYaxisAlongTan	是否设置Y轴与切向一致

struct_PQPFEPathTransitInfor：

bEntryExitPt	是否设置出入刀点
dEntryOffset	入刀偏移量（mm）
iEntryDir	入刀方向，默认z轴，赋值0 1 2 0 x轴 1 y轴 2 z轴
dExitOffset	出刀偏移量（mm）

bEntryExitPt	是否设置出入刀点
iExitDir	出刀方向，默认z轴，赋值0 1 2 0 x轴 1 y轴 2 z轴

C++参考代码

```
 void SampleFunc() {
ULONG uRobotID = 0;
GetObjIDByName(32, _T(""), uRobotID);
//名称自行填入
ULONG vCurveID[2] = {
3774873628,3774873629}
;
struct_PQCurveShapeParm pqPathParm;
spDataExchange->Kit_init_curveshape_pram_struct(&pqPathParm);
ULONG* vecPathId = nullptr;
int num(0);
m_ptrKit->Doc_create_path_from_curve(vCurveID, 2, pqPathParm, ulRobotID, &vecPathId, &num);
return;
}
```

轨迹组更名

调用Path_set_folder_name可以设置指定轨迹组/轨迹文件夹的名称。

HRESULT Path_set_folder_name(ULONG i_ulPathFolderID, const WCHAR* i_wsName)	
SDK:11.2.0.7865	
Parameters：	
i_ulPathFolderID	轨迹组/轨迹文件夹ID
i_wsName	新名称
## 获取平坦面轨迹截面法向	

HRESULT Path_get_flat_surfpath_slice_normals (ULONG i_uPathID, DOUBLE** o_FaceNormal,INT* o_nFaceNormalCount)	
11.2.0.7895 (函数本体最低支持版本)	
Parameters：	
i_uPathID	轨迹ID
o_FaceNormal	传出平坦面截面的法向数组
o_nFaceNormalCount	传出数组法向个数

C++参考代码段

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
double* dNormals = nullptr;
INT nCountFace = 0;
m_ptrKit-> Path_get_flat_surfpath_slice_normals (ulPathID, &dNormals, &nCountFace);
```

轨迹点

获取轨迹点ID

HRESULT Path_get_point_id(ULONG i_ulPathID, INT* o_nPointsCount, ULONG** o_ulPointIDs)	
SDK:10.0.0.7068	
Parameters：	
i_ulPathID	轨迹ID
o_nPointsCount	轨迹点个数

HRESULT Path_get_point_id(ULONG i_ulPathID, INT* o_nPointsCount, ULONG** o_ulPointIDs)	
o_ulPointIDs	轨迹点ID集合，数据在PQKit内部创建，需要调用PQAPIFreeArray释放

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"轨迹4", ulPathID);
int nCount = 0;
ULONG* ulPtIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nCount, &ulPtIDs);
ULONG ulPointID= ulPtIDs[0];
m_ptrKit->PQAPIFreeArray((LONG_PTR*)ulPtIDs);
```

获取轨迹点信息

PQKit提供了多种获取轨迹点信息的接口。

6.2.1 PQAPIGetPointInfo可以获取指定ID的轨迹点基于世界坐标系的点位姿、点线速度、点运动指令等。

HRESULT PQAPIGetPointInfo(ULONG i_uID, PQPostureType i_nPostureType, INT* o_nPostureCount, DOUBLE** o_dPointPosture, DOUBLE* o_dVelocity, DOUBLE* o_dSpeedPercent, INT* o_nInstruct, INT* o_nApproach)	
SDK:10.0.0.7068	
Parameters：	
i_uID	轨迹点ID
i_nPostureType	指定姿态表示方式：欧拉角、四元素
i_nPostureCount	返回轨迹点位姿数组的长度

HRESULT PQAPIGetPointInfo(ULONG i_uID, PQPostureType i_nPostureType, INT* o_nPostureCount, DOUBLE** o_dPointPosture, DOUBLE* o_dVelocity, DOUBLE* o_dSpeedPercent, INT* o_nInstruct, INT* o_nApproach)	
o_dPointPosture	返回轨迹点位姿数组，位姿数组需要调用PQAPIFreeArray释放
o_dVelocity	轨迹点速度
o_dSpeedPercent	轨迹点速度百分比
o_nInstruct	点指令
o_nApproach	轨迹逼近

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
PQPostureType nPostureType = EULERANGLEXYZ;
INT nPostureCount = 6;
double* dPointPosture = nullptr;
double dVelocity = 0;
double dSpeedPercent = 0;
PQPointInstruction nInstruct= PQ_LINE;
INT nApproach = 0;
m_ptrKit->PQAPIGetPointInfo(ulPointID, nPostureType, &nPostureCount, &dPointPosture, &dVelocity, &d
m_ptrKit->PQAPIFree((LONG_PTR*)dPointPosture);
```

6.2.2 Point_get_posture_in_path_tcp_coordinate，可以获取指定ID的轨迹点在其所属轨迹使用的TCP和坐标系下的位姿。

HRESULT Point_get_posture_in_path_tcp_coordinate(ULONG i_ulPointID, PQPostureType i_ePostureType, INT* o_nPostureCount, DOUBLE o_dPointPosture)**	
SDK:10.0.0.7068	
Parameters：	
i_ulPointID	轨迹点ID
i_nPostureType	指定姿态表示方式：欧拉角、四元素
i_nPostureCount	返回轨迹点位姿数组的长度
o_dPointPosture	返回轨迹点位姿数组，位姿数组需要调用PQAPIFreeArray释放

Point_get_posture_in_path_tcp_coordinate接口中轨迹点位姿数据内存是在PQKit中分配创建，调用后需要手动调用PQAPIFreeArray释放。PQKit也提供了另一版本接口Point_get_posture_in_path_tcp_coordinate_unsafe，调用者需要创建数组，将指针传入，PQKit写入点位姿数据。

HRESULT Point_get_posture_in_path_tcp_coordinate_unsafe(ULONG i_ulPointID, PQPostureType i_nPostureType, DOUBLE* i_dPointPosture, INT i_nPostureArraySize)	
Parameters：	
i_ulPointID	轨迹点ID
i_nPostureType	指定姿态表示方式：欧拉角、四元素

HRESULT Point_get_posture_in_path_tcp_coordinate_unsafe(ULONG i_ulPointID, PQPostureType i_nPostureType, DOUBLE* i_dPointPosture, INT i_nPostureArraySize)	
i_dPointPosture	接收轨迹点位姿的数组
i_nPostureArraySize	接收轨迹点位姿的数组的大小

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
int nPostureCount = 0;
double* dPosture = nullptr;
m_ptrKit->Point_get_posture_in_path_tcp_coordinate(ulPointID, EULERANGLEXYZ, &nPostureCount, &dPost
m_ptrKit->PQAPIFreeArray((LONG_PTR*)dPosture);
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
double dPosture[6] = {
0.00 }
;
m_ptrKit->Point_get_posture_in_path_tcp_coordinate_unsafe(ulPointID, EULERANGLEXYZ, dPosture, 6);
```

获取轨迹点自定义事件

获取轨迹点下的自定义事件信息

HRESULT PQAPIGetPointCustomEventInfo(ULONG i_uID, BOOL* o_bHasCustomEvent, BSTR* o_sName, BSTR* o_sContent, BSTR* o_sPosition)	
SDK:10.0.0.7068	
Parameters：	
i_uID	轨迹点ID
o_bHasCustomEvent	轨迹点是否有自定义事件
o_sName	轨迹点下自定义事件名称，多个自定义事件以#分割
o_sContent	轨迹点下自定义事件内容，多个自定义事件以#分割
o_sPosition	轨迹点下自定义事件点前点后位置，多个自定义事件以#分割。其中0点后执行，1点前执行

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
long bHasCustomEvent = 0;
CComBSTR sName="";
CComBSTR sContent="";
CComBSTR sPosition="";
m_ptrKit->PQAPIGetPointCustomEventInfo(ulPointID, &bHasCustomEvent, &sName, &sContent, &sPosition);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

轨迹点逆运动学求解

根据轨迹点位姿进行逆运动学求解（原PQIKCalInverseKinematics6R废弃）

HRESULT PQAPIInverseKinematics(ULONG i_ulRobotID, DOUBLE* i_EndPosture,INT i_nEndPostureCount, PQPostureType i_ePostureType, DOUBLE* io_pJointValues, INT i_nJointValuesCount, INT* i_nAxisCfg, INT i_nAxisCfgCount, INT* o_pPtStatus, BOOL i_bToolEndPosture)	
SDK:10.0.0.7068	
Parameters：	
i_ulRobotID	指定机器人ID
i_EndPosture	欲逆运动学求解点位姿，以欧拉角或四元素表示
i_nEndPostureCount	i_EndPosture数组个数
i_nPostureType	点位姿表达方式
io_pJointValues	传入上一组关节角(弧度)，输出逆运动学求解结果
i_nJointValuesCount	io_pJointValues 数组个数
i_nAxisCfg	轴配置数组（按1246轴顺序），详见9.7轴配置定义
i_nAxisCfgCount	轴配置数组个数
o_pPtStatus	该点机器人可达性，详见9.6轨迹可达性状态定义

HRESULT PQAPIInverseKinematics(ULONG i_ulRobotID, DOUBLE* i_EndPosture,INT i_nEndPostureCount, PQPostureType i_ePostureType, DOUBLE* io_pJointValues, INT i_nJointValuesCount, INT* i_nAxisCfg, INT i_nAxisCfgCount, INT* o_pPtStatus, BOOL i_bToolEndPosture)	
i_bToolEndPosture	表示传入位姿是否为工具末端位姿

C++参考代码

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobotID);
DOUBLE EndPosture[6] = {
}
;
INT nEndPostureCount = 6;
DOUBLE pJointValues[6] = {
}
;
INT nJointValuesCount = 6;
INT nAxisCfg[4] = {
}
;
INT nAxisCfgCount = 4;
INT pPtStatus = 0;
BOOL  bToolEndPosture = true;
m_ptrKit->PQAPIInverseKinematics(ulRobotID, EndPosture, nEndPostureCount, EULERANGLEXYZ, pJointValu
```

FANUC轨迹点逆运动学求解

发那科机器人逆运动学求解调用此接口。

HRESULT PQAPIInverseKinematicsFanuc(ULONG i_ulRobotID, DOUBLE* i_EndPosture, INT i_nEndPostureCount, DOUBLE* io_pJointValues, INT i_nJointValuesCount, INT* i_nAxisCfg, INT i_nAxisCfgCount, INT* o_pPtStatus)	
SDK:10.0.0.7068	
Parameters：	
i_ulRobotID	指定机器人ID
i_EndPosture	欲逆运动学求解点位姿
i_nEndPostureCount	i_EndPosture数组个数
io_pJointValues	传入上一组关节角(弧度)，输出逆运动学求解结果
i_nJointValuesCount	io_pJointValues 数组个数
i_nAxisCfg	轴配置数组
i_nAxisCfgCount	轴配置数组个数
o_pPtStatus	该点机器人可达性，详见9.6轨迹可达性状态定义

C++参考代码

```
ULONG ulRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobotID);
DOUBLE EndPosture[6] = {
}
;
INT nEndPostureCount = 6;
DOUBLE pJointValues[6] = {
```

```
}
;
INT nJointValuesCount = 6;
INT nAxisCfg[6] = {
}
;
INT nAxisCfgCount = 6;
INT o_pPtStatus = 0;
m_ptrKit->PQAPIInverseKinematicsFanuc(ulRobotID, EndPosture, nEndPostureCount, pJointValues, nJoint
```

注意：

- i_EndPosture为X Y Z W P R格式顺序，robx工程场景中若机器人未安装工具则传入FL数据，若机器人安装工具则传入TCP数据
- i_nAxisCfg为6位int数组，对应发那科轴配置转换如下：

前三位N：0 F：1 U：0 D：1 T：0 B：1

后三位数字直接传

轨迹点对应关节角

获取轨迹点处机器人关节角数据，轨迹需编译后才能获取

HRESULT PQAPIGetRobotJointsFromPoints(ULONG i_ulPointID, VARIANT* o_varJointsArray)	
SDK:10.0.0.7068	
Parameters：	
i_ulPointID	轨迹点ID
o_varJointsArray	机器人关节角(弧度)

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
```

```
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
VARIANT varJointsArray;
m_ptrKit->PQAPIGetRobotJointsFromPoints(ulPointID, &varJointsArray);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

修改轨迹点位姿

在PQKit产品中修改轨迹点API为PQAPIModifyPointPosture。

HRESULT PQAPIModifyPointPosture(ULONG i_ulPointID, DOUBLE* i_dPosition, INT i_nPositionCount , PQPostureType i_ePostureType)	
SDK:10.0.0.7068	
Parameters：	
i_ulPointID	轨迹点ID
i_dPosition	轨迹点位姿
i_nPositionCount	轨迹点姿态数组大小
i_nPosType	指定姿态表示方式：欧拉角、四元素

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
DOUBLE dPosition[6] = {
}
;
```

```
INT nPositionCount = 6;
INT nPosType = 0;
m_ptrKit->PQAPIModifyPointPosture(ulPointID, dPosition, nPositionCount, EULERANGLEXYZ);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

标准二次开发中修改轨迹点API为Point_modify_poture，其参数与PQAPIModifyPointPosture一致。

批量修改轨迹点位姿

HRESULT Point_modify_poture_batch(ULONG i_ulPathID, INT i_nStartIndex, INT i_nEndIndex, DOUBLE *i_dPosture, INT i_nPostureArraySize, PQPostureType i_ePostureType)	
SDK:10.0.0.7068	
Parameters：	
i_ulPathID	欲批量修改的轨迹点所归属的轨迹ID
i_nStartIndex	一批轨迹点的起始序号，从0开始计数
i_nEndIndex	一批轨迹点的终止序号
i_dPosture	这一批轨迹点的位姿数据
i_nPostureArraySize	位姿数组数组大小
i_nPostureType	位姿姿态表达方式

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nStartIndex = 0;
int nEndIndex = 5;
double dPosition[30] = {
}
;
int nPostureSize = 30;
m_ptrKit->Point_modify_poture_batch(ulPathID, nStartIndex, nEndIndex, dPosition, nPostureSize, EULE
```

添加自定义事件

HRESULT PQAPIAddCustomEvent(ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, BSTR i_bsEventName, BSTR i_bsContent)	
SDK:10.0.0.7068	
Parameters：	
i_uPointsID	欲添加自定义事件的轨迹点ID
i_nPointCount	欲添加自定义事件的轨迹点数组大小
i_uExecuteObjID	自定义事件的执行设备ID
i_nEventPosition	自定义事件点前点后执行，1：点前 0：点后
i_bsEventName	自定义事件名称
i_bsContent	自定义事件内容

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
ULONG ulRobID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobID);
CComBSTR bsEventName = "";
CComBSTR bsContent = "";
m_ptrKit->PQAPIAddCustomEvent(&ulPointID, 1, ulRobID, 1, bsEventName, bsContent);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

添加协同事件

HRESULT Path_add_SynSetting_event (ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, WCHAR* i_bsEventName, ULONG* i_uSynSettingEngineID, INT i_nSynSettingEngineCount)	
SDK:10.0.0.7068	
Parameters：	
i_uPointsID	欲添加协同事件的轨迹点ID
i_nPointCount	欲添加协同事件的轨迹点数组大小
i_uExecuteObjID	自定义事件的执行设备ID
i_nEventPosition	自定义事件点前点后执行，1：点前 0：点后

HRESULT Path_add_SynSetting_event (ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, WCHAR* i_bsEventName, ULONG* i_uSynSettingEngineID, INT i_nSynSettingEngineCount)	
i_bsEventName	自定义事件名称
i_uSynSettingEngineID	协同的设备id数组
i_nSynSettingEngineCount	协同设备数组的个数

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
ULONG ulRobID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobID);
ULONG ulSettingEngineID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulSettingEngineID);
CComBSTR bsEventName = "";
m_ptrKit->Path_add_SynSetting_event(&ulPointID, 1, ulRobID, 1, bsEventName, &ulSettingEngineID, 1);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

添加抓取放开事件

HRESULT PQAPICreateCatchEvent(ULONG i_uPartID, ULONG i_uPointID,BOOL i_bPrePoint, BOOL i_bCatch, BSTR i_EventName)	
SDK:10.0.0.7068	
Parameters：	

HRESULT PQAPICreateCatchEvent(ULONG i_uPartID, ULONG i_uPointID,BOOL i_bPrePoint, BOOL i_bCatch, BSTR i_EventName)	
i_uPartID	欲抓取的零件ID
i_uPointID	事件要挂载到的轨迹点的ID
i_bPrePoint	事件点前点后执行 1：点前 0：点后
i_bCatch	是否抓取事件 1：抓取 0：放开
i_EventName	事件名称，可不指定

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
ULONG ulPartID = 0;
GetObjIDByName(PQ_WORKINGPART, L"", ulPartID);
CComBSTR bsEventName = "";
m_ptrKit->PQAPICreateCatchEvent(ulPartID ,ulPointID, 1,  1, bsEventName);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

删除轨迹点

删除轨迹点需要构建欲删除轨迹点的ID数组，并通过varParam参数传递。删除支持单独删除和多选删除，即使单选也需要构造一个只有一个元素的数组。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:10.0.0.7068	
Parameters：	
bsCommandID	"RO_CMD_DELETE_POINT"
wParam	0或NULL
lParam	0或NULL
bsParam	""
varParam	欲删除轨迹点的ID数组
lResult	0

C#参考代码段

```
private void func_delete_pt(object sender, EventArgs e) { string strCMD = "RO_CMD_DELETE_POINT"; st
```

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG uIDs[3] = {
ulPointsIDs[0],ulPointsIDs[1],ulPointsIDs[2]}
;
ULONG* pdData=NULL;
SAFEARRAY* psa = NULL;
VARIANT pvarArray;
pvarArray.parray = NULL;
SafeArrayAccessData(psa, (void**)&pdData);
for (int i = 0; i < 3; i++) pdData[i] = ((ULONG*)uIDs)[i];
SafeArrayUnaccessData(psa); pvarArray.parray = psa; long long result;
```

```
CComBSTR cmd = "RO_CMD_DELETE_POINT";
CComBSTR bsParam="";
m_ptrKit->pq_RunCommand(cmd, NULL, NULL, bsParam, pvarArray, &result);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

批量获取轨迹点位姿

HRESULT Point_get_posture_batch(ULONG i_ulPathID, INT i_nStartIndex, INT i_nCount, PQPostureType i_nPostureType, ULONG i_uCoordinateID, INT* o_nPostureArraySize，DOUBLE **o_dPosture)	
SDK:10.0.0.7068	
Parameters：	
ulPathID	轨迹点归属的轨迹ID
I_nStartIndex	批量获取的轨迹点起始索引，从0开始计数
i_nCount	批量获取的轨迹点的个数
i_nPostureType	指定姿态表示方式：欧拉角、四元素
i_uCoordinateID	坐标系ID，指定获取基于哪个坐标系下的位姿
o_nPostureArraySize	传出轨迹点位姿数组大小
o_dPosture	轨迹点位姿数组，按指定姿态表达方式按序排列，轨迹点位姿数组需要调用PQAPIFree释放

C#参考代码段

```
private
void func () {
uint uPathID = m_listPaths.First();
```

```
IntPtr ptrPosture = IntPtr.Zero;
int nCount = 0;
uint uCoordianteID = 0;
// GetObjIDByName("your coordinate name", 176, out uCoordianteID);
m_ptrKit.Point_get_posture_batch(uPathID, 0,82,1, uCoordianteID,out nCount, ref ptrPosture);
double[] dPostures = new double[nCount];
Marshal.Copy(ptrPosture, dPostures, 0, nCount);
m_ptrKit.PQAPIFreeArray((long)ptrPosture);
}
```

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
ULONG ulCoorID = 0;
GetObjIDByName(PQ_COORD, L"", ulCoorID);
int nPostureSize = 0;
double *dPosture = nullptr;
m_ptrKit->Point_get_posture_batch(ulPathID, 5, 5, EULERANGLEXYZ, ulCoorID, &nPostureSize, &dPosture
m_ptrKit->PQAPIFreeArray((LONG_PTR*) dPosture);
```

添加发送事件

HRESULT Path_add_send_event (ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, WCHAR* i_bsEventName)	
SDK:10.0.0.7068	
Parameters：	
i_uPointsID	欲添加发送事件的轨迹点ID
i_nPointCount	欲添加发送事件的轨迹点数组大小
i_uExecuteObjID	发送事件的执行设备ID

HRESULT Path_add_send_event (ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, WCHAR* i_bsEventName)	
i_nEventPosition	发送事件点前点后执行，1：点前 0：点后
i_bsEventName	发送事件名称

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
ULONG ulRobID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobID);
CComBSTR bsEventName = "";
m_ptrKit->Path_add_send_event(&ulPointID, 1, ulRobID, 1 , bsEventName);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

添加等待时间事件

HRESULT Path_add_waittime_event (ULONG * i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, DOUBLE i_dWaitTime, WCHAR* i_bsEventName)	
SDK:10.0.0.7068	
Parameters：	
i_uPointsID	欲添加等待时间事件的轨迹点ID

HRESULT Path_add_waittime_event (ULONG * i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, DOUBLE i_dWaitTime, WCHAR* i_bsEventName)	
i_nPointCount	欲添加等待时间事件的轨迹点数组大小
i_uExecuteObjID	等待时间事件的执行设备ID
i_nEventPosition	等待时间事件点前点后执行，1：点前 0：点后
i_dWaitTime	需要等待的事件，单位秒
i_bsEventName	等待时间事件名称

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
ULONG ulRobID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobID);
CComBSTR bsEventName = "";
m_ptrKit->Path_add_waittime_event(&ulPointID, 1, ulRobID, 1 , 3.5,bsEventName);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

添加等待事件

HRESULT Path_add_wait_event(ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, WCHAR* i_bsEventName, WCHAR* i_bsWaitEventName)	
SDK:10.0.0.7068	
Parameters：	
i_uPointsID	欲添加等待事件的轨迹点ID
i_nPointCount	欲添加等待事件的轨迹点数组大小
i_uExecuteObjID	等待事件的执行设备ID
i_nEventPosition	等待事件点前点后执行，1：点前 0：点后
i_bsEventName	等待事件名称
i_bsWaitEventName	等待事件内容

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
ULONG ulRobID = 0;
GetObjIDByName(PQ_ROBOT, L"", ulRobID);
CComBSTR bsEventName = "";
CComBSTR bsWaitEventName = "";
m_ptrKit->Path_add_wait_event(&ulPointID, 1, ulRobID, 1 ,bsEventName, bsWaitEventName);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

获取轨迹点的轨迹ID

获取轨迹点所属的轨迹ID

HRESULT Point_get_parent_path(ULONG i_uPointID, ULONG * o_uPathID)	
SDK:10.0.0.7068	
Parameters：	
i_uPointID	传入轨迹点ID
o_uPathID	传出轨迹点所属的轨迹ID

C++参考代码

```
ULONG uPointID = 65475624;
ULONG uPathID= 0;
m_ptrKit->Point_get_parent_path(uPointID, &uPathID);
```

获取零件下的轨迹点

通过零件ID获取零件下轨迹点的个数、名字及ID

HRESULT Part_get_path(ULONG i_ulPartID, INT* o_nCount, WCHAR o_whPathNames, ULONG** o_uPathIDs)**	
SDK:10.0.0.7068	
Parameters：	
i_ulPartID	传入零件ID
o_nCount	传出轨迹点个数
o_whPath	传出轨迹点名字,轨迹点名字需要调用PQAPIFree释放

HRESULT Part_get_path(ULONG i_ulPartID, INT* o_nCount, WCHAR** o_whPathNames, ULONG** o_uPathIDs)	
o_uPathIDs	传出轨迹点ID，轨迹点ID数组需要调用PQAPIFree释放

C++参考代码

```
ULONG uPartID = 0;
GetObjIDByName(PQ_WORKINGPART, L"", uPartID);
INT nCount = 0;
WCHAR* whPathNames = nullptr;
ULONG* uPathIDs = nullptr;
m_ptrKit->Part_get_path(uPartID, &nCount, &whPathNames, &uPathIDs);
m_ptrKit->PQAPIFree((LONG_PTR*)whPathNames);
m_ptrKit->PQAPIFree((LONG_PTR*)uPathIDs);
```

轨迹点过渡

通过设置起点和终点实现轨迹过渡

HRESULT Path_set_transition(ULONG i_ulPointIDStart, DOUBLE i_dXStart, DOUBLE i_dYStart, DOUBLE i_dZStart, ULONG i_ulPointIDEnd, DOUBLE i_dXEnd, DOUBLE i_dYEnd, DOUBLE i_dZEnd)	
SDK:10.0.0.7068	
Parameters：	
i_ulPointIDStart	传入轨迹起点ID
i_dXStart	传入轨迹起点x轴坐标
i_dYStart	传入轨迹起点y

HRESULT Path_set_transition(ULONG i_ulPointIDStart, DOUBLE i_dXStart, DOUBLE i_dYStart, DOUBLE i_dZStart, ULONG i_ulPointIDEnd, DOUBLE i_dXEnd, DOUBLE i_dYEnd, DOUBLE i_dZEnd)	
	轴坐标
i_dZStart	传入轨迹起点z轴坐标
i_ulPointIDEnd	传入轨迹终点ID
i_dXEn	传入轨迹终点x轴坐标
i_dyEn	传入轨迹终点y轴坐标
i_dzEn	传入轨迹终点z轴坐标

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointIDStart = ulPointsIDs[0];
ULONG ulPointIDEnd = ulPointsIDs[nPointsCount - 1];
m_ptrKit->Path_set_transition(ulPointIDStart, 20, 3, 5, ulPointIDEnd, 3, 10, 8);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

添加喷涂事件

HRESULT Path_add_spray_event (ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, WCHAR* i_bsEventName, LONG* i_ulNozzleID, ULONG* i_ulAssistorID, INT i_nNozzleSize,INT i_nAssistorSize, BOOL i_bUnion)	
SDK:10.0.0.7068	
Parameters：	
i_uPointsID	欲添加喷涂事件的轨迹点ID
i_nPointCount	欲添加喷涂事件的轨迹点数组大小
i_uExecuteObjID	喷涂事件的执行设备ID
i_nEventPosition	喷涂事件点前点后执行，1：点前0：点后
i_bsEventName	喷涂事件名称
i_ulNozzleID	喷嘴ID数组
i_ulAssistorID	喷刷ID数组
i_nNozzleSize	喷嘴个数
i_nAssistorSize	喷刷个数
i_bUnion	是否为并，传入false为交集

C++参考代码

```
ULONG uPath1 = 0;
GetObjIDByName(PQ_PATH, L"过渡点1", uPath1);
int nPointsCount1 = 0;
ULONG* ulPointsIDs1 = nullptr;
m_ptrKit->Path_get_point_id(uPath1, &nPointsCount1, &ulPointsIDs1);
ULONG uIDs= ulPointsIDs1[0];
ULONG uPath2 = 0;
GetObjIDByName(PQ_PATH, L"过渡点2", uPath2);
int nPointsCount2 = 0;
ULONG* ulPointsIDs2 = nullptr;
m_ptrKit->Path_get_point_id(uPath2, &nPointsCount2, &ulPointsIDs2);
ULONG uIDs2 = ulPointsIDs2[0];
ULONG uRobID = 0;
GetObjIDByName(PQ_ROBOT, L"ABB-IRB1410", uRobID);
int nNozzleSize = 0;
WCHAR* wszNozzleName = NULL;
ULONG* ulNozzleIDs = NULL;
m_ptrKit->Doc_get_nozzle(uRobID, &nNozzleSize, &wszNozzleName, &ulNozzleIDs);
int nAssistorSize = 0;
WCHAR* wszAssistorName = NULL;
ULONG* ulAssistorIDs = NULL;
m_ptrKit->Doc_get_assistor(ulNozzleIDs[0], &nAssistorSize, &wszAssistorName, &ulAssistorIDs);
int nAssistorSize1 = 0;
WCHAR* wszAssistorName1 = NULL;
ULONG* ulAssistorIDs1 = NULL;
m_ptrKit->Doc_get_assistor(ulNozzleIDs[1], &nAssistorSize1, &wszAssistorName1, &ulAssistorIDs1);
ULONG ulNozzleID[2] = {
ulNozzleIDs[0],ulNozzleIDs[1] }
;
ULONG ulAssistorID[2] = {
ulAssistorIDs[1],ulAssistorIDs1[1] }
;
m_ptrKit->Path_add_spray_event(&uIDs, 1, uRobID, 0, L"开枪", ulNozzleID, ulAssistorID, 2,2, false);
ULONG ulAssistorID1[2] = {
ulAssistorIDs[0],ulAssistorIDs1[0] }
;
m_ptrKit->Path_add_spray_event(&uIDs2, 1, uRobID, 0, L"关枪", ulNozzleID, ulAssistorID1, 2,2, false
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) wszAssistorName);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulAssistorIDs);
```

```
m_ptrKit->PQAPIFreeArray((LONG_PTR*) wszAssistorName1);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulAssistorIDs1);
```

添加寻位轨迹

HRESULT Path_add_positioning(ULONG i_ulRobotID, const WCHAR* i_wSurfaceName, double* i_dPoint, int i_dPointsize, PQNormType* i_pqNormaltype, double i_dTiltAngle, double i_dHeight, double i_dOffset, double i_dDepth, WCHAR* i_whGroupName, PQPointInstruction i_eInstruction, ULONG* o_ulPathID)	
SDK:10.0.0.7314	
Parameters：	
i_ulRobotID	机器人ID
i_wSurfaceName	点在面上的ID,以“，”分隔开，比如“6171#5,6172#6,6175#2”
i_dPoint	寻位点的坐标xyz
i_dPointsize	寻位点个数
i_pqNormaltype	工具方向
i_dTiltAngle	倾斜角
i_dHeight	寻位高度
i_dOffset	偏移量
i_dDepth	寻位深度
i_whGroupName	轨迹名
i_eInstruction	指令设置
o_ulPathID	轨迹ID

C++参考代码

```
ULONG uRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"EFORT-ARC12-2000", uRobotID);
double point[9] = {
6056.736685 ,-1523.027771 ,354.432950 ,5915.9281 ,-1464.222291 ,324.54300 ,5939.164001,-1555.390946
;
PQPointInstruction type1 = PQ_LINE;
PQNormType type[3] = {
PQNormType_Norm ,PQNormType_Norm ,PQNormType_Norm }
;
ULONG uid=0;
wchar_t FACE[256] = L"6171#5,6172#6,6175#2";
CComBSTR PathName = L"Group";
m_ptrKit->Path_add_positioning(uRobotID, LPWSTR(FACE), point, 3, type, 45, 10, 0, 50, PathName, typ
```

修改轨迹点速度和逼近方式

HRESULT Point_modify_velocity_and_approch(ULONG i_ulPointID, DOUBLE i_dVelocity,INT i_nApproach)	
SDK:10.0.0.7314	
Parameters：	
i_ulPointID	轨迹点ID
i_dVelocity	修改轨迹点速度
i_nApproach	修改轨迹点逼近方式

C++参考代码

```
ULONG uID=0;
GetObjIDByName(PQ_PATH, L"过渡点6", uID);
VARIANT varPoint;
varPoint.parray = NULL;
m_ptrKit->PQAPIGetPointsID(uID, &varPoint);
```

```
long lenPointCount = varPoint.parray->rgsabound[0].cElements;
ULONG* t_bufPoint = NULL;
SafeArrayAccessData(varPoint.parray, (void**)&t_bufPoint);
ULONG uPointID = t_bufPoint[0];
m_ptrKit->Point_modify_velocity_and_approch(uPointID, 100, - 1);
```

添加寻位事件

注：所有寻位事件添加完成后需要调用Doc_update_view刷新轨迹树

HRESULT Point_add_search_instruct_event(ULONG i_ulPointID, INT i_nEventPosition, LPWSTR i_whEventName, LPWSTR i_whSaveResult, LPWSTR i_whSearchResult, PQSearchDirection i_eSearchDirection, DOUBLE* i_dSearchDistance, INT i_nDistanceCount)	
SDK:10.0.0.7314	
Parameters：	
i_ulPointID	轨迹点ID
i_nEventPosition	输出位置，1代表点前，0代表点后
i_whEventName	事件名称
i_whSaveResult	结果保存
	已寻位置结果名称（若第一次添加事件或者名称错误，会设置为无）
i_eSearchDirection	寻位方向x,y,z,xyz分别用PQ_DirectionX，PQ_DirectionY，PQ_DirectionZ，PQ_DirectionXYZ表示
i_dSearchDistance	寻位距离数组

HRESULT Point_add_search_instruct_event(ULONG i_ulPointID, INT i_nEventPosition, LPWSTR i_whEventName, LPWSTR i_whSaveResult, LPWSTR i_whSearchResult, PQSearchDirection i_eSearchDirection, DOUBLE* i_dSearchDistance, INT i_nDistanceCount)	
i_nDistanceCount	寻位距离数组个数

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
wchar_t i_whEventName[] = L"name";
wchar_t i_whSaveResult[] = L"saveResult";
wchar_t i_whSearchResult[] = L"m";
PQSearchDirection i_eSearchDirection = PQ_DirectionXYZ;
DOUBLE i_dSearchDistance[3] = {
1,2,3 }
;
m_ptrKit->Point_add_search_instruct_event(ulPointID, 0, i_whEventName, i_whSaveResult,i_whSearchRes
m_ptrKit->Doc_update_view();
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

按照指定位置插入点

在某轨迹点后延着轨迹线，在指定距离的位置插入轨迹点，需要保证轨迹点所属轨迹个数大于1

注：所有点添加完成后需要调用Doc_update_view刷新轨迹树

HRESULT Path_insert_point_distance (ULONG i_ulPointID, DOUBLE i_dInsertDistance, ULONG* o_uInsertPointID)	
SDK:10.0.0.7314	
Parameters：	

HRESULT Path_insert_point_distance (ULONG i_ulPointID, DOUBLE i_dInsertDistance, ULONG* o_uInsertPointID)	
i_ulPointID	轨迹点ID
i_dInsertDistance	指定距离,单位mm
o_uInsertPointID	传出新增加点的id

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
int nPointsCount = 0;
ULONG* ulPointsIDs = nullptr;
m_ptrKit->Path_get_point_id(ulPathID, &nPointsCount, &ulPointsIDs);
ULONG ulPointID = ulPointsIDs[0];
ULONG o_uInsertPointID = 0;
m_ptrKit->Path_insert_point_distance(ulPointID, 5, &o_uInsertPointID);
m_ptrKit->Doc_update_view();
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

批量获取轨迹中轨迹点使用tcp坐标系下的位姿

批量获取数据量可能较大，使用接口后请及时调用PQFreeArray释放内存

HRESULT Point_get_posture_in_path_tcp_coordinate_batch (ULONG i_ulPathID, INT i_nStartIndex, INT i_nCount, PQPostureType i_nPostureType, INT* o_nPostureCount, DOUBLE** o_dPointPosture)	
SDK:10.0.0.7314	
Parameters：	
i_ulPathID	轨迹ID
i_nStartIndex	一批轨迹点的起始

HRESULT Point_get_posture_in_path_tcp_coordinate_batch (ULONG i_ulPathID, INT i_nStartIndex, INT i_nCount, PQPostureType i_nPostureType, INT* o_nPostureCount, DOUBLE** o_dPointPosture)	
	序号，从0开始计数
i_nCount	一批轨迹点的终止序号
i_nPostureType	指定传出位姿态类型
o_nPostureCount	传出轨迹点位姿数组的长度
o_dPointPosture	传出轨迹点位姿数组

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT nStartIndex = 0;
INT nCount = 5;
INT nPostureCount = 0;
DOUBLE* dPointPosture = NULL;
m_ptrKit->Point_get_posture_in_path_tcp_coordinate_batch(ulPathID, nStartIndex, nCount,EULERANGLEXY
m_ptrKit->PQAPIFreeArray(((LONG_PTR*)dPointPosture));
```

获取轨迹中轨迹点分组信息

HRESULT Path_get_point_group_info(ULONG i_ulPathID, INT* o_nGroupCount, INT** o_nGroupArray)	
SDK:10.0.0.7314	
Parameters：	
i_ulPathID	轨迹ID
o_nGroupCount	传出轨迹组个数数组的长度
o_nGroupArray	传出轨迹组个数数组

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT nPostureType = 0;
INT nGroupCount = 0;
INT* nGroupArray = NULL;
m_ptrKit->Path_get_point_group_info(ulPathID,&nGroupCount, &nGroupArray);
m_ptrKit->PQAPIFreeArray(((LONG_PTR*)nGroupArray));
```

批量获取轨迹中轨迹点的关节角

批量获取数据量可能较大，使用接口后请及时调用PQFreeArray释放内存

HRESULT Path_get_point_joint_batch (ULONG i_ulPathID, INT i_nStartIndex, INT i_nCount, INT* o_dJointArrayCount, DOUBLE** o_nGroupArray)	
SDK:10.0.0.7314	
Parameters：	
i_ulPathID	轨迹点ID
i_nStartIndex	一批轨迹点的起始序号，从0开始

HRESULT Path_get_point_joint_batch (ULONG i_ulPathID, INT i_nStartIndex, INT i_nCount, INT* o_dJointArrayCount, DOUBLE o_nGroupArray)**	
	计数
i_nCount	一批轨迹点的终止序号
o_dJointArrayCount	传出关节角数组的长度
o_nGroupArray	传出关节角数组，弧度

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT nStartIndex = 0;
INT nCount = 5;
INT dJointArrayCount = 0;
DOUBLE* nGroupArray = NULL;
m_ptrKit->Path_get_point_joint_batch(ulPathID, nStartIndex, nCount, &dJointArrayCount, &nGroupArray
m_ptrKit->PQAPIFreeArray(((LONG_PTR*)nGroupArray));
```

批量获取轨迹中轨迹点外部轴位置信息

批量获取数据量可能较大，使用接口后请及时调用PQFreeArray释放内存

HRESULT Path_get_point_external_joint_batch (ULONG i_ulPathID, INT i_nStartIndex, INT i_nCount, INT* o_nExtCountArray, ULONG o_uExtJointID, INT* o_nExtJointArrayCount, DOUBLE** o_nExtJointArray)**	
SDK:10.0.0.7314	
Parameters：	
i_ulPathID	轨迹点ID

HRESULT Path_get_point_external_joint_batch (ULONG i_ulPathID, INT i_nStartIndex, INT i_nCount, INT* o_nExtCountArray, ULONG o_uExtJointID, INT* o_nExtJointArrayCount, DOUBLE** o_nExtJointArray)**	
i_nStartIndex	一批轨迹点的起始序号，从0开始计数
i_nCount	一批轨迹点的终止序号
i_nPostureType	指定传出位姿态类型，详情科参考9.1
o_nExtCountArray	传出外部轴id数组长度，如有三个外部轴就会返回3
o_uExtJointID	传出外部轴id数组
o_nExtJointArrayCount	传出外部轴位置/角度数组长度
o_nExtJointArray	传出外部轴数组

C++参考代码

```
ULONG ulPathID = 0;
GetObjIDByName(PQ_PATH, L"", ulPathID);
INT nStartIndex = 0;
INT nCount = 5;
INT nExtCountArray = 0;
ULONG* dExtJointID = NULL;
INT o_nExtJointArrayCount = 0;
DOUBLE* dExtJointArray = NULL;
```

```
m_ptrKit->Path_get_point_external_joint_batch(ulPathID, nStartIndex,nCount,&nExtCountArray, &dExtJo
m_ptrKit->PQAPIFreeArray(((LONG_PTR*)dExtJointID));
m_ptrKit->PQAPIFreeArray(((LONG_PTR*)dExtJointArray));
```

添加单点拍照事件

HRESULT Path_add_shooting_event(ULONG i_uPointID, ULONG i_uExecuteObjID， WCHAR* i_bsEventName,WCHAR * i_bsContent, double i_dTime)	
SDK:10.0.0.7314	
Parameters：	
i_uPointID	轨迹点ID
i_uExecuteObjID	单点拍照事件的执行设备ID
i_bsEventName	事件名字
i_bsContent	事件内容
i_dTime	拍照时间

C++参考代码

```
ULONG uRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"SJPTSROBOT", uRobotID);
ULONG uID = 0;
GetObjIDByName(PQ_PATH, L"轨迹点388", uID);
int nPointCount = 0;
ULONG* ulPointID = nullptr;
m_ptrKit->Path_get_point_id(uID, &nPointCount, &ulPointID);
CComBSTR bsEventName = L"单点拍照事件";
CComBSTR bsContent = L"";
m_ptrKit->Path_add_shooting_event(ulPointID[0], uRobotID, bsEventName, bsContent, 1.0);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

添加摆动事件

HRESULT Path_add_swing_event (ULONG* i_uPointsID, INT i_nPointCount, ULONG i_uExecuteObjID, INT i_nEventPosition, const WCHAR* i_bsEventName, BOOL i_bIsStop, INT i_nSwingJoint, DOUBLE i_SwingAngle, DOUBLE i_SwingValue)	
SDK:10.0.0.7314	
Parameters：	
i_uPointID	轨迹点ID数组
i_nPointCount	轨迹点数组长度
i_uExecuteObjID	事件的执行设备ID
i_nEventPosition	事件插入位置，0为点后，1为点前
i_bsEventName	事件名字
i_bIsStop	是否为停止摆动，True为停止摆动，设置停止摆动时摆动轴与摆动角度设置不生效
i_nSwingJoint	摆动轴设置，0为倒数第二轴摆动，1为倒数第一轴摆动
i_SwingAngle	摆动角度，角度值
i_SwingValue	摆动速度

C++参考代码

```
ULONG uRobotID = 0;
GetObjIDByName(PQ_ROBOT, L"SJPTSROBOT", uRobotID);
ULONG uID = 0;
```

```
GetObjIDByName(PQ_PATH, L"轨迹点388", uID);
int nPointCount = 0;
ULONG* ulPointID = nullptr;
m_ptrKit->Path_get_point_id(uID, &nPointCount, &ulPointID);
CComBSTR bsEventName = L"摆动开始";
m_ptrKit->Path_add_swing_event(ulPointID, nPointCount,  uRobotID,0,  bsEventName,false, 0, 30, 30);
m_ptrKit->PQAPIFreeArray((LONG_PTR*) ulPointsIDs);
```

获取喷涂事件信息

调用Point_get_spray_event_content可以获取指定轨迹点下喷涂事件信息。

HRESULT Point_get_spray_event_content(ULONG i_ulPointID, WCHAR o_wsName, ULONG* o_ulExecuteDeviceID, BOOL* o_bPointForward, WCHAR** o_wsEventTemplate, WCHAR** o_wsEventContent, BOOL* o_bUnion, ULONG** o_ulNozzleIDs, ULONG** o_ulAssistorIDs, INT* o_nulAssistorCount)**	
SDK:11.2.0.7865	
Parameters：	
i_ulPointID	轨迹点ID
o_wsName	事件名称，需要调用PQAPIFreeArray释放内存
o_ulExecuteDeviceID	事件执行设备ID
o_bPointForward	点前、点后执行，1点前、0点后
o_wsEventTemplate	事件模板名称，需要调用PQAPIFreeArray释放内存

HRESULT Point_get_spray_event_content(ULONG i_ulPointID, WCHAR o_wsName, ULONG* o_ulExecuteDeviceID, BOOL* o_bPointForward, WCHAR** o_wsEventTemplate, WCHAR** o_wsEventContent, BOOL* o_bUnion, ULONG** o_ulNozzleIDs, ULONG** o_ulAssistorIDs, INT* o_nulAssistorCount)**	
o_wsEventContent	事件模板内容，需要调用PQAPIFreeArray释放内存
o_bUnion	交集、并集，1并集、0交集
o_ulNozzleIDs	事件关联的喷嘴ID，需要调用PQAPIFreeArray释放内存
o_ulAssistorIDs	事件关联的喷刷ID，需要调用PQAPIFreeArray释放内存
o_nulAssistorCount	事件关联的喷嘴个数
# PQ 常用模块	

以下代码示例均为C#语言示例。

导入三维模型文件

PQ Kit目前支持多种形式的三维模型文件导入方法，可以执行导入模块，弹出文件选择对话框，选择文件导入。也可以调用各种接口导入。

导入三维模型文件模块

执行导入三维模型文件模块C++参考代码

```
wchar_t whMoudle[] = _T("RO_CMD_IMPORT_ACCESSORY_PART");
m_ptrKit->Doc_start_module(whMoudle);
```

导入STEP成零件

无界面导入STEP模型文件并定义为零件，并返回id。

HRESULT Doc_import_part (const WCHAR* i_wsAbsPath, const WCHAR* i_wsFileType, ULONG* o_ulObjId);	
SDK:11.1.0.7308	
Parameters：	
i_wsAbsPath	模型文件的绝对路径
i_wsFileType	模型文件的类型，具体格式详见9.4 3D文件格式匹配符，目前仅支持STEP with colors
o_ulObjId	返回对象的ID

C++参考代码段

```
ULONG ulPartID = 0;
wchar_t whPath[] = L"D:\\Temp\\Model\\发动机叶片.stp";
wchar_t whType[] = _T("STEP with colors");
m_ptrKit->Doc_import_part(whPath, whType, &ulPartID);
```

导入三维模型文件

<1>导入三维模型文件

调用接口Doc_import_accessory_part传入三维模型文件的绝对路径，将该三维模型导入场景，返回生成的场景ID。点云文件的导入参见接口Doc_import_ply_file。

HRESULT Doc_import_accessory_part(const WCHAR* i_whFilePath, BOOL i_bShowProgress, ULONG* o_ulPartID)	
SDK: 11.1.0.7310	
Parameters：	
i_whFilePath	三维模型文件的绝对路径
i_bShowProgress	是否显示进度条
o_ulPartID	返回生成的场景ID

C++参考代码段

```
ULONG ulPartID = 0;
wchar_t whPath[] = L"D:\\Temp\\Model\\发动机叶片.stp";
m_ptrKit->Doc_import_accessory_part(whPath,1,&ulPartID);
```

<2>导入三维模型文件同时设置位置

导入三维模型文件（step、stl等）成为场景，同时设置其基于世界坐标系的位姿，返回导入后场景的ID

HRESULT Doc_import_accessory_part_with_posture(const WCHAR* i_wsAbsPath, const WCHAR* i_wsFileType, DOUBLE* i_dPosture, INT i_nPostureArraySize, INT i_nPostureType, ULONG i_ulCoorID,ULONG* o_ulObjId);	
SDK:10.0.0.7068	
Parameters：	
i_wsAbsPath	模型文件的绝对路径

HRESULT Doc_import_accessory_part_with_posture(const WCHAR* i_wsAbsPath, const WCHAR* i_wsFileType, DOUBLE* i_dPosture, INT i_nPostureArraySize, INT i_nPostureType, ULONG i_ulCoorID,ULONG* o_ulObjId);	
i_wsFileType	模型文件的类型，具体格式详见9.4 3D文件格式匹配符
i_dPosture	模型位姿的数组
i_nPostureArraySize	模型位姿的数组长度
i_nPostureType	指定姿态表示方式：欧拉角、四元素，详见9.2 位姿描述定义
i_ulCoorID	指定传入位姿参考坐标系ID，世界坐标系传0
o_ulObjId	返回对象的ID

C++参考代码段

```
ULONG ulPartID = 0;
wchar_t whPath[] = L"D:\\Temp\\Model\\发动机叶片.stp";
wchar_t whType[] = _T("STEP with colors");
double dPosition[6] = {
989.0,281.0,452.0,0.0,0.0,0.0}
;
```

```
m_ptrKit->Doc_import_accessory_part_with_posture(whPath, whType, dPosition, 6, EULERANGLEXYZ, 0, &u
```

导入点云文件

调用Doc_import_ply_file接口，传入点云文件的绝对路径和类型，将该点云文件导入场景，返回生成的场景ID。

HRESULT Doc_import_ply_file(const WCHAR* i_whFilePath, PQPlyType i_ePlyType, ULONG* o_ulPartID)	
SDK: 11.1.0.7310	
Parameters：	
i_whFilePath	点云文件的绝对路径
i_ePlyType	点云文件的类型
o_ulPartID	返回生成的场景ID

C++参考代码段

```
ULONG ulPartID = 0;
wchar_t whPath[] = L"D:\\Temp\\ct.ply";
m_ptrKit->Doc_import_ply_file(whPath, PLY_POINT_FORMAT, &ulPartID);
```

点云文件类型的定义如下

Value	Description
PLY_POINT_FORMAT	点数据文件
PLY_STANFORD_TRIANGLE_MESH	三角面片数据文件
## 仿真	

调用Doc_start_module传入"RO_CMD_SIMULATE"字符串即可进入仿真状态，调用Doc_end_module接口传入"RO_CMD_SIMULATE"字符串可退出仿真状态。

HRESULT Doc_start_module (const WCHAR* i_whModuleID)	
SDK:10.0.0.7068	
Parameters：	
i_whModuleID	模块ID，"RO_CMD_SIMULATE"

C++参考代码段

```
wchar_t whMoudle[] = L"RO_CMD_SIMULATE";
m_ptrKit->Doc_start_module(whMoudle);
```

生成轨迹

通过Doc_start_module执行"RO_CMD_EXTERNAL_PATH_GENERATE"或者"RO_CMD_GENERATE_PATH"命令均可弹出PQArt轨迹生成面板进行轨迹生成操作，区别在于执行"RO_CMD_EXTERNAL_PATH_GENERATE"命令会有相应的回调函数通知用户轨迹生成状态及结果。

HRESULT Doc_ start _module (const WCHAR* i_whModuleID)	
SDK:10.0.0.7068	
Parameters：	
i_whModuleID	模块ID，" RO_CMD_GENERATE_PATH "

C++参考代码段

```
……
CComBSTR strCMD = " RO_CMD_GENERATE_PATH ";
m_ptrKit->Doc_start_module (strCMD);
```

C#参考代码段

```
private void buttonCreatePath_Click(object sender, EventArgs e) { //生成轨迹 string strCMD = "RO_CM
```

PQKit生成轨迹回调

执行"RO_CMD_EXTERNAL_PATH_GENERATE"命令会触发回调函数，PQKit用户需要实现IPQPlatformComponentCallBack接口上的Fire_Path_Generate_Result方法来接收轨迹生成状态及结果。如一次轨迹生成生成了多条轨迹，则Fire_Path_Generate_Result会被调用多次。

HRESULT Fire_Path_Generate_Result(BOOL i_bSuccess,INT i_nPathCount, INT i_nIndex, ULONG i_ulPathID)	
SDK:10.0.0.7068	
Parameters：	
i_bSuccess	TRUE：轨迹生成成功 FALSE：轨迹生成失败或者取消轨迹生成
i_nPathCount	生成的轨迹总个数
i_nIndex	当前返回的轨迹在总生成个数中是第几个，从0计数
i_ulPathID	轨迹生成结果中i_nIndex序号对应的轨迹ID

C++参考代码段

```
CComBSTR cmd = "RO_CMD_EXTERNAL_PATH_GENERATE";
m_ptrKit->Doc_start_module(cmd);
```

测量

HRESULT Doc_ start _module (const WCHAR* i_whModuleID)	
SDK:10.0.0.7068	
Parameters：	
i_whModuleID	模块 ID，"RO_CMD_MEASUREMENT"

C++参考代码段

```
……
CComBSTR strCMD = "RO_CMD_MEASUREMENT";
m_ptrKit->Doc_ start _module (strCMD);
```

C#参考代码段

```
private void Measure_Click(object sender, EventArgs e) { //测量 string strCMD = "RO_CMD_MEASUREMENT
```

后置

无界面后置

PQKit提供了3个接口以供开发者实现无界面的后置。

1. Robot_post_file

调用Robot_post_file接口，传入参数，即可将指定机器人的轨迹点后置出来。

HRESULT Robot_post_file(ULONG i_ulRobotID, const WCHAR* i_whPostFilePath, const WCHAR* i_whPostFileName, const WCHAR* i_whUseTCP, PQPostOption* i_stPostOption)	
SDK:10.0.0.7068	
Parameters：	

HRESULT Robot_post_file(ULONG i_ulRobotID, const WCHAR* i_whPostFilePath, const WCHAR* i_whPostFileName, const WCHAR* i_whUseTCP, PQPostOption* i_stPostOption)	
i_ulRobotID	欲执行后置操作的机器人ID
i_whPostFilePath	后置文件路径
i_whPostFileName	后置文件名称
i_whUseTCP	使用的TCP名称，可传空，传空默认使用机器人当前使用的工具当前使用的TCP
i_stPostOption	后置参数结构体

后置参数结构体参数含义如下：

Parameters：	
nFirstPtIndex	轨迹点命名，第一个点编号
nOffsetDir	轨迹点偏移，偏移方向，0:X轴 1:Y轴 2:Z轴
nOffsetData	轨迹点偏移，偏移数据
nPtDataType	表明基于哪个对象输出点位，0:机器人末端, 1:工具末端, 2:关节空间
nPtCoor	后置使用的坐标系, 0:轨迹坐标系 1:机器人基坐标系
nLineIndex	行号起始
nOutputIO	是否输出IO，1输出
nUseComment	是否使用注释，1使用
nIgnorePathState	是否忽略轨迹错误，1忽略

后置参数结构体对应的是PQArt软件中后置参数对话框中内容。

PQPostOption结构体对象可使用接口Kit_init_post_option_struct进行初始化。

C++参考代码段

```
ULONG uRobotID = 0;
GetObjIDByName(32, _T("STAUBLI-TX200"), uRobotID);
struct_PQPostOption stPostOp;
m_ptrKit->Kit_init_post_option_struct(&stPostOp);
wchar_t whPath[] = L"E:\\Delete\\";
wchar_t whName[] = L"JustName";
wchar_t whTCPName[] = L"";
m_ptrKit->Robot_post_file(uRobotID, whPath, whName, whTCPName, &stPostOp);
```

1. Path_post_file

调用Path_post_file接口，传入参数，即可将指定机器人的轨迹点后置出来。

HRESULT Path_post_file (ULONG* i_ulPathIDs, INT i_nPathCount, const WCHAR* i_whPostFilePath, const WCHAR* i_whPostFileName, const WCHAR* i_whUseTCP, PQPostOption* i_stPostOption)	
SDK:10.0.0.7068	
Parameters：	
i_ulPathIDs	欲执行后置操作的轨迹ID数组，注意，只能后置同一个机器人下的轨迹
i_nPathCount	轨迹数组的个数
i_whPostFilePath	后置文件路径
i_whPostFileName	后置文件名称
i_whUseTCP	使用的TCP名称，可传空，传空默认使用机器人当前使用的工具当前使用的TCP
i_stPostOption	后置参数结构体

后置参数结构体参数含义如下：

Parameters：	
nFirstPtIndex	轨迹点命名，第一个点编号
nOffsetDir	轨迹点偏移，偏移方向，0:X轴 1:Y轴 2:Z轴
nOffsetData	轨迹点偏移，偏移数据
nPtDataType	表明基于哪个对象输出点位，0:机器人末端, 1:工具末端, 2:关节空间
nPtCoor	后置使用的坐标系, 0:轨迹坐标系 1:机器人基坐标系
nLineIndex	行号起始
nOutputIO	是否输出IO，1输出
nUseComment	是否使用注释，1使用
nIgnorePathState	是否忽略轨迹错误，1忽略

后置参数结构体对应的是PQArt软件中后置参数对话框中内容。

PQPostOption结构体对象可使用接口Kit_init_post_option_struct进行初始化。

C++参考代码段

```
ULONG uPathIDs[1] = {
0}
;
GetObjIDByName(32, _T("STAUBLI-TX200"), uRobotID);
struct_PQPostOption stPostOp;
m_ptrKit->Kit_init_post_option_struct(&stPostOp);
wchar_t whPath[] = L"E:\\Delete\\";
wchar_t whName[] = L"JustName";
wchar_t whTCPName[] = L"";
m_ptrKit->Robot_post_file(uPathIDs,1, whPath, whName, whTCPName, &stPostOp);
```

2、指定后置文件绝对路径，实现无界面后置。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:10.0.0.7068	
Parameters：	
bsCommandID	"RO_CMD_POST"
wParam	0或NULL
lParam	0或NULL
bsParam	""
varParam	指定后置文件的绝对路径，多文件后置如KUKA机器人后置，指定src、dat之一即可
lResult	0

C++参考代码段

```
CComBSTR strCMD = "RO_CMD_POST";
CComBSTR strParam = "";
std::string strFilePath = "";
CComVariant varParam(strFilePath.c_str());
long long lResult = 0;
m_ptrKit->pq_RunCommand(strCMD, NULL, NULL, strParam, varParam, &lResult);
```

C#参考代码段

```
private void post_without_ui(object sender, EventArgs e) { string strCMD = "RO_CMD_POST"; string st
```

轨迹组后置

传入指定轨迹组名称和后置文件的绝对路径，实现轨迹组的无界面后置。

HRESULT pq_RunCommand(BSTR bsCommandID, ULONGLONG wParam, LONGLONG lParam, BSTR bsParam, VARIANT varParam, LONGLONG* lResult)	
SDK:10.0.0.7068	
Parameters：	
bsCommandID	"RO_CMD_SELECT_POST"
wParam	0或NULL
lParam	0或NULL
bsParam	指定的轨迹组名称
varParam	指定后置文件的绝对路径，多文件后置如KUKA机器人后置，指定src、dat之一即可
lResult	0

C++参考代码段

```
CComBSTR strCMD = "RO_CMD_SELECT_POST";
CComBSTR strParam = "";
std::string strFilePath = "";
CComVariant varParam(strFilePath.c_str());
long long lResult = 0;
m_ptrKit->pq_RunCommand(strCMD, NULL, NULL, strParam, varParam, &lResult);
```

替换后置文件

传入需要替换后置文件的绝对路径，实现后置文件替换。

HRESULT Robot_set_custom_post(ULONG i_ulRobotID, const WCHAR* i_whPostFile)	
SDK:10.0.0.7068	
Parameters：	

HRESULT Robot_set_custom_post(ULONG i_ulRobotID, const WCHAR* i_whPostFile)	
i_ulRobotID	需要替换后置文件的机器人ID
i_whPostFile	替换后置文件的绝对路径

C++参考代码段

```
ULONG ulRobotID = 0;
WCHAR* whPostFile;
m_ptrKit->Robot_set_custom_post(ulRobotID, whPostFile);
```

拾取实体元素

调用Doc_start_pickup_element，PQKit可进入实体元素拾取状态，即可通过鼠标单击绘图区实体元素上的点、线、面进行拾取，根据传参设置拾取指定类型。通过7.6.1回调查看拾取结果

注意：结束拾取后需要使用Doc_stop_pickup_element结束拾取状态，拾取状态中无法执行其他命令；

开始拾取

HRESULT Doc_start_pickup_element(const PQPickupType i_eSelectType)	
SDK:10.0.0.7068	
Parameters：	
i_eSelectType	指定拾取类型

C++参考代码段

```
PQPickupType ePickupType = PQ_SELECT_ANY;
m_ptrKit->Doc_start_pickup_element(ePickupType);
```

结束拾取

HRESULT Doc_stop_pickup_element (const PQPickupType i_eSelectType)

Parameters：

PQKit拾取实体回调

执行"RO_CMD_PICKUP_ELEMENT"模块会触发拾取回调函数，PQKit用户需要实现IPQPlatformComponentCallBack接口上的Fire_Element_Pickup方法来接收实体拾取结果。

HRESULT Fire_Element_Pickup(ULONG i_ulObjID, const WCHAR* i_wsEntityName, INT i_nEntityType, double i_dPointX, double i_dPointY, double i_dPointZ)	
SDK:10.0.0.7068	
Parameters：	
i_ulObjID	选中实体对应PQKit的对象ID
i_wsEntityName	选中实体的ID
i_nEntittType	选中实体的类型，具体值为： 面 -2 边/线 -4 点 -8
i_dPointX	拾取点世界坐标系下x值
i_dPointY	拾取点世界坐标系下y值
i_dPointZ	拾取点世界坐标系下z值

C++参考代码段

```
ULONG ulObjID = 0;
WCHAR* wsEntityName;
,
```

```
INT nEntityType = -2;
m_ptrKitCallback->Fire_Element_Pickup(ulObjID, wsEntityName, nEntityType, i_dPointX,       i_dPoint
```

PQkit命令ID

编程仿真常用命令

功能名称	命令ID
打开文件	RO_CMD_FILE_OPEN
保存	RO_CMD_FILE_SAVE
另存为	RO_CMD_FILE_SAVE_AS
轻量化	RO_CMD_FILE_LITE
三位球	RO_CMD_ARCBALL_PROXY
输入	RO_CMD_IMPORT_ACCESSORY_PART
测量	RO_CMD_MEASUREMENT
新建坐标系	RO_CMD_CREATE_FRAME
生成轨迹	RO_CMD_GENERATE_PATH
贴图	RO_CMD_START_CHARTLET
AGV路径规划	RO_CMD_AGV_PATH
导入轨迹	RO_CMD_IMPORT_PATH
新建轨迹	RO_CMD_ROCREATEEMPTYPATH
编译	RO_CMD_COMPILE
后置	RO_CMD_POST
导入程序	RO_CMD_KUKA_IMPORT_MORE_CODE
控制器仿真	RO_CMD_CONTROLLER_SIM
输出动画	RO_CMD_EXPORT_ANIMATION

功能名称	命令ID
输出视频	RO_CMD_EXPORT_VIDEO
性能分析	RO_CMD_GIRD
碰撞设置	RO_CMD_COLLISION_OPTION
获取数据	RO_CMD_GETROBOTJOINTS
数据分析	RO_CMD_ANALYZEJOINTSDATA
选项	RO_CMD_MOTION_OPTION
### 定义组件命令	

定义机构	RO_CMD_DEFINE_MECHANISM
导入自定义机构	RO_CMD_DEFROBOT_SETUP
定义工具	RO_CMD_CREATE_TOOL
导入工具	RO_CMD_IMPORT_TOOL
定义零件	RO_CMD_CREATE_WORKINGPART
导入零件	RO_CMD_IMPORT_PART
定义底座	RO_CMD_CREATE_SEAT
导入底座	RO_CMD_IMPORT_SEAT
定义状态机	RO_CMD_DEFINE_MECHSTATE
导入状态机	RO_CMD_IMPORT_MECHSTATE
定义效果	RO_CMD_DEFINE_EFFECT
管理效果	RO_CMD_EFFECT_MGR
自定义后置	RO_CMD_ROGENCUSPOSTFORMAT
### 校准命令	

测量	RO_CMD_MEASUREMENT
三点校准	RO_CMD_AlignPart3Point
多点校准	RO_CMD_AlignPart5Point
点轴校准	RO_CMD_AlignPartPointAxis
三面校准	RO_CMD_AlignPart3Plane
坐标系标定	
新建坐标系	RO_CMD_CREATE_FRAME
计算方向	RO_CMD_CAL_COODE_DIRECTION
反推坐标系	RO_CMD_CREAT_BASEFRAME
### 自由设计命令	

创建草图	RO_CMD_CREATE_NEWSKETCH
创建点	RO_CMD_SKETCHER_CREATEPOINT
创建一条直线	RO_CMD_SKETCHER_LINE
创建一条圆弧	RO_CMD_SKETCHER_ARC_1
创建一个整圆	RO_CMD_SKETCHER_CIRC_1
创建椭圆弧	RO_CMD_SKETCHER_ELLIPTICALARC
创建椭圆	RO_CMD_SKETCHER_ELLIPSE
创建矩形	RO_CMD_SKETCHER_RECTANGLE
创建正多边形	RO_CMD_SKETCHER_REGPOLYGON
键槽	RO_CMD_SKETCHER_SLOT
创建多段线	RO_CMD_SKETCHER_POLYLINE
圆角	RO_CMD_SKETCHER_FILLET
创建文字	RO_CMD_SKETCHER_CREATETEXT

创建草图	RO_CMD_CREATE_NEWSKETCH
裁剪	RO_CMD_SKETCHER_TRIMING
封闭图形	RO_CMD_SKETCHER_CLOSESHAPE
投影	RO_CMD_SKETCHER_PROJECTION

程序编辑	
同步	RO_CMD_EDITOR_SYNC_SHELL
编辑	RO_CMD_EDITOR_EDIT
显示	
管理树	RO_CMD_SHOW_MANAGE_ROBOT_TREE
控制面板	RO_CMD_SHOW_ROBOT_CONTRL
显示全部	RO_CMD_SHOW_ALL_HIDE_OBJ
地面	RO_CMD_FLOOR
显示时序图	RO_CMD_SHOW_SEQUCENE
### 其他命令	
## 框选实体元素	

调用Doc_start_module接口传入模块ID：RO_CMD_SELECTION_ELEMENT，PQKit可进入实体元素拾取状态，即可通过鼠标单击绘图区实体元素上的多 个点，形成框选区域，从而获取框选区域内点坐标以及点、线、面的ID。

HRESULT Doc_start_module(const WCHAR* i_whModuleID)	
SDK:10.0.0.7068	
Parameters：	
i_whModuleID	RO_CMD_SELECTION_ELEMENT

C++参考代码段

```
CComBSTR cmd = "RO_CMD_SELECTION_ELEMENT";
m_ptrKit->Doc_start_module(cmd);
```

PQKit框选回调

执行" RO_CMD_SELECTION_ELEMENT "模块会触发拾取回调函数，PQKit用户需要实现IPQPlatformComponentCallBack接口上的Fire_Element_Selection方法来接收实体拾取结果。

HRESULT Fire_Element_Selection(LPWSTR i_wObjNames, LPWSTR i_wFaceNames, double* i_dPointXYZ, int i_nSize)	
SDK:10.0.0.7068	
Parameters：	
i_wObjNames	选中对应点线面的对象名字，用"#"拼接
i_wFaceNames	选中实体的ID，并用"#"拼接
i_dPointXYZ	框选点的坐标
i_nSize	框选点个数
# PQ 回调函数	

用户需要实现IPQPlatformComponentCallBack接口中的对应函数实现回调相应功能。（必须实现，否则编译出错。）

PQKit初始化回调

通知PQKit初始化结果。

void Fire_Initialize_Result(int lResult)	
Parameters：	
lResult	Kit初始化状态返回值
返回值状态如下：	

void Fire_Initialize_Result(int lResult)	
1	初始化成功
-1	一般性错误
-2	初始化超时
-3、-4、-6	加载组件动态库错误
-5	用户校验失败
-7	组件重复初始化
-9、-10、-11、-12	内部数据错误

C++参考代码段

```
int nResult =1 ;
m_ptrKitCallback->Fire_Initialize_Result(nResult);
```

用户登陆回调

通知PQKit登录结果。

void Fire_Login_Result (int lResult)	
SDK:10.0.0.7068	
Parameters：	
lResult	Kit登录状态返回值
返回值状态如下：	
1	登录成功
-1	密码错误
-2	用户不存在
-4	用户已过期

void Fire_Login_Result (int lResult)	
-6	网络连接失败
-7	用户未激活
-14	账号不匹配

C++参考代码段

```
int lResult = 1;
m_ptrKitCallback->Fire_Login_Result(lResult);
```

置顶窗口回调

PQ Kit 中机器人加工管理树、调试窗口、输出窗口、机器人控制窗口在浮动模式，是由用户控制显示的，当PQKit内部需要置顶这4个窗口时，会通过Notify_Raise_Dockwindow回调予以通知。

HRESULT Notify_Raise_Dockwindow(int i_nType)	
SDK:10.0.0.7068	
Parameters：	
i_nType	Dock类型
1	机器人加工管理树
4	机器人控制窗口
7	调试窗口
8	输出窗口

C++参考代码段

```
int nType = 1;
m_ptrKitCallback->Notify_Raise_Dockwindow(nType);
```

生成轨迹回调

详见7.3.1。

拾取实体回调

详见7.6.1。

绘图区鼠标右键回调

HRESULT Fire_RButton_Up(LONG i_lPosX,LONG i_lPosY)	
SDK:10.0.0.7068	
Parameters：	
i_lPosX	鼠标右键的x值（相对于绘图区世界坐标系原点）
i_lPosY	鼠标右键的y值（相对于绘图区世界坐标系原点）

C++参考代码段

```
LONG lPosX = 1;
LONG lPosY = 1;
m_ptrKitCallback->Fire_RButton_Up(lPosX, lPosY);
```

绘图区鼠标左键回调

HRESULT Fire_LButton_Up(LONG i_lPosX, LONG i_lPosY)	
SDK:10.0.0.7068	
Parameters：	
i_lPosX	鼠标左键的x值（相对于绘图区世界坐标系原点）

HRESULT Fire_LButton_Up(LONG i_lPosX, LONG i_lPosY)	
i_lPosY	鼠标左键的y值（相对于绘图区世界坐标系原点）

C++参考代码段

```
LONG lPosX = 1;
LONG lPosY = 1;
m_ptrKitCallback->Fire_LButton_Up(lPosX, lPosY);
```

绘图区右键菜单回调

PQKit在绘图区中弹出菜单之前会通过此回调函数告知用户，如用户需要实现自己的右键菜单，则将o_nHandled置1，并自行处理弹出菜单的逻辑部分。如用户无需弹出自己的右键菜单，将o_nHandled置0即可。

HRESULT Fire_Menu_Pop(ULONG i_ulObjID, LONG i_lPosX, LONG i_lPosY, INT *o_nHandled)	
Parameters：	
i_ulObjID	选中物体ID
i_lPosX	鼠标右键的y值（屏幕坐标系下的值）
i_lPosY	鼠标右键的y值（屏幕坐标系下的值）
o_nHandled	传出参数，需要在回调函数中给o_nHandled赋值，如弹出 自己的右键菜单则o_nHandled置1，不处理该右键菜单事件则置0，继续弹出PQKit菜单

C++参考代码段

```
ULONG ulObjID = 0;
LONG lPosX = 0;
```

```
LONG lPosY = 0;
INT o_nHandled;
m_ptrKitCallback->Fire_Menu_Pop(ulObjID, lPosX, lPosY, &o_nHandled);
```

拾取状态下绘制回调

在拾取状态下（即执行7.6）鼠标在目标物体移动时，会触发Fire_Draw回调，拾取状态下对绘图区的绘制需求均需要在此回调函数中实现。

HRESULT Fire_Draw()

SDK:10.0.0.7068

Parameters：

如在绘图区画点，参考1.36。

删除对象回调

在PQKit中删除对象时会触发Fire_Object_Deleted回调，告知被删除对象的ID、名称、类型。

HRESULT Fire_Object_Deleted(ULONG i_ulObjID, const WCHAR* i_wObjName, PQDataType i_eObjType)	
SDK:11.2.0.7732	
Parameters：	
i_ulObjID	被删除物体的ID
i_wObjName	被删除物体的名称
i_eObjType	被删除物体的类型
# PQ 常用数据术语	
## 数据类型定义	

ObjType	ID
机器人	PQ_ROBOT
工件	PQ_WORKINGPART
工具	PQ_TOOL
底座	PQ_SEAT
轨迹	PQ_PATH
轨迹点	PQ_POINT
场景文件	PQ_ACCESSORY_PART
工作单元	PQ_WORKCELL
状态机	PQ_MECHSTATE
坐标系	PQ_COORD
轨迹文件夹	PQ_PATH_FOLDER
## 位姿描述定义	

欧拉角数据中的旋转角数据均为度数单位

Description	Value
四元素	QUATERNION
欧拉角XYZ	EULERANGLEXYZ
欧拉角ZYX	EULERANGLEZYX
欧拉角ZXZ	EULERANGLEZXZ
欧拉角ZYZ	EULERANGLEZYZ
## 轨迹点指令描述定义	

Description	Value
直线	PQ_LINE

Description	Value
圆弧	PQ_CIRCLE
样条	PQ_SPLINE
关节角	PQ_JOINT
AbsJ	PQ_ABSJOINT
## 3D文件格式匹配符	

Description	Value
Alias Mesh	*.obj
BREP format	*.brep; *.brp
Binary Mesh	*.bms
IGES format	*.iges; *.igs
Inventor V2.1	*.iv
Jupiter Tessellation V8.0-V9.5	*.jt
Object File Format Mesh	*.off
Point formats	*.asc; *.pcd; *.ply
PQ 3D document	*.pq3d
STEP with colors	*.step; *.stp
STL Mesh	*.stl; *.ast
Stanford Triangle Mesh	*.ply
VRML V2.0	*.wrl; *.vrml; *.wrz; *.wrl.gz
## 轨迹逆运动学求解方式定义	

Description	Value
常规	PQ_ROINVERSETYPE_STATION

Description	Value
联动(位置优先)	PQ_ROINVERSETYPE_EXTERNALAXLE_POS_PRIORITY
联动(自动连接)	PQ_ROINVERSETYPE_EXTERNALAXLE_AS_MECH
联动(角度优先)	PQ_ROINVERSETYPE_EXTERNALAXLE_ANGLE_PRIORITY
联动(Z轴向上)	PQ_ROINVERSETYPE_EXTERNALAXLE_FIXED_DIR
## 轨迹可达性状态定义	

Description	Value
可达	PQ_PPS_BE_REACH
不可达	PQ_PPS_BE_NOTREACH
轴超限	PQ_PPS_BE_OUTRANGE
奇异点	PQ_PPS_BE_SINGULAR
未知	PQ_PPS_BE_UNKNOW
轴配置变化	PQ_PPS_BE_CONFIGDIFF
碰撞	PQ_PPS_BE_COLLISION
## 轴配置定义	

Description	Value
6R	
1轴配置 PQEFrontOrBack	向前（PQ_INVERSE_FRONT）、向后（PQ_INVERSE_BACK）
2轴配置 PQEUpOrDown	向上（PQ_INVERSE_UP）、向下（传 PQ_INVERSE_DWON）
5轴配置 PQEFlipOrNoFlip	不翻转（PQ_INVERSE_NOFLIP）、翻转（PQ_INVERSE_FLIP）

Description	Value
1轴角度 PQEEndRange	自动（PQ_INVERSE_CONTINUATION） 小于等于-180（PQ_INVERSE_SMALL） -180 与 180 之间（PQ_INVERSE_MIDDLE） 大于等于 180（PQ_INVERSE_LARGE）
4轴角度 PQEEndRange	自动（PQ_INVERSE_CONTINUATION） 小于等于-180（PQ_INVERSE_SMALL） -180 与 180 之间（PQ_INVERSE_MIDDLE） 大于等于 180（PQ_INVERSE_LARGE）
6轴 角度 PQEEndRange	自动（PQ_INVERSE_CONTINUATION） 小于等于-180（PQ_INVERSE_SMALL） -180 与 180 之间（PQ_INVERSE_MIDDLE） 大于等于 180（PQ_INVERSE_LARGE）
3R1P	
左手系 PQESCaRaConfig	PQ_ESCaRaConfig_leftHand
右手系 PQESCaRaConfig	PQ_ESCaRaConfig_rightHand
## 坐标系类型	

Value	Description
PQ_COORINATE_WORLD	世界坐标系，接口中需要指定坐标系的地方，如需指定基于世界坐标系，传入0即可
PQ_COORINATE_USER	用户新建的坐标系
PQ_COORINATE_ROBOTBASE	机器人基坐标系
PQ_COORINATE_WKT	零件自身的坐标系
## 机器人类型	

Value	Description
PQ_Robot	机器人
PQ_Guide	导轨

Value	Description
PQ_Positioner	变位机
PQ_COORINATE_WKT	零件自身的坐标系
## 边几何类型	

Value	Description
PQ_CURVE_UNKNOWN	未知
PQ_CURVE_STRAIGNT	直线
PQ_CURVE_CIRCLE	圆弧
PQ_CURVE_SPLINE	样条曲线
## 面几何类型	

Value	Description
PQ_RACE_GENERAL	一般、普通
PQ_RACE_PLANE	平面
PQ_RACE_CYLINDER	圆柱面
PQ_RACE_CONE	圆锥面
PQ_RACE_REVOLUTION	旋转面
# PQUtility	

PQUtility中包含了一些PQKit所需方法集合。C++开发环境中创建IPQComponentUtility与创建IPQPlatformComponent类似，C#开发环境中通过LoadPQKitUtility方法创建IPQComponentUtility对象。

获取硬件锁内信息

PQKit用户可通过PQKitLicGenerator.exe（PQKit开发版安装根目录下）来向PQKit硬件锁内写入用户自己的授权信息，并通过License_get_hardkey_info方法获取用户写入硬件锁内的信息。

HRESULT License_get_hardkey_info(WCHAR o_wsKeyInfo, WCHAR** o_wsStatusMsg)**	
Parameters：	
o_wsKeyInfo	锁内用户自写入信息字符串,字符串需要调用PQAPIFreeArray释放
o_wsStatusMsg	读取锁内信息过程中的状态信息，状态信息需要调用PQAPIFreeArray释放

C++参考代码段

```
void GetHardkeyInfo() {
WCHAR* wsKeyInfo= nullptr;
WCHAR* wsStatusMsg= nullptr;
m_ptrKitUtility->License_get_hardkey_info(&wsKeyInfo, &wsStatusMsg);
m_ptrKitUtility->PQAPIFreeArray((LONG_PTR*)wsKeyInfo);
m_ptrKitUtility->PQAPIFreeArray ((LONG_PTR*)wsStatusMsg);
}
```

多语言

PQKit支持多语言安装切换。目前PQKit通过打补丁来实现多语言，即先安装中文开发版后再安装对应语言的补丁包即可。PQKit提供了App_set_language方法设置语言。

HRESULT App_set_language(const WCHAR* i_chLang)	
Parameters：	
i_chLang	语言ID字符串

C++参考代码段

```
void SetLanguage() {
wchar_t whLanguage[256] = L"en-US";
ptrKitUtility->App_set_language(whLanguage);
}
```

PQKit支持的语言ID如下

语言	ID字符串
中文	zh-CN
英语	en-US
中文繁体	zh-TW
俄语	ru-RU
捷克语	cs-CZ
日语	ja-JP
西班牙语	es-ES
## 版本信息	

调用App_get_version接口可以获取Kit SDK的版本信息字符串，如10.0.0.7241 (x64)。

HRESULT App_get_version (WCHAR** o_wsVersion)	
Parameters：	
o_wsVersion	SDK版本字符串，需要调用PQAPIFreeArray释放

C++调用示例：

```
void GetVersion() {
CComPtr<IPQComponentUtility>
ptrKitUtility;
HRESULT hr =
ptrKitUtility.CoCreateInstance(__uuidof(PQComponentUtility));
WCHAR* whVersion = nullptr;
ptrKitUtility->App_get_version(&whVersion);
std::wstring t_wsVersion = whVersion;
ptrKitUtility->PQAPIFreeArray((LONG_PTR*)whVersion);
```

```
}
```